

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
```

```
In [ ]: df=pd.read_csv("/content/Dataset - HR Employee Attrition.csv")
```

```
In [ ]: df
```

```
Out[ ]:
```

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	Distance
0	1	Yes	41	Travel_Rarely	1102	Sales	
1	2	No	49	Travel_Frequently	279	Research & Development	
2	3	Yes	37	Travel_Rarely	1373	Research & Development	
3	4	No	33	Travel_Frequently	1392	Research & Development	
4	5	No	27	Travel_Rarely	591	Research & Development	
...	...	...	...	...	...	...	...
2935	2936	No	36	Travel_Frequently	884	Research & Development	
2936	2937	No	39	Travel_Rarely	613	Research & Development	
2937	2938	No	27	Travel_Rarely	155	Research & Development	
2938	2939	No	49	Travel_Frequently	1023	Sales	
2939	2940	No	34	Travel_Rarely	628	Research & Development	

2940 rows × 35 columns



```
In [ ]: df.shape
```

```
Out[ ]: (2940, 35)
```

```
In [ ]: #showing first five rows
df.head()
```

Out[]:

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFro
0	1	Yes	41	Travel_Rarely	1102	Sales	
1	2	No	49	Travel_Frequently	279	Research & Development	
2	3	Yes	37	Travel_Rarely	1373	Research & Development	
3	4	No	33	Travel_Frequently	1392	Research & Development	
4	5	No	27	Travel_Rarely	591	Research & Development	

5 rows × 35 columns

In []: *#last five rows*
df.tail()

Out[]:

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	Distance
2935	2936	No	36	Travel_Frequently	884	Research & Development	
2936	2937	No	39	Travel_Rarely	613	Research & Development	
2937	2938	No	27	Travel_Rarely	155	Research & Development	
2938	2939	No	49	Travel_Frequently	1023	Sales	
2939	2940	No	34	Travel_Rarely	628	Research & Development	

5 rows × 35 columns

In []: *#finding out the missing value*
df.isna().sum()

```
Out[ ]: EmployeeNumber      0
        Attrition           0
        Age                 0
        BusinessTravel      0
        DailyRate           0
        Department          0
        DistanceFromHome    0
        Education            0
        EducationField       0
        EmployeeCount        0
        EnvironmentSatisfaction 0
        Gender               0
        HourlyRate           0
        JobInvolvement       0
        JobLevel             0
        JobRole              0
        JobSatisfaction      0
        MaritalStatus        0
        MonthlyIncome        0
        MonthlyRate          0
        NumCompaniesWorked   0
        Over18               0
        OverTime             0
        PercentSalaryHike    0
        PerformanceRating    0
        RelationshipSatisfaction 0
        StandardHours        0
        StockOptionLevel     0
        TotalWorkingYears    0
        TrainingTimesLastYear 0
        WorkLifeBalance      0
        YearsAtCompany       0
        YearsInCurrentRole   0
        YearsSinceLastPromotion 0
        YearsWithCurrManager 0
        dtype: int64
```

```
In [ ]: df.shape
```

```
Out[ ]: (2940, 35)
```

```
In [ ]: df
```

Out[]:	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	Distance
0	1	Yes	41	Travel_Rarely	1102	Sales	
1	2	No	49	Travel_Frequently	279	Research & Development	
2	3	Yes	37	Travel_Rarely	1373	Research & Development	
3	4	No	33	Travel_Frequently	1392	Research & Development	
4	5	No	27	Travel_Rarely	591	Research & Development	
...	...	...	...	...	...	...	...
2935	2936	No	36	Travel_Frequently	884	Research & Development	
2936	2937	No	39	Travel_Rarely	613	Research & Development	
2937	2938	No	27	Travel_Rarely	155	Research & Development	
2938	2939	No	49	Travel_Frequently	1023	Sales	
2939	2940	No	34	Travel_Rarely	628	Research & Development	

2940 rows × 35 columns



```
In [ ]: df=pd.get_dummies(df).astype(int)
```

```
In [ ]: df
```

Out[]:

	EmployeeNumber	Age	DailyRate	DistanceFromHome	Education	EmployeeCount
0	1	41	1102	1	2	1
1	2	49	279	8	1	1
2	3	37	1373	2	2	1
3	4	33	1392	3	4	1
4	5	27	591	2	1	1
...	...	...	...	...	...	...
2935	2936	36	884	23	2	1
2936	2937	39	613	6	1	1
2937	2938	27	155	4	3	1
2938	2939	49	1023	2	3	1
2939	2940	34	628	8	3	1

2940 rows × 57 columns



In []: `df.shape`

Out[]: (2940, 57)

In []: `#Label encoding`
`df=pd.read_csv("/content/Dataset - HR Employee Attrition.csv")`

In []: `df.head()`

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFro
0	1	Yes	41	Travel_Rarely	1102	Sales	
1	2	No	49	Travel_Frequently	279	Research & Development	
2	3	Yes	37	Travel_Rarely	1373	Research & Development	
3	4	No	33	Travel_Frequently	1392	Research & Development	
4	5	No	27	Travel_Rarely	591	Research & Development	

5 rows × 35 columns



```
In [ ]: #df.Attrition.unique()
```

```
In [ ]: from sklearn.preprocessing import LabelEncoder  
le=LabelEncoder()
```

```
In [ ]: #df['Attrition']=le.fit_transform(df['Attrition'])
```

```
In [ ]: df
```

```
Out[ ]:
```

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	Distance
0	1	Yes	41	Travel_Rarely	1102	Sales	
1	2	No	49	Travel_Frequently	279	Research & Development	
2	3	Yes	37	Travel_Rarely	1373	Research & Development	
3	4	No	33	Travel_Frequently	1392	Research & Development	
4	5	No	27	Travel_Rarely	591	Research & Development	
...	...	...	...	...	...	...	...
2935	2936	No	36	Travel_Frequently	884	Research & Development	
2936	2937	No	39	Travel_Rarely	613	Research & Development	
2937	2938	No	27	Travel_Rarely	155	Research & Development	
2938	2939	No	49	Travel_Frequently	1023	Sales	
2939	2940	No	34	Travel_Rarely	628	Research & Development	

2940 rows × 35 columns



```
In [ ]: df.Attrition.unique()
```

```
Out[ ]: array(['Yes', 'No'], dtype=object)
```

```
In [ ]: df.JobRole.unique()
```

```
Out[ ]: array(['Sales Executive', 'Research Scientist', 'Laboratory Technician',  
             'Manufacturing Director', 'Healthcare Representative', 'Manager',  
             'Sales Representative', 'Research Director', 'Human Resources'],  
           dtype=object)
```

```
In [ ]: df.BusinessTravel.unique()
```

```
Out[ ]: array(['Travel_Rarely', 'Travel_Frequently', 'Non-Travel'], dtype=object)
```

```
In [ ]: df.Department.unique()
```

```
Out[ ]: array(['Sales', 'Research & Development', 'Human Resources'], dtype=object)
```

```
In [ ]: df.EducationField.unique()
```

```
Out[ ]: array(['Life Sciences', 'Other', 'Medical', 'Marketing',  
              'Technical Degree', 'Human Resources'], dtype=object)
```

```
In [ ]: #df.Gender.unique()
```

```
In [ ]: df.MaritalStatus.unique()
```

```
Out[ ]: array(['Single', 'Married', 'Divorced'], dtype=object)
```

```
In [ ]: df.Over18.unique()
```

```
Out[ ]: array(['Y'], dtype=object)
```

```
In [ ]: df.OverTime.unique()
```

```
Out[ ]: array(['Yes', 'No'], dtype=object)
```

```
In [ ]: from sklearn.preprocessing import LabelEncoder  
le=LabelEncoder()
```

```
In [ ]: df['BusinessTravel']=le.fit_transform(df['BusinessTravel'])
```

```
In [ ]: df['Department']=le.fit_transform(df['Department'])
```

```
In [ ]: df['EducationField']=le.fit_transform(df['EducationField'])
```

```
In [ ]: #df['Gender']=le.fit_transform(df['Gender'])
```

```
In [ ]: df['JobRole']=le.fit_transform(df['JobRole'])
```

```
In [ ]: df['MaritalStatus']=le.fit_transform(df['MaritalStatus'])
```

```
In [ ]: df['Over18']=le.fit_transform(df['Over18'])
```

```
In [ ]: df['OverTime']=le.fit_transform(df['OverTime'])
```

```
In [ ]: df
```

Out[]:	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFr
0	1	Yes	41	2	1102	2	
1	2	No	49	1	279	1	
2	3	Yes	37	2	1373	1	
3	4	No	33	1	1392	1	
4	5	No	27	2	591	1	
...	...	...	...	...	...	...	...
2935	2936	No	36	1	884	1	
2936	2937	No	39	2	613	1	
2937	2938	No	27	2	155	1	
2938	2939	No	49	1	1023	2	
2939	2940	No	34	2	628	1	

2940 rows × 35 columns



In []: `df.isna().sum()`

```
Out[ ]: EmployeeNumber      0
        Attrition           0
        Age                 0
        BusinessTravel      0
        DailyRate           0
        Department          0
        DistanceFromHome    0
        Education            0
        EducationField       0
        EmployeeCount        0
        EnvironmentSatisfaction 0
        Gender               0
        HourlyRate           0
        JobInvolvement       0
        JobLevel             0
        JobRole              0
        JobSatisfaction      0
        MaritalStatus        0
        MonthlyIncome        0
        MonthlyRate          0
        NumCompaniesWorked   0
        Over18               0
        OverTime             0
        PercentSalaryHike    0
        PerformanceRating    0
        RelationshipSatisfaction 0
        StandardHours        0
        StockOptionLevel     0
        TotalWorkingYears    0
        TrainingTimesLastYear 0
        WorkLifeBalance      0
        YearsAtCompany       0
        YearsInCurrentRole   0
        YearsSinceLastPromotion 0
        YearsWithCurrManager 0
        dtype: int64
```

```
In [ ]: df.describe().T
```

Out[]:

	count	mean	std	min	25%	50%	75%
EmployeeNumber	2940.0	1470.500000	848.849221	1.0	735.75	1470.5	2200.0
Age	2940.0	36.923810	9.133819	18.0	30.00	36.0	44.0
BusinessTravel	2940.0	1.607483	0.665342	0.0	1.00	2.0	3.0
DailyRate	2940.0	802.485714	403.440447	102.0	465.00	802.0	1150.0
Department	2940.0	1.260544	0.527703	0.0	1.00	1.0	3.0
DistanceFromHome	2940.0	9.192517	8.105485	1.0	2.00	7.0	14.0
Education	2940.0	2.912925	1.023991	1.0	2.00	3.0	4.0
EducationField	2940.0	2.247619	1.331143	0.0	1.00	2.0	3.0
EmployeeCount	2940.0	1.000000	0.000000	1.0	1.00	1.0	1.0
EnvironmentSatisfaction	2940.0	2.721769	1.092896	1.0	2.00	3.0	4.0
HourlyRate	2940.0	65.891156	20.325969	30.0	48.00	66.0	84.0
JobInvolvement	2940.0	2.729932	0.711440	1.0	2.00	3.0	4.0
JobLevel	2940.0	2.063946	1.106752	1.0	1.00	2.0	3.0
JobRole	2940.0	4.458503	2.461402	0.0	2.00	5.0	7.0
JobSatisfaction	2940.0	2.728571	1.102658	1.0	2.00	3.0	4.0
MaritalStatus	2940.0	1.097279	0.729997	0.0	1.00	1.0	2.0
MonthlyIncome	2940.0	6502.931293	4707.155770	1009.0	2911.00	4919.0	8380.0
MonthlyRate	2940.0	14313.103401	7116.575021	2094.0	8045.00	14235.5	20460.0
NumCompaniesWorked	2940.0	2.693197	2.497584	0.0	1.00	2.0	3.0
Over18	2940.0	0.000000	0.000000	0.0	0.00	0.0	0.0
OverTime	2940.0	0.282993	0.450530	0.0	0.00	0.0	1.0
PercentSalaryHike	2940.0	15.209524	3.659315	11.0	12.00	14.0	17.0
PerformanceRating	2940.0	3.153741	0.360762	3.0	3.00	3.0	4.0
RelationshipSatisfaction	2940.0	2.712245	1.081025	1.0	2.00	3.0	4.0
StandardHours	2940.0	80.000000	0.000000	80.0	80.00	80.0	80.0
StockOptionLevel	2940.0	0.793878	0.851932	0.0	0.00	1.0	2.0
TotalWorkingYears	2940.0	11.279592	7.779458	0.0	6.00	10.0	14.0
TrainingTimesLastYear	2940.0	2.799320	1.289051	0.0	2.00	3.0	4.0
WorkLifeBalance	2940.0	2.761224	0.706356	1.0	2.00	3.0	4.0
YearsAtCompany	2940.0	7.008163	6.125483	0.0	3.00	5.0	10.0

	count	mean	std	min	25%	50%	75%
YearsInCurrentRole	2940.0	4.229252	3.622521	0.0	2.00	3.0	4.0
YearsSinceLastPromotion	2940.0	2.187755	3.221882	0.0	0.00	1.0	2.0
YearsWithCurrManager	2940.0	4.123129	3.567529	0.0	2.00	3.0	4.0

```
In [ ]: #Normalization and display the result
columns_to_normalize = [
    'DailyRate', 'Education', 'MonthlyIncome'
    , 'NumCompaniesWorked', 'PercentSalaryHike', 'TotalWorkingYears'
    , 'TrainingTimesLastYear', 'YearsAtCompany', 'YearsInCurrentRole'
    , 'YearsWithCurrManager'
]
# Removed extra spaces and comma
scaler = MinMaxScaler()
df[columns_to_normalize] = scaler.fit_transform(df[columns_to_normalize])

df.head()
```

```
Out[ ]: EmployeeNumber  Attrition  Age  BusinessTravel  DailyRate  Department  DistanceFromHome
```

0	1	Yes	41	2	0.715820	2
1	2	No	49	1	0.126700	1
2	3	Yes	37	2	0.909807	1
3	4	No	33	1	0.923407	1
4	5	No	27	2	0.350036	1

5 rows × 35 columns



```
In [ ]: pd.DataFrame(df).describe()
```

Out []:

	EmployeeNumber	Age	BusinessTravel	DailyRate	Department	DistanceFrom
count	2940.000000	2940.000000	2940.000000	2940.000000	2940.000000	29
mean	1470.500000	36.923810	1.607483	0.501421	1.260544	
std	848.849221	9.133819	0.665342	0.288791	0.527703	
min	1.000000	18.000000	0.000000	0.000000	0.000000	
25%	735.750000	30.000000	1.000000	0.259843	1.000000	
50%	1470.500000	36.000000	2.000000	0.501074	1.000000	
75%	2205.250000	43.000000	2.000000	0.755190	2.000000	
max	2940.000000	60.000000	2.000000	1.000000	2.000000	

8 rows × 33 columns



DATA visualization

In []: `df.head()`

Out []:

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFrom
0	1	Yes	41	2	0.715820	2	
1	2	No	49	1	0.126700	1	
2	3	Yes	37	2	0.909807	1	
3	4	No	33	1	0.923407	1	
4	5	No	27	2	0.350036	1	

5 rows × 35 columns



In []: `df.Age.unique()`

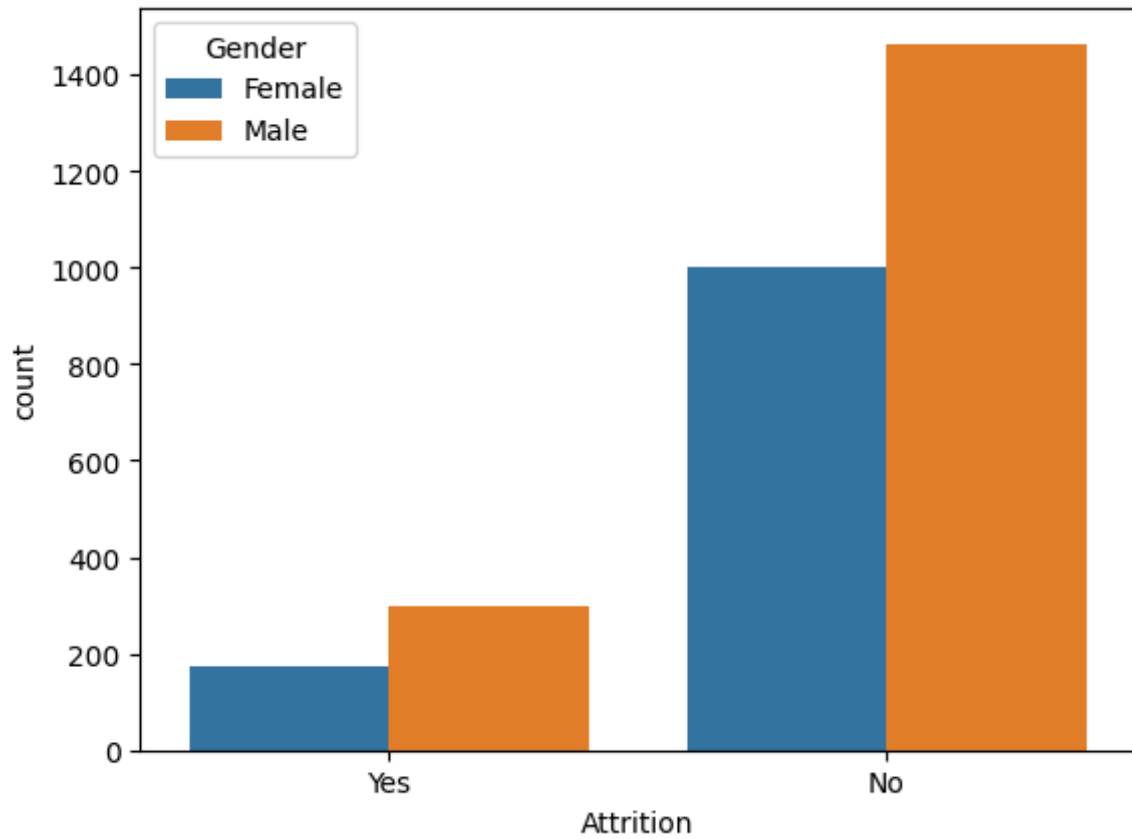
Out []: `array([41, 49, 37, 33, 27, 32, 59, 30, 38, 36, 35, 29, 31, 34, 28, 22, 53, 24, 21, 42, 44, 46, 39, 43, 50, 26, 48, 55, 45, 56, 23, 51, 40, 54, 58, 20, 25, 19, 57, 52, 47, 18, 60])`

In []: *#males are denotes by 1 and females are denoted by 0*

Bar chart

In []: `sns.countplot(x = 'Attrition', hue= 'Gender', data = df)`

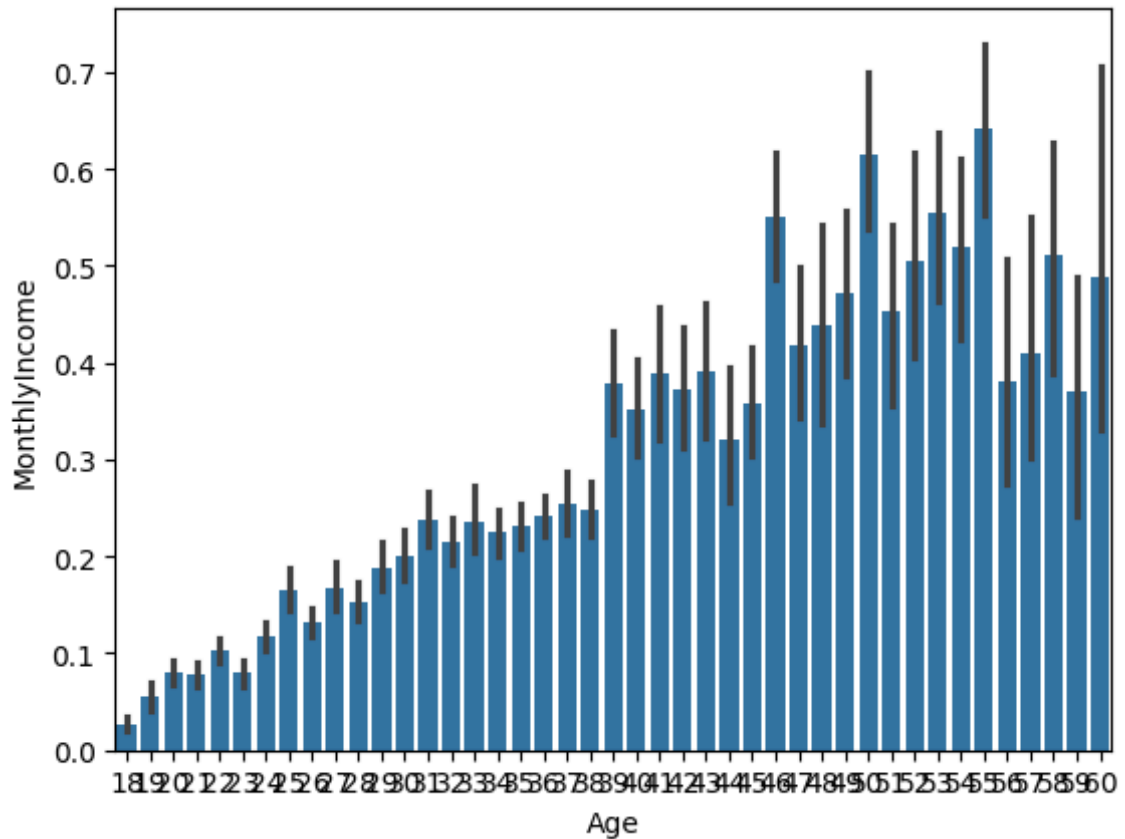
Out []: `<Axes: xlabel='Attrition', ylabel='count'>`



males seem to hold the edge over which sex was layed off amongst the 2940 employees

```
In [ ]: sns.barplot(x='Age', data=df, y='MonthlyIncome')
```

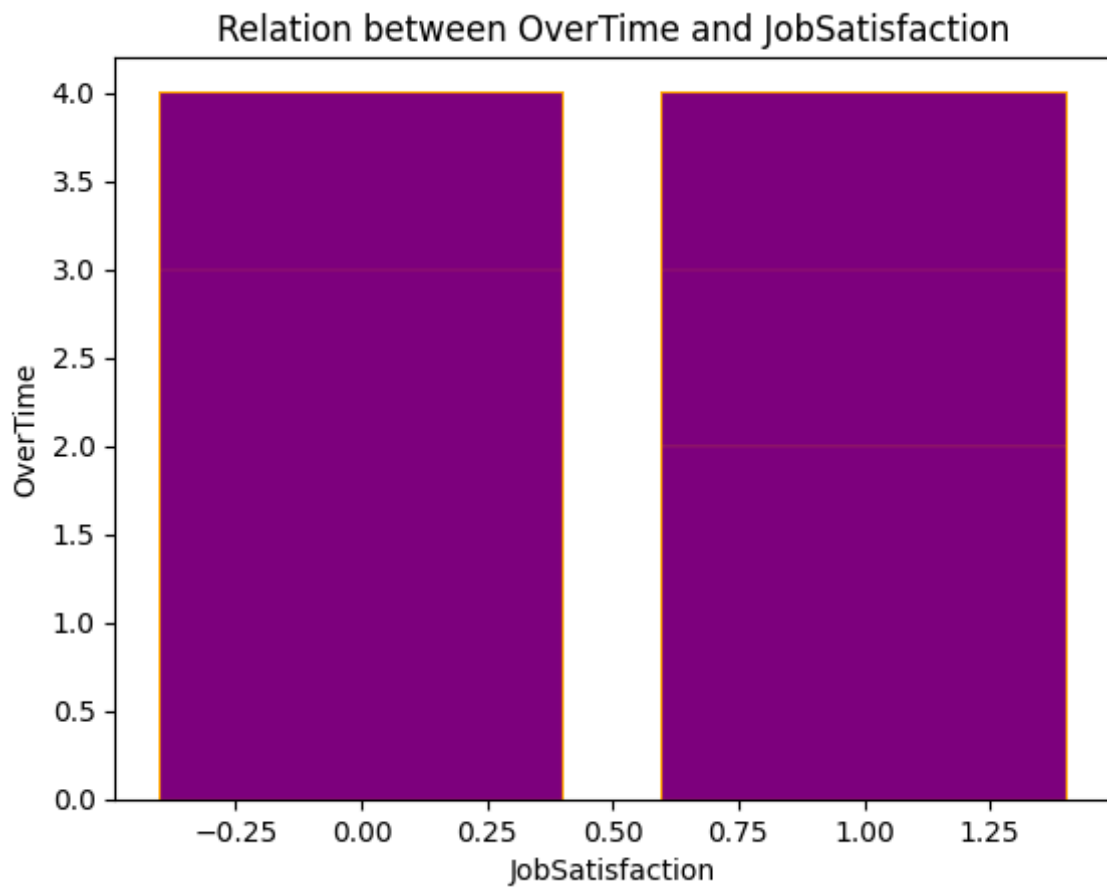
```
Out[ ]: <Axes: xlabel='Age', ylabel='MonthlyIncome'>
```



people who are older and supposedly having more experience tend to earn more than their younger counterparts but the the number of younger workers is significantly higher than the older members of the industry as well

```
In [ ]: plt.bar(df['OverTime'],df['JobSatisfaction'],color = 'purple', edgecolor = 'orange')
plt.title('Relation between OverTime and JobSatisfaction')
plt.xlabel('JobSatisfaction')
plt.ylabel('OverTime')

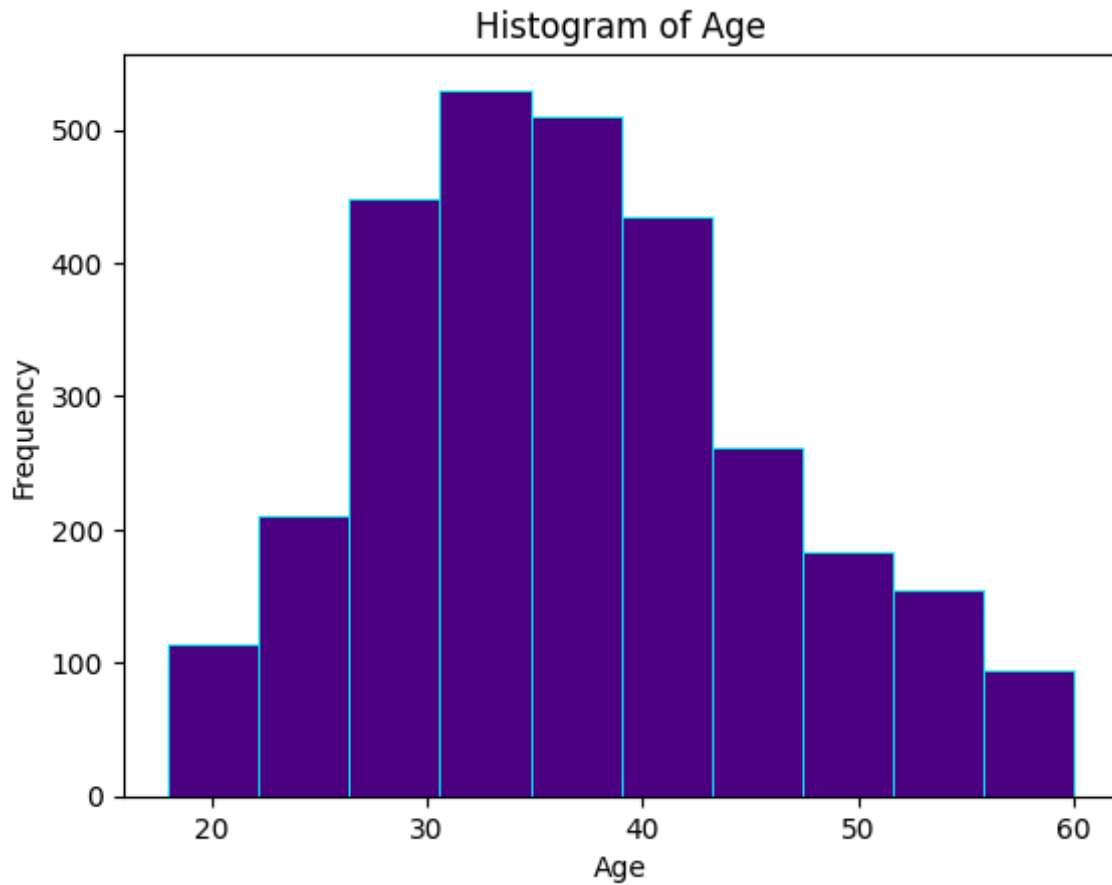
plt.show()
```



Histogram

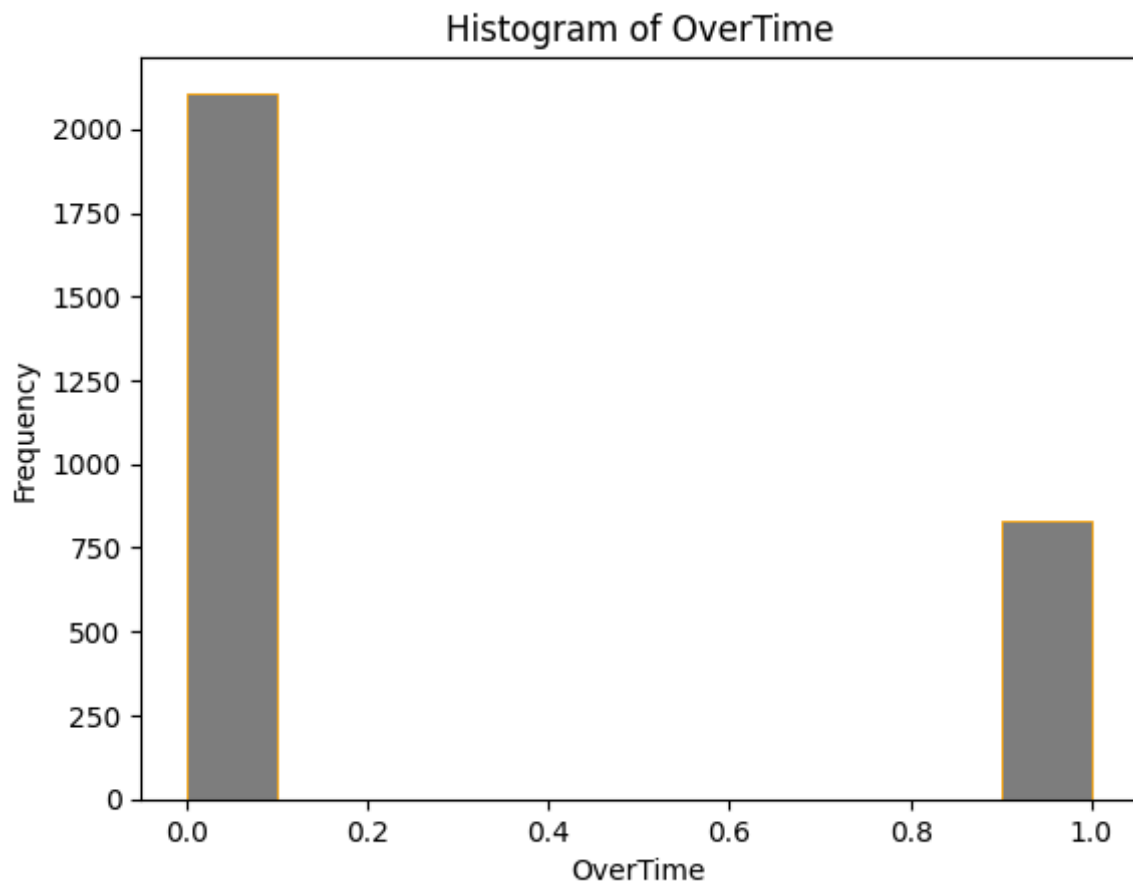
```
In [ ]: plt.hist(df['Age'],color = 'indigo', edgecolor = 'aqua', linewidth = 0.6)
plt.title('Histogram of Age')

plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



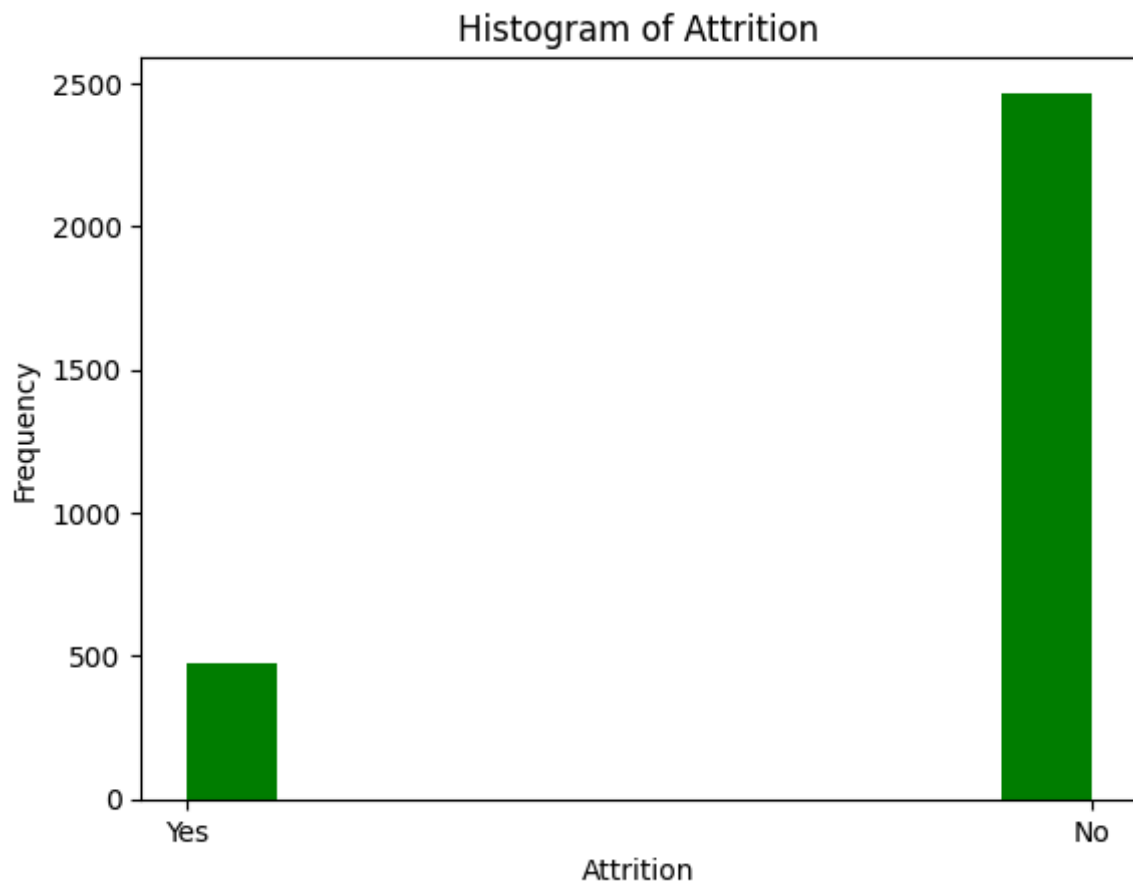
majority of the workforce was made up of people in their mid 30s and early 40s

```
In [ ]: plt.hist(df['OverTime'],color = 'grey', edgecolor = 'orange', linewidth = 0.6)
plt.title('Histogram of OverTime')
plt.xlabel('OverTime')
plt.ylabel('Frequency')
plt.show()
```



the frequency of overtime working is less hence it seems like a rare occurrence to have a large group of people working over time

```
In [ ]: plt.hist(df['Attrition'], color = 'green')
plt.title('Histogram of Attrition')
plt.xlabel('Attrition')
plt.ylabel('Frequency')
plt.show()
```

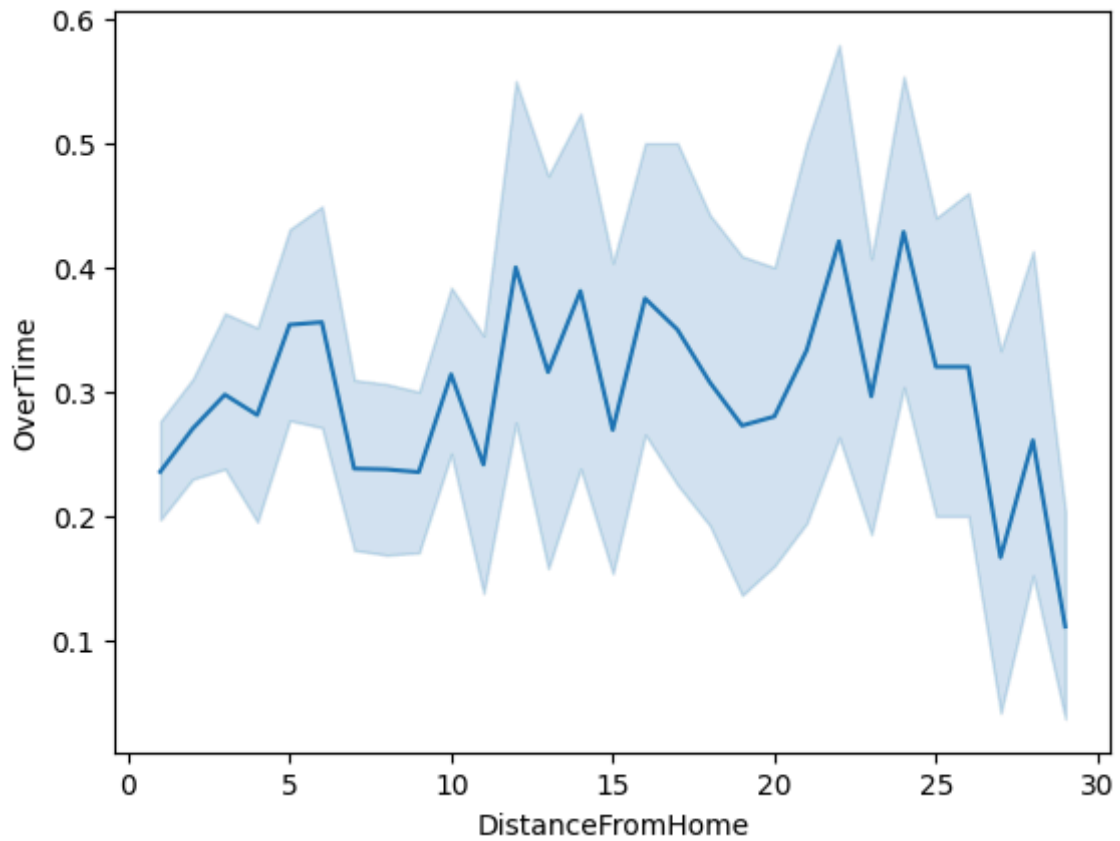


the amount of people layed of is lesser in ratio compared to the overall strenght of the workforce

LINE PLOT

```
In [ ]: sns.lineplot(x = 'DistanceFromHome', y = 'OverTime', data = df)
```

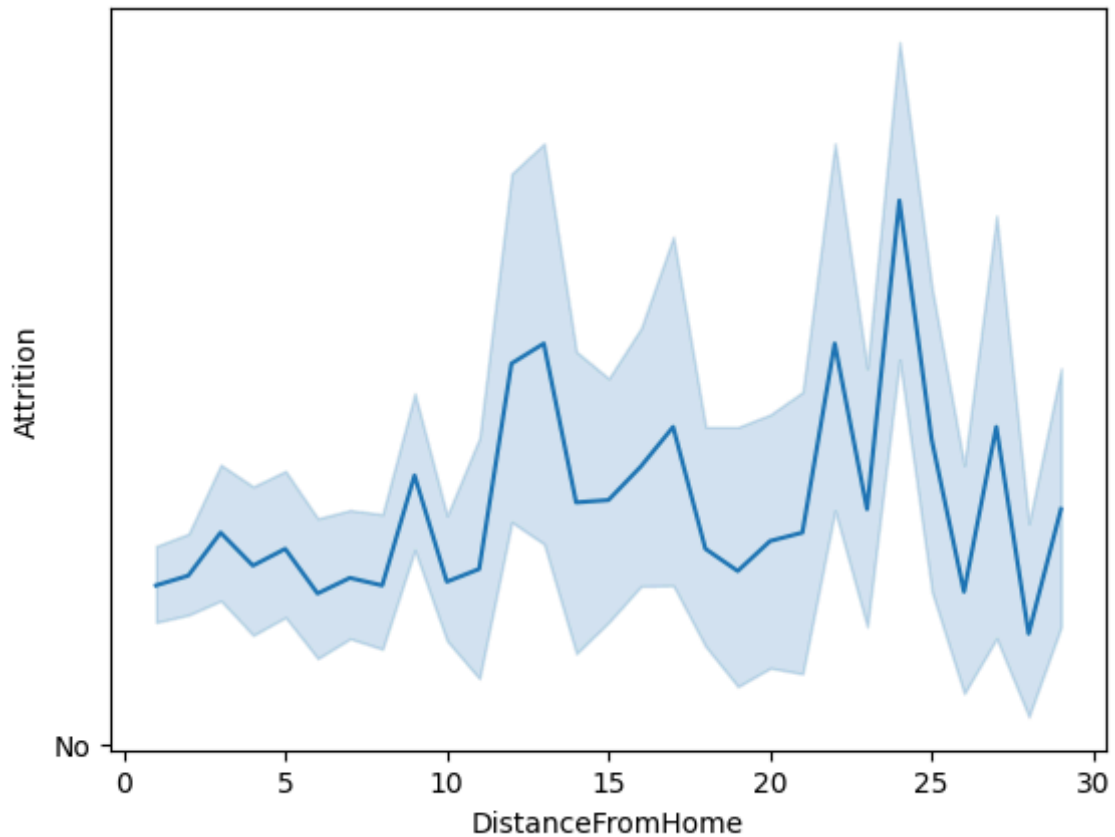
```
Out[ ]: <Axes: xlabel='DistanceFromHome', ylabel='OverTime'>
```



employees commting from far away places seem to rarely take up the overtime role in the company

```
In [ ]: sns.lineplot(x = 'DistanceFromHome', y = 'Attrition', data = df)
```

```
Out[ ]: <Axes: xlabel='DistanceFromHome', ylabel='Attrition'>
```

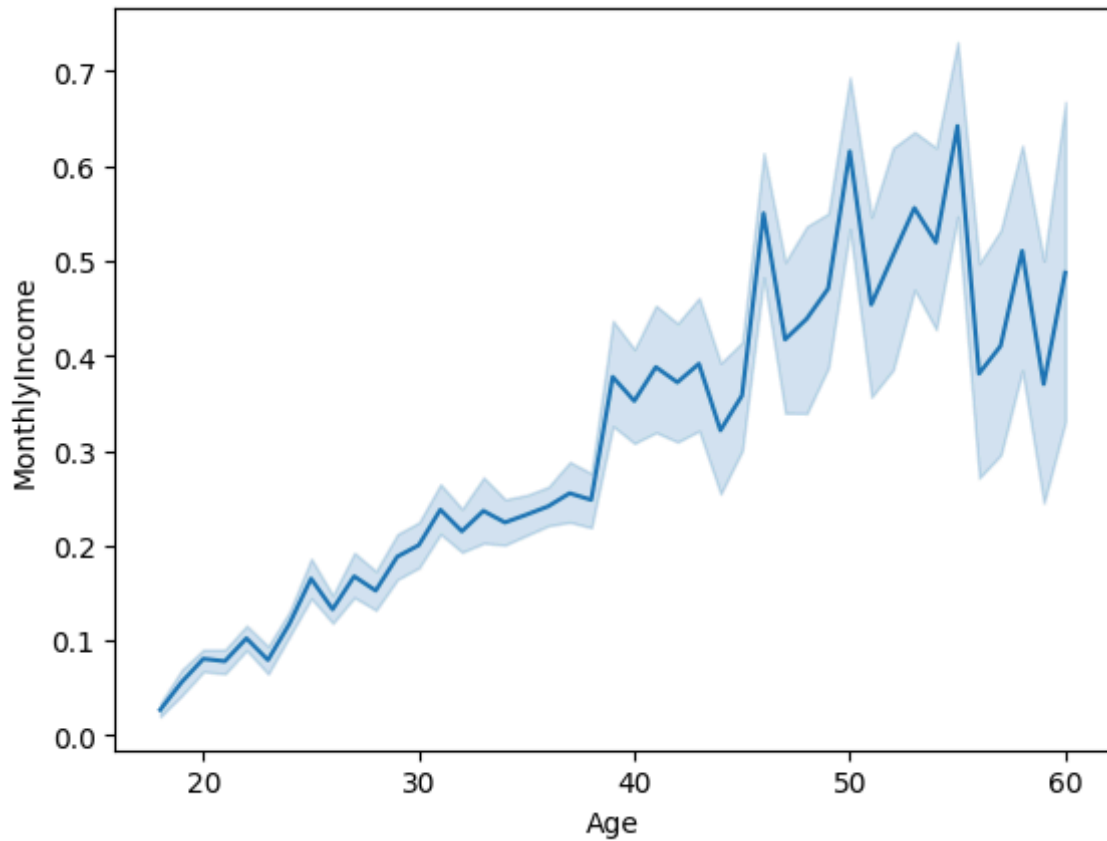


the graph does not offer any conclusive evidence about the lay off pattern and distance form home but a significant amount of people seemed to have been layed off who commute between 23-26 kms away. 12-14 and 21-23 are tied after

```
In [ ]: import seaborn as sns
```

```
In [ ]: sns.lineplot(x = 'Age', y = 'MonthlyIncome', data = df)
```

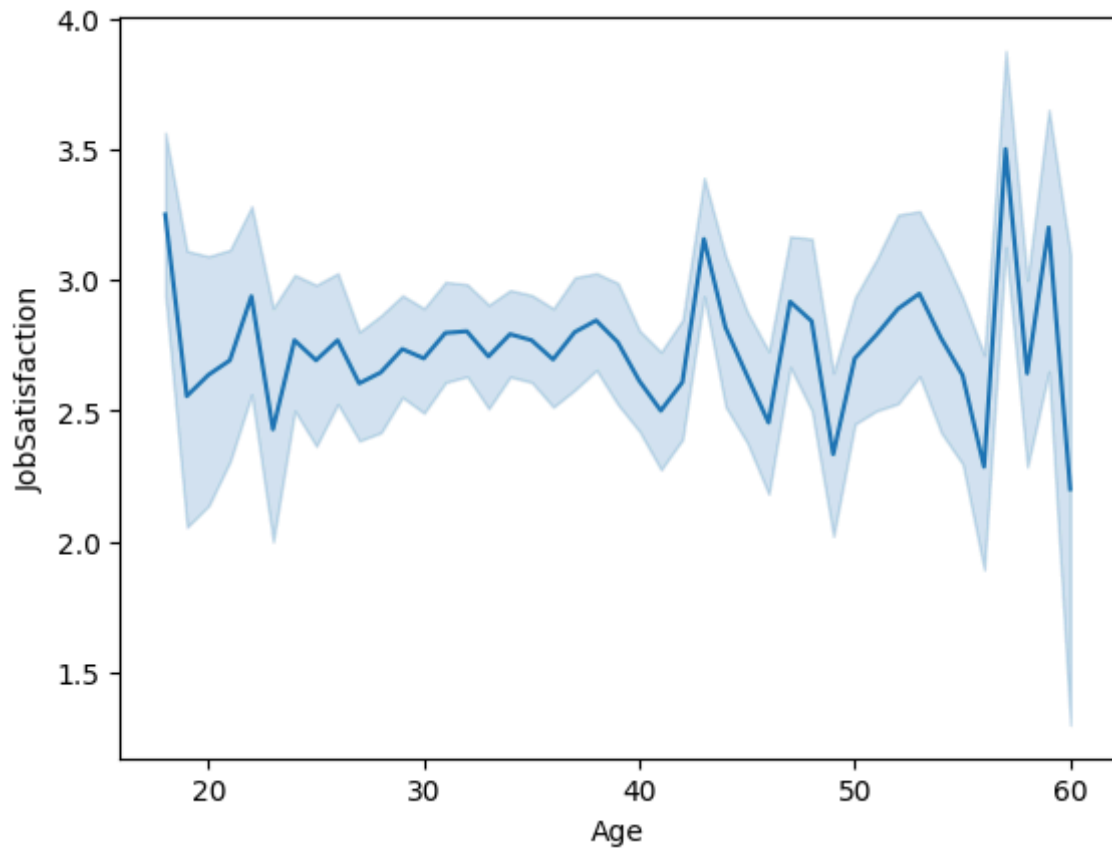
```
Out[ ]: <Axes: xlabel='Age', ylabel='MonthlyIncome'>
```



monthly income of older employees is higher than younger employees and almost follows a linear pattern.hence hikes in salary is reflective of either work experience outside the company or years spent in service for the company

```
In [ ]: sns.lineplot(x = 'Age', y = 'JobSatisfaction', data = df)
```

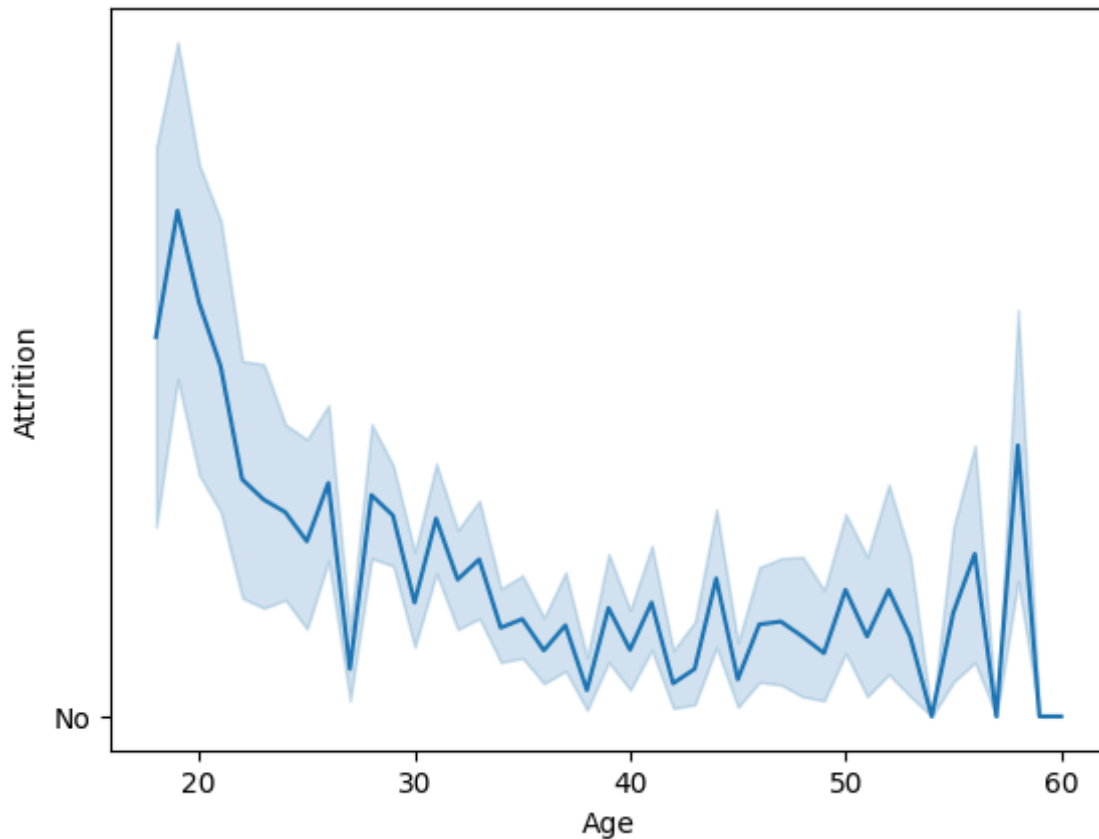
```
Out[ ]: <Axes: xlabel='Age', ylabel='JobSatisfaction'>
```



job satisfaction seems to hold a median level among all age groups but the older employees edge the others in this aspect probably due to a more salary package or just they just seem to enjoy the stage in their late years

```
In [ ]: sns.lineplot(x = 'Age', y = 'Attrition', data = df)
```

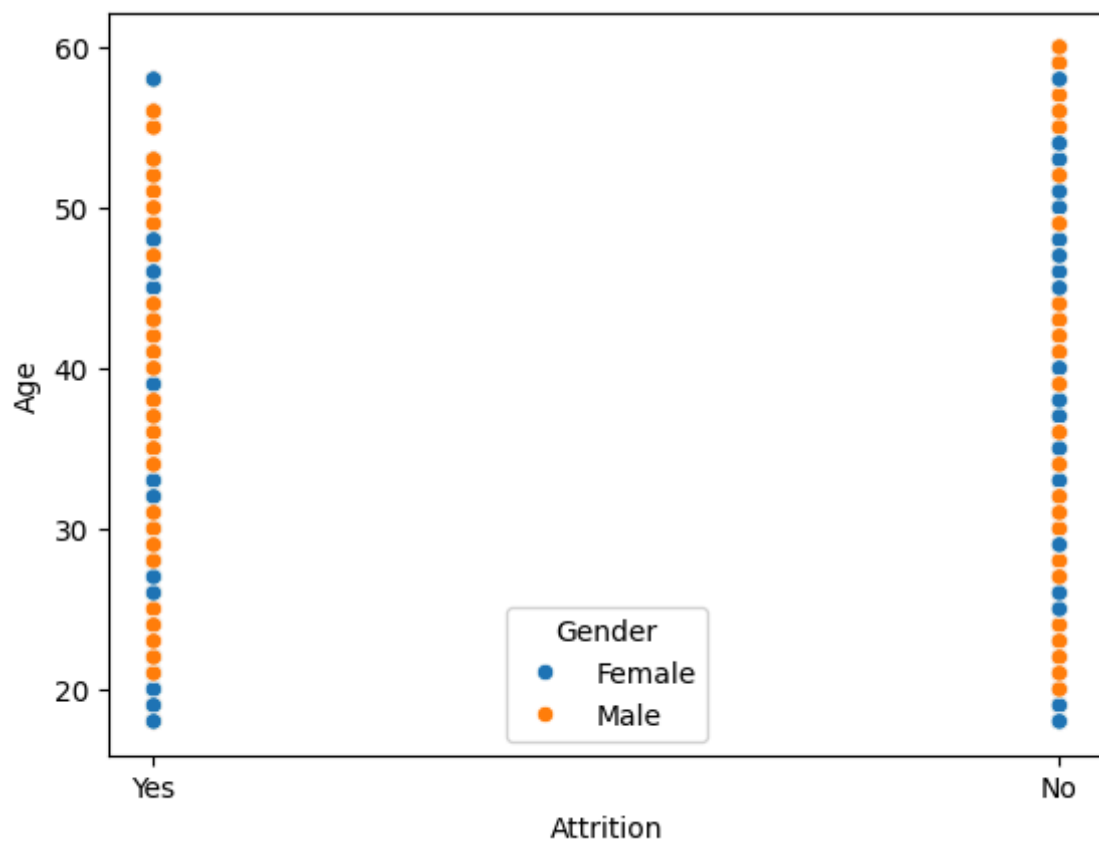
```
Out[ ]: <Axes: xlabel='Age', ylabel='Attrition'>
```



a significant amount of people layed seems to constitute the younger workforce compared to older employees but the majority group which are the middle aged employees seem to have not experienced much change in personell

```
In [ ]: #dependency on Attition based on Gender  
sns.scatterplot(y = 'Age', x = 'Attrition', data = df , hue = 'Gender')
```

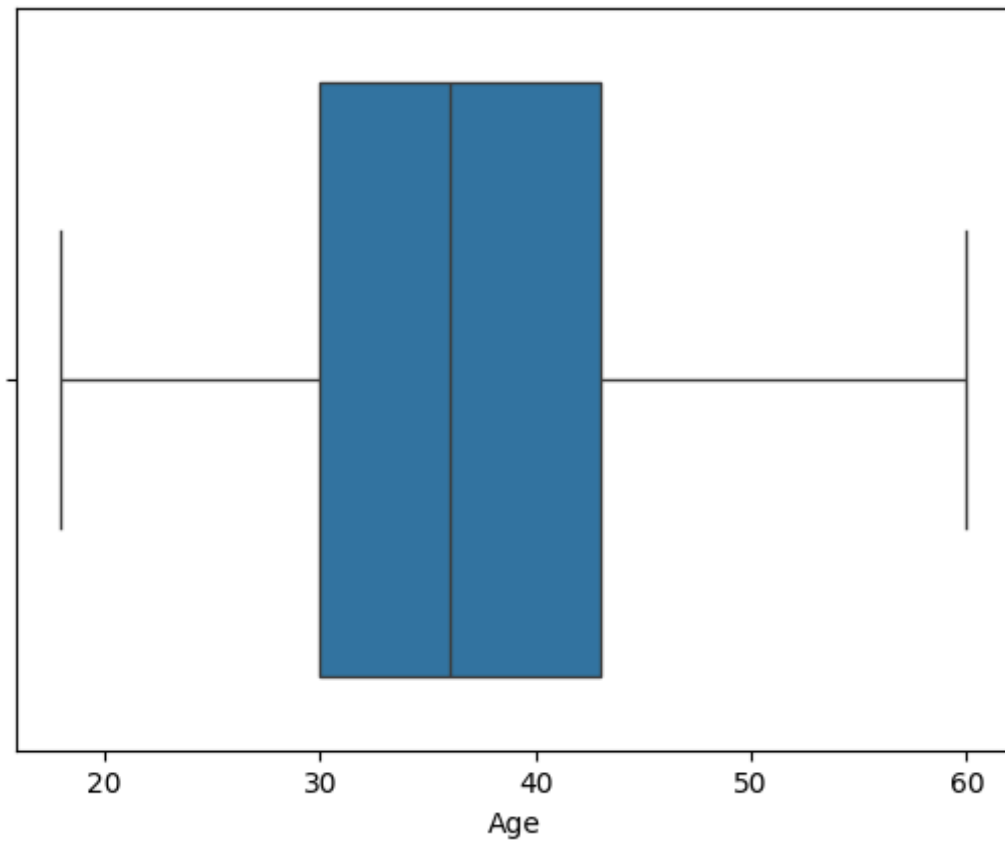
```
Out[ ]: <Axes: xlabel='Attrition', ylabel='Age'>
```



Barplot

```
In [ ]: #boxplot
sns.boxplot(x = 'Age', data = df)
```

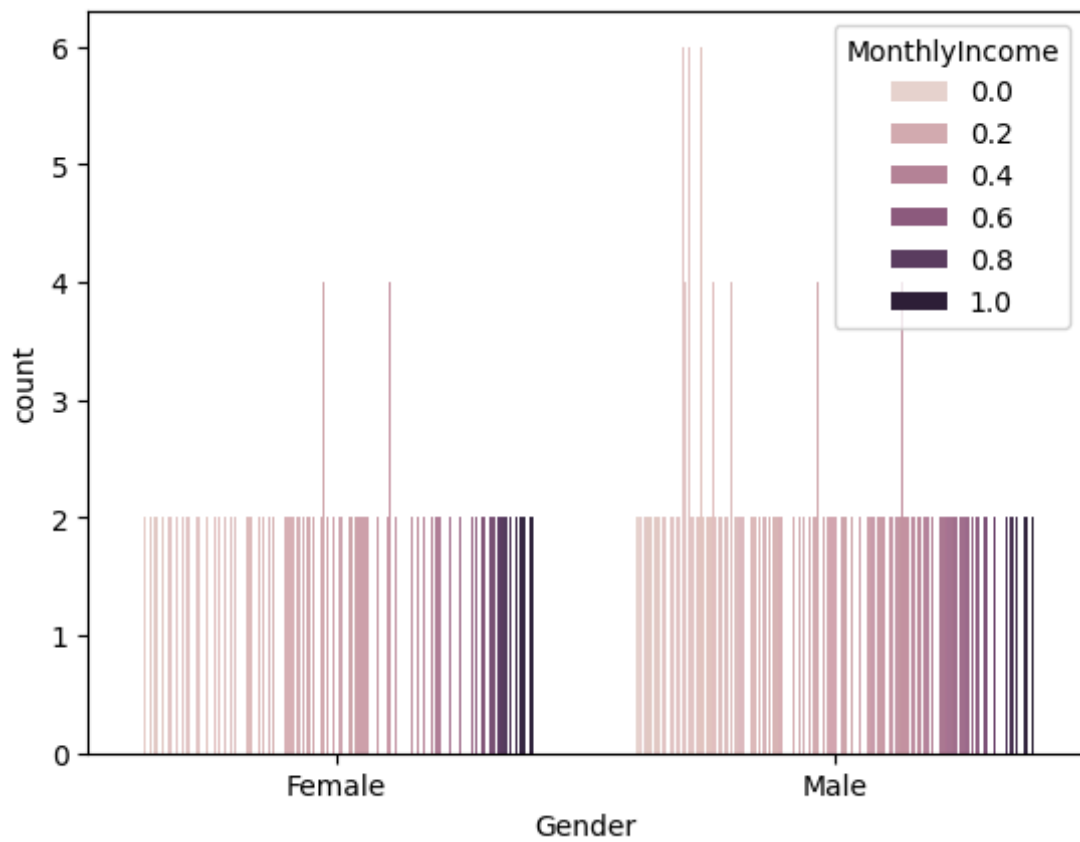
```
Out[ ]: <Axes: xlabel='Age'>
```



50% of workforc is between the ages of 30 and 44 and the upper 3rd quartile seems to have the bigger chunk compared to 1st quartile

```
In [ ]: #countplot
sns.countplot(x= 'Gender', data = df, hue = 'MonthlyIncome')
```

```
Out[ ]: <Axes: xlabel='Gender', ylabel='count'>
```



males are denoted as 1 and females are denoted as 0 and the male population seems to have more high earners compared to their female counterparts

In []: df

```
Out[ ]:
```

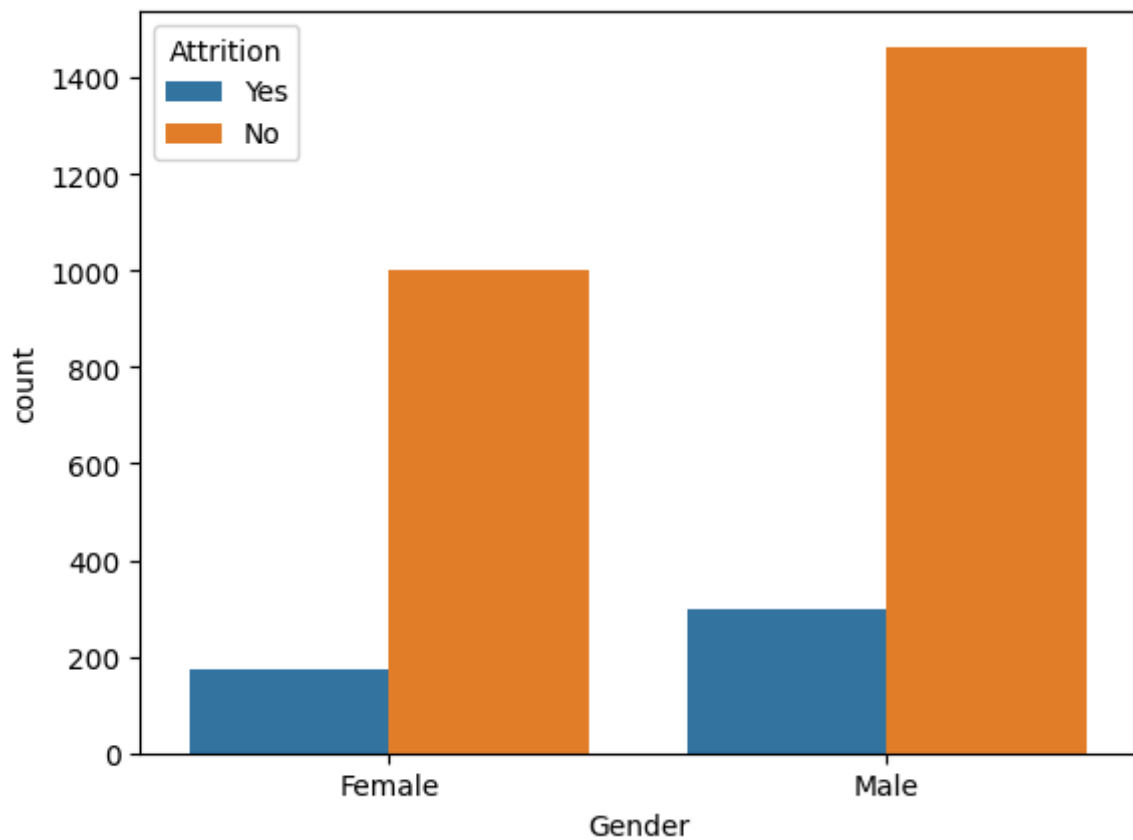
	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFr
0	1	Yes	41	2	0.715820	2	
1	2	No	49	1	0.126700	1	
2	3	Yes	37	2	0.909807	1	
3	4	No	33	1	0.923407	1	
4	5	No	27	2	0.350036	1	
...	...	...	...	...	...	...	...
2935	2936	No	36	1	0.559771	1	
2936	2937	No	39	2	0.365784	1	
2937	2938	No	27	2	0.037938	1	
2938	2939	No	49	1	0.659270	2	
2939	2940	No	34	2	0.376521	1	

2940 rows × 35 columns



```
In [ ]: sns.countplot(x= 'Gender', data = df, hue = 'Attrition')
```

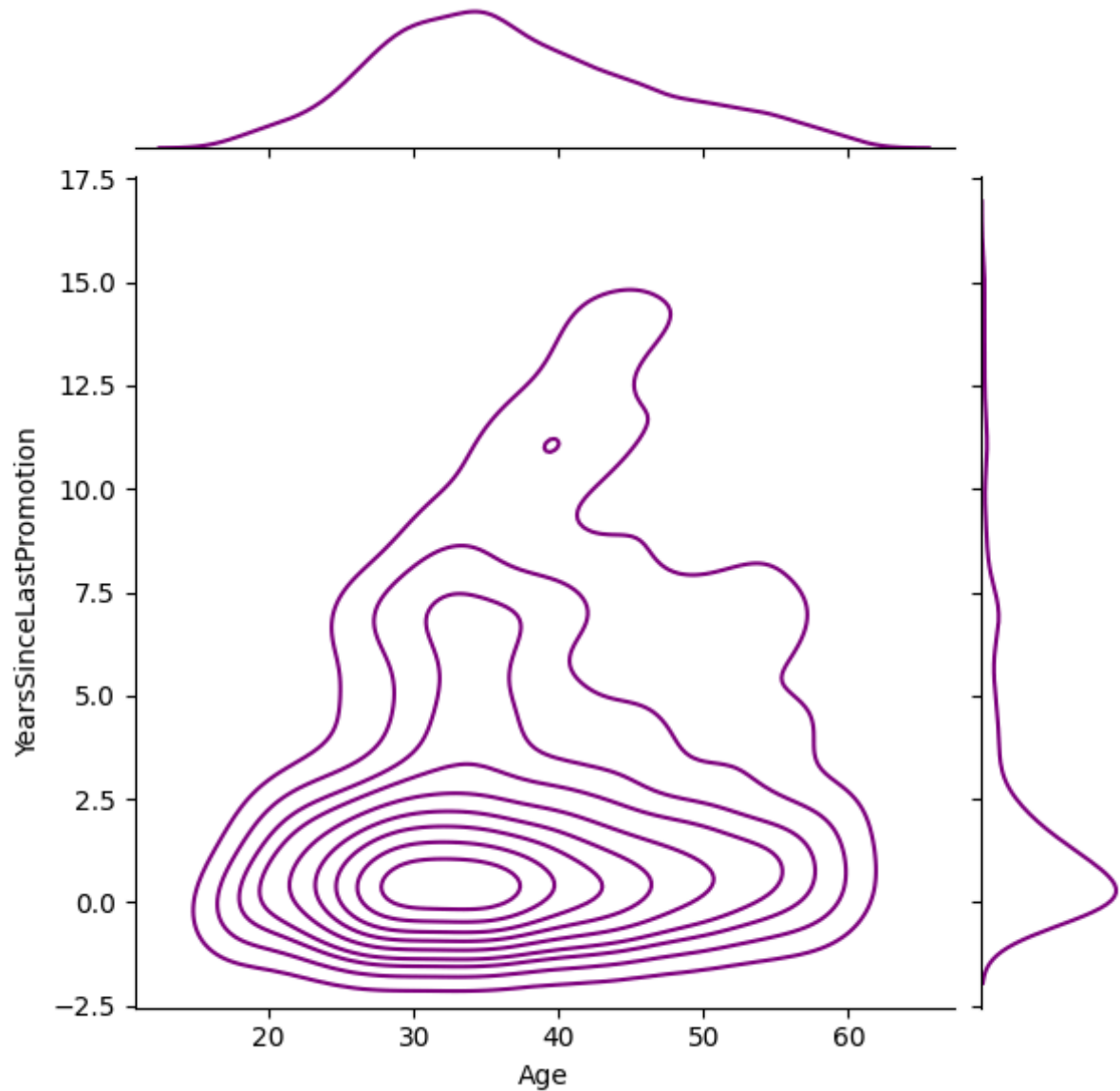
```
Out[ ]: <Axes: xlabel='Gender', ylabel='count'>
```



Bivariate graphs

```
In [ ]: #jointplot
sns.jointplot(x= 'Age', y = 'YearsSinceLastPromotion',data = df, kind = 'kde', color
```

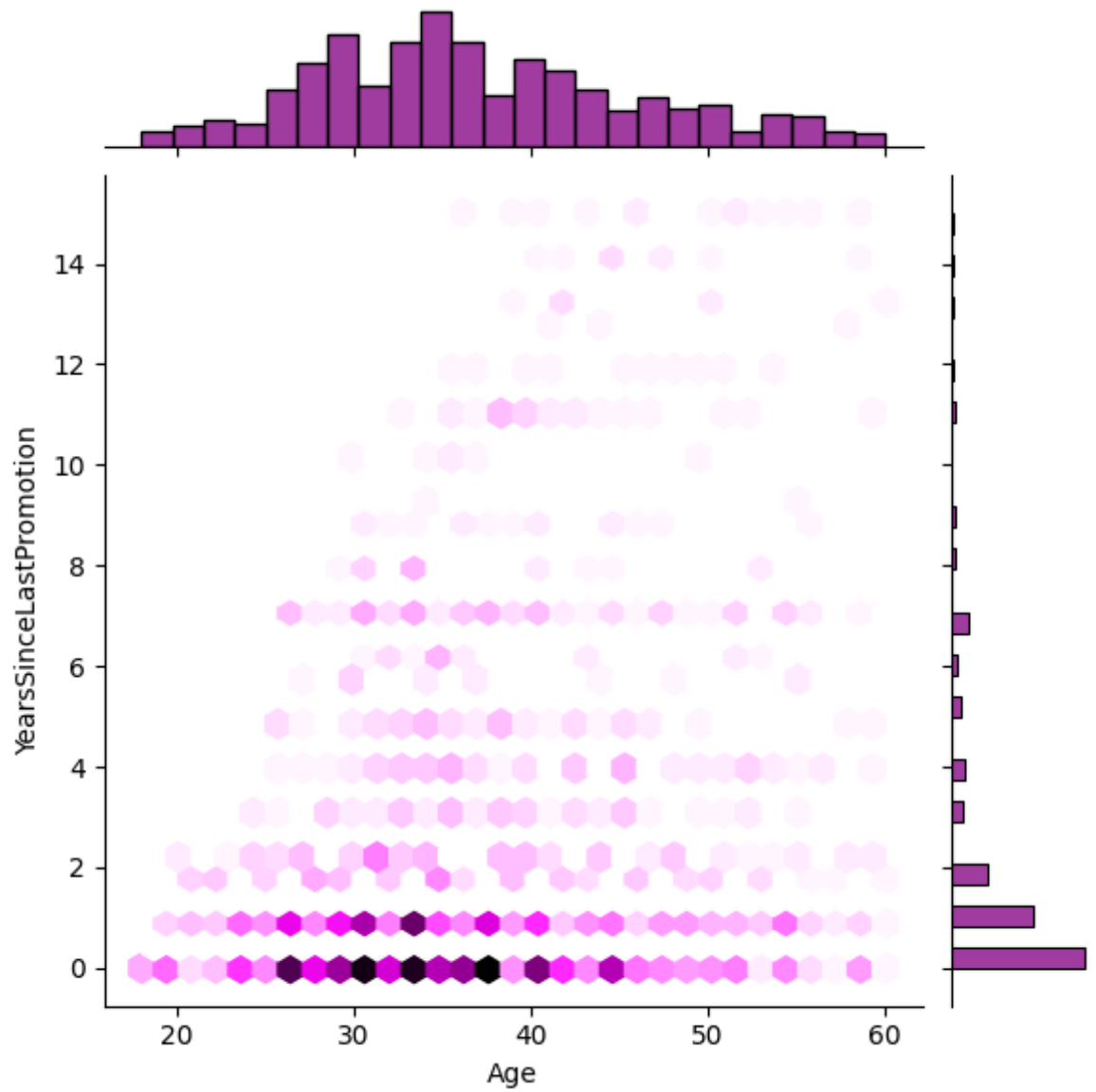
```
Out[ ]: <seaborn.axisgrid.JointGrid at 0x79561988e2f0>
```



middle aged category seemed to have experienced a more stagnant career progression and have been due for a promotion for longer time than other groups

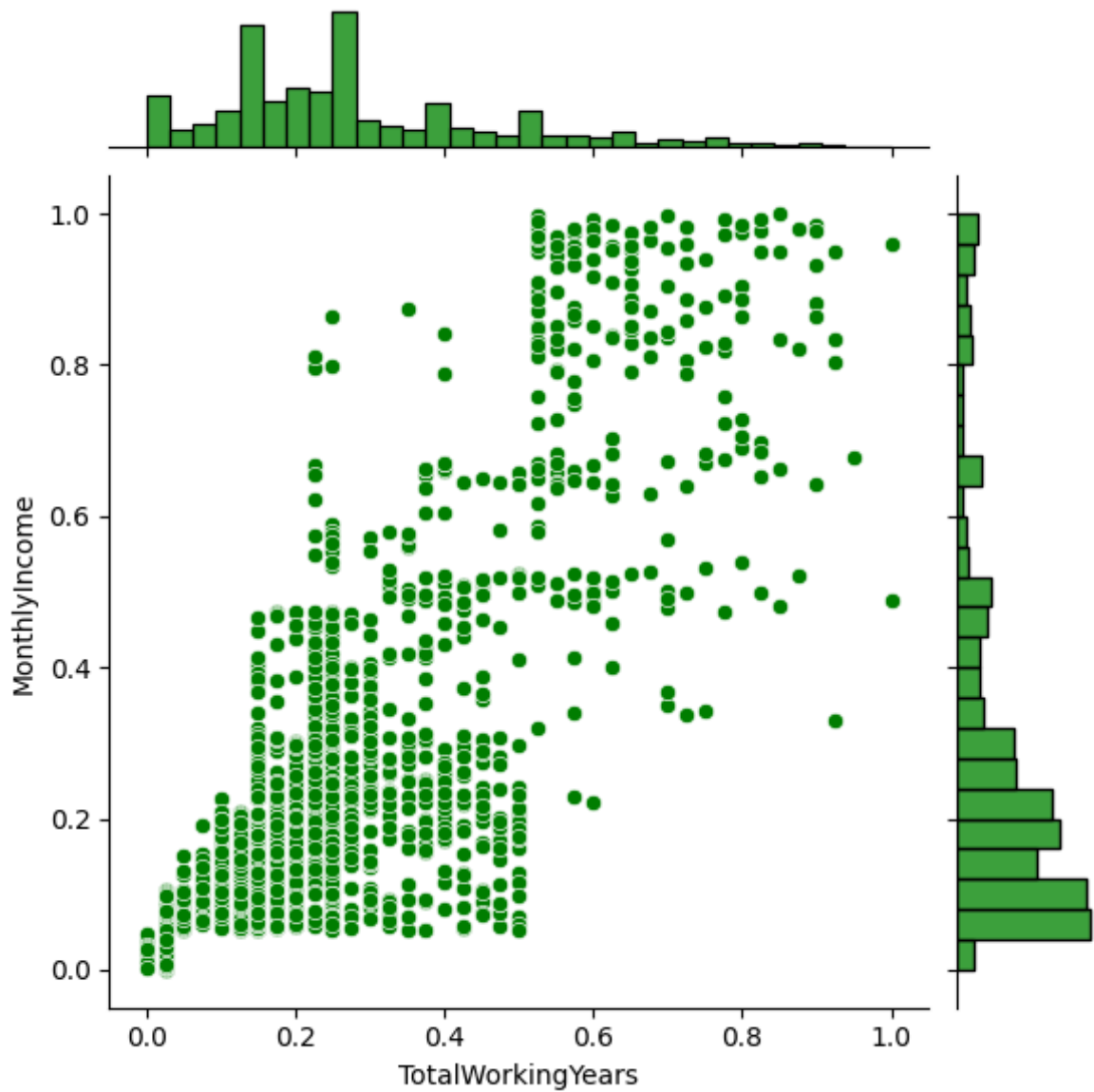
```
In [ ]: #jointplot
sns.jointplot(x = 'Age', y = 'YearsSinceLastPromotion',data = df, kind = 'hex', co
```

```
Out[ ]: <seaborn.axisgrid.JointGrid at 0x7955f7e11690>
```



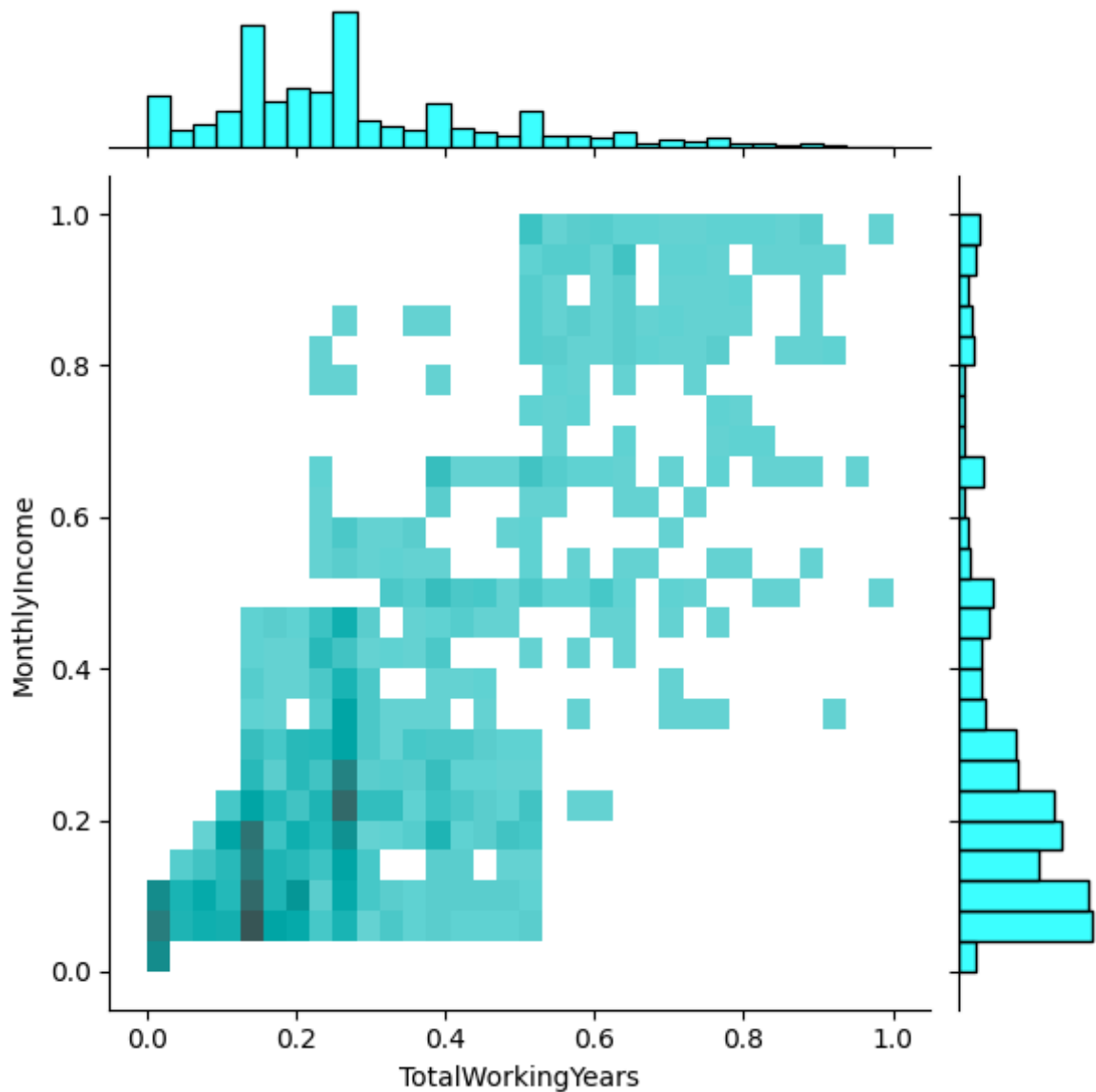
```
In [ ]: sns.jointplot(x = 'TotalWorkingYears', y = 'MonthlyIncome', data = df, kind = 'scat
```

```
Out[ ]: <seaborn.axisgrid.JointGrid at 0x7955f7e119f0>
```



```
In [ ]: sns.jointplot(x = 'TotalWorkingYears', y = 'MonthlyIncome', data = df, kind = 'hist
```

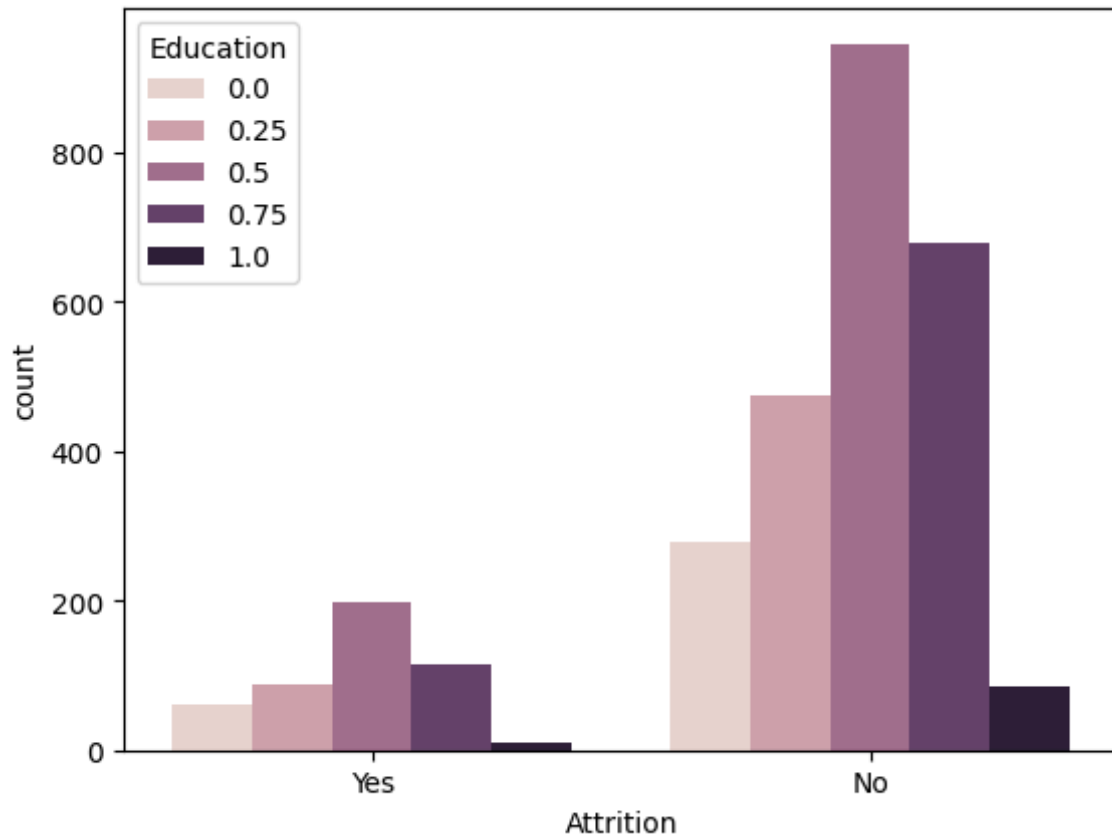
```
Out[ ]: <seaborn.axisgrid.JointGrid at 0x7955f7d931f0>
```



Monthly income of employees seems to vary linearly for the majority with the total working years spent in the company and majority of employees seems to make up the younger quartile of the total strength.

```
In [ ]: sns.countplot(x= 'Attrition', data = df, hue = 'Education')
```

```
Out[ ]: <Axes: xlabel='Attrition', ylabel='count'>
```



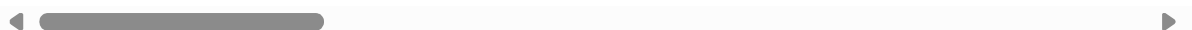
the lower the education the higher number of employees among that category got layed off

In []: df

Out[]:

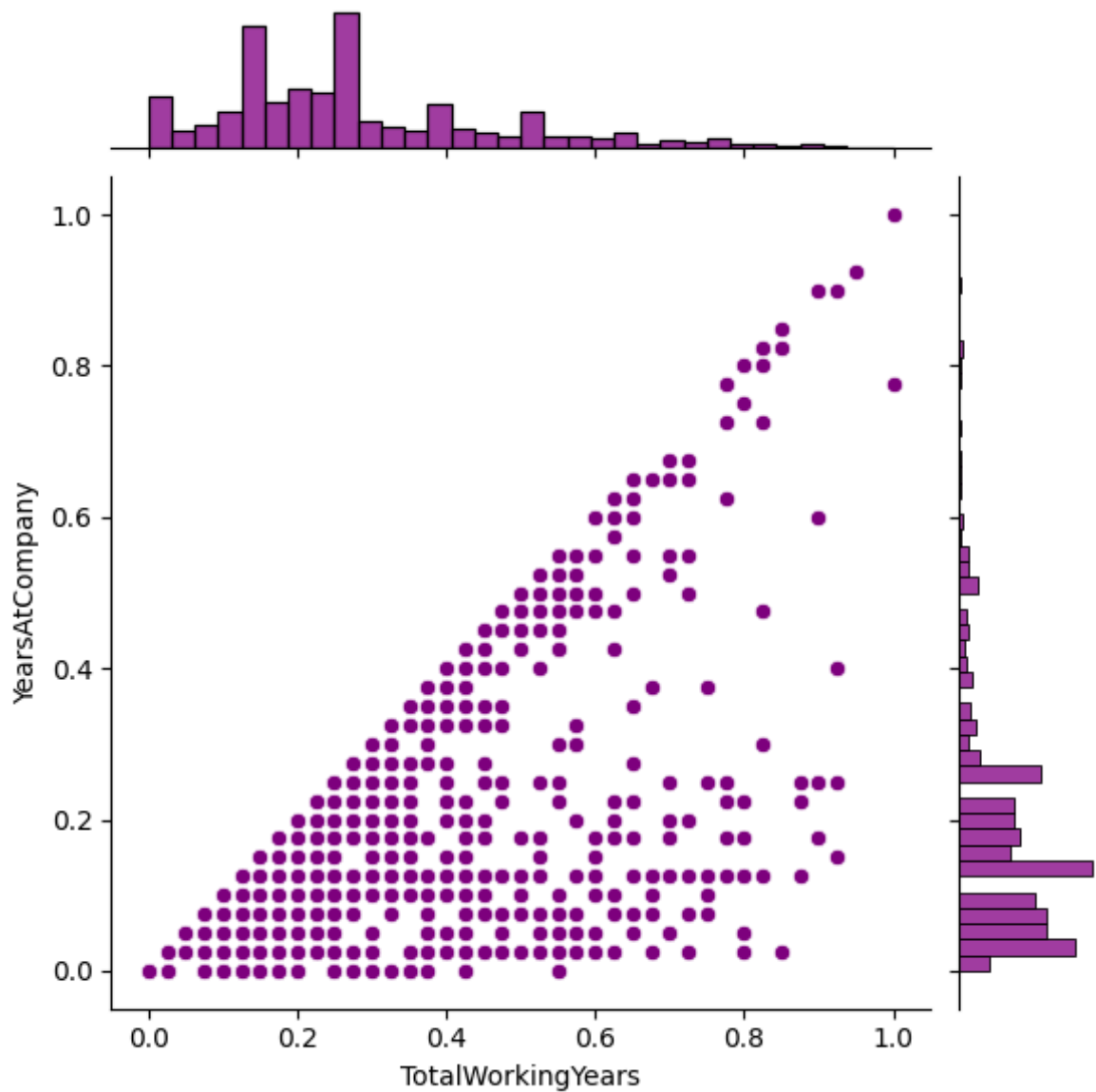
	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFr
0	1	Yes	41	2	0.715820	2	
1	2	No	49	1	0.126700	1	
2	3	Yes	37	2	0.909807	1	
3	4	No	33	1	0.923407	1	
4	5	No	27	2	0.350036	1	
...	...	...	...	...	...	...	
2935	2936	No	36	1	0.559771	1	
2936	2937	No	39	2	0.365784	1	
2937	2938	No	27	2	0.037938	1	
2938	2939	No	49	1	0.659270	2	
2939	2940	No	34	2	0.376521	1	

2940 rows × 35 columns



```
In [ ]: sns.jointplot(x= 'TotalWorkingYears', y = 'YearsAtCompany', data = df, kind = 'scat
```

```
Out[ ]: <seaborn.axisgrid.JointGrid at 0x795618195d20>
```



A significant amount of employees seem to be recruited from other companies and do not qualify as freshers so the workforce in general seems to be well experienced. there appears to be a linear progression in terms of recruitment strategies from the companies end who have made it a point to maintain experinece and youth

```
In [ ]: df.info()
```

```

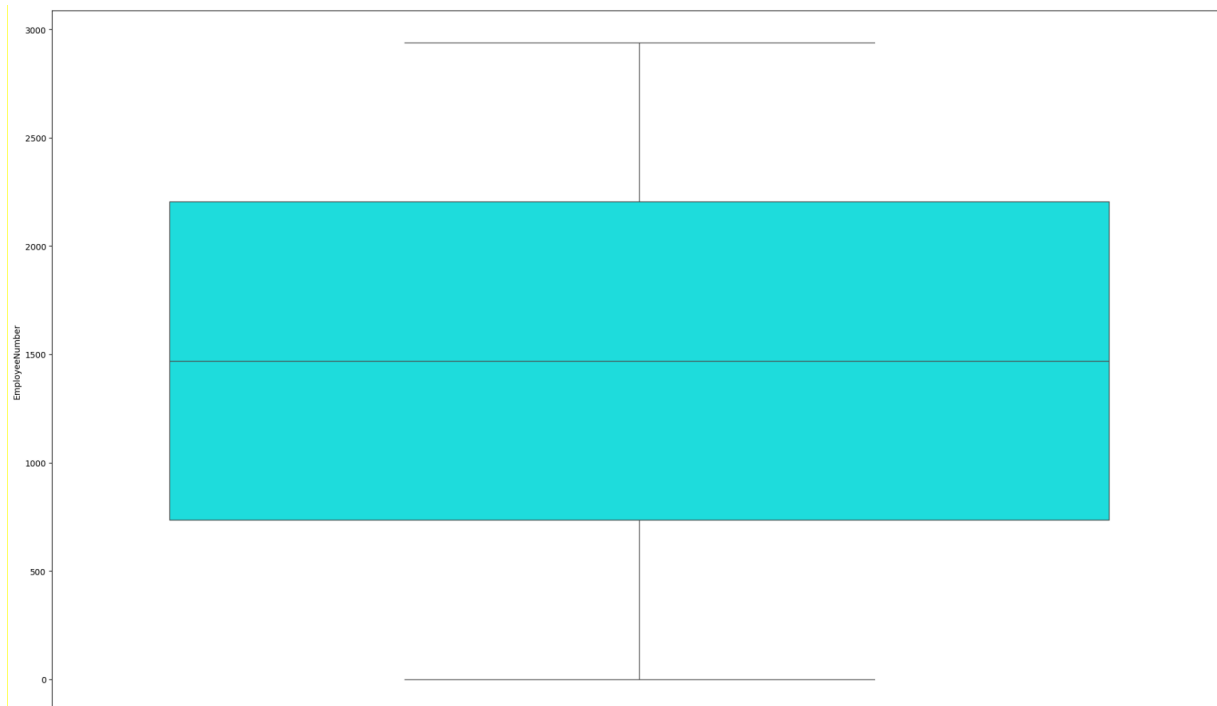
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2940 entries, 0 to 2939
Data columns (total 35 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   EmployeeNumber                        2940 non-null   int64
1   Attrition                            2940 non-null   object
2   Age                                   2940 non-null   int64
3   BusinessTravel                       2940 non-null   int64
4   DailyRate                            2940 non-null   float64
5   Department                           2940 non-null   int64
6   DistanceFromHome                     2940 non-null   int64
7   Education                             2940 non-null   float64
8   EducationField                       2940 non-null   int64
9   EmployeeCount                        2940 non-null   int64
10  EnvironmentSatisfaction               2940 non-null   int64
11  Gender                               2940 non-null   object
12  HourlyRate                           2940 non-null   int64
13  JobInvolvement                       2940 non-null   int64
14  JobLevel                             2940 non-null   int64
15  JobRole                              2940 non-null   int64
16  JobSatisfaction                      2940 non-null   int64
17  MaritalStatus                       2940 non-null   int64
18  MonthlyIncome                       2940 non-null   float64
19  MonthlyRate                          2940 non-null   int64
20  NumCompaniesWorked                   2940 non-null   float64
21  Over18                              2940 non-null   int64
22  OverTime                             2940 non-null   int64
23  PercentSalaryHike                    2940 non-null   float64
24  PerformanceRating                    2940 non-null   int64
25  RelationshipSatisfaction              2940 non-null   int64
26  StandardHours                       2940 non-null   int64
27  StockOptionLevel                     2940 non-null   int64
28  TotalWorkingYears                    2940 non-null   float64
29  TrainingTimesLastYear                2940 non-null   float64
30  WorkLifeBalance                      2940 non-null   int64
31  YearsAtCompany                       2940 non-null   float64
32  YearsInCurrentRole                   2940 non-null   float64
33  YearsSinceLastPromotion              2940 non-null   int64
34  YearsWithCurrManager                 2940 non-null   float64
dtypes: float64(10), int64(23), object(2)
memory usage: 804.0+ KB

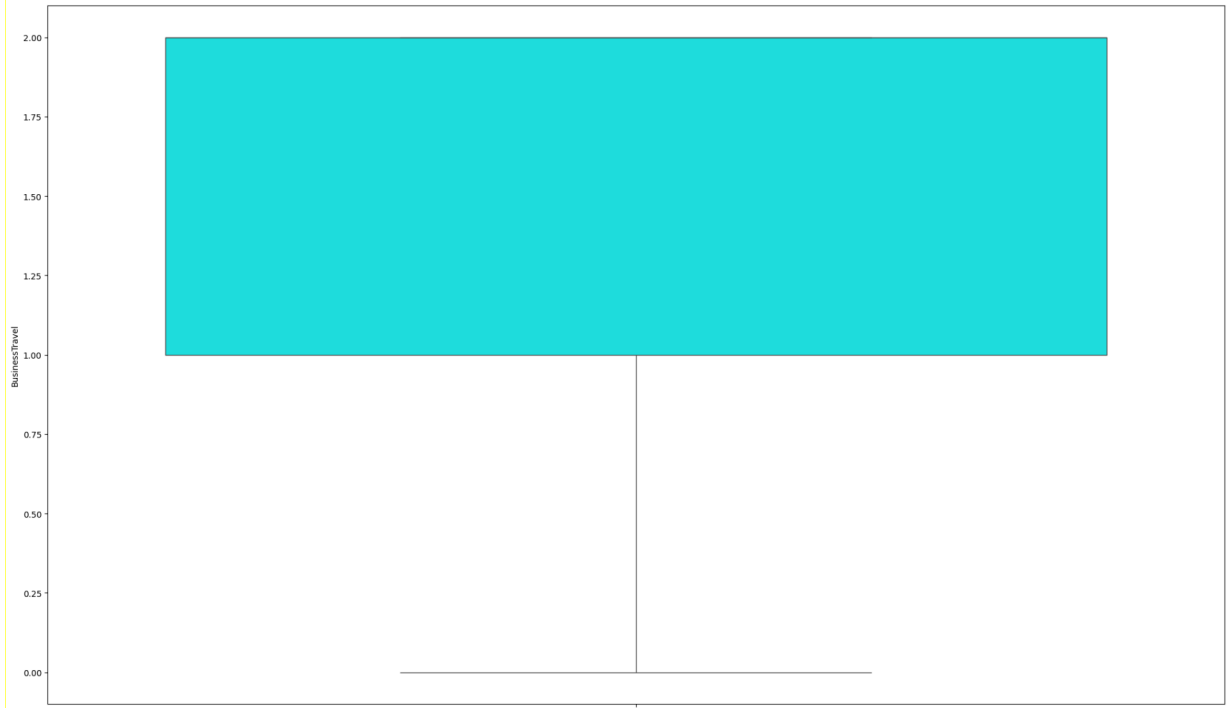
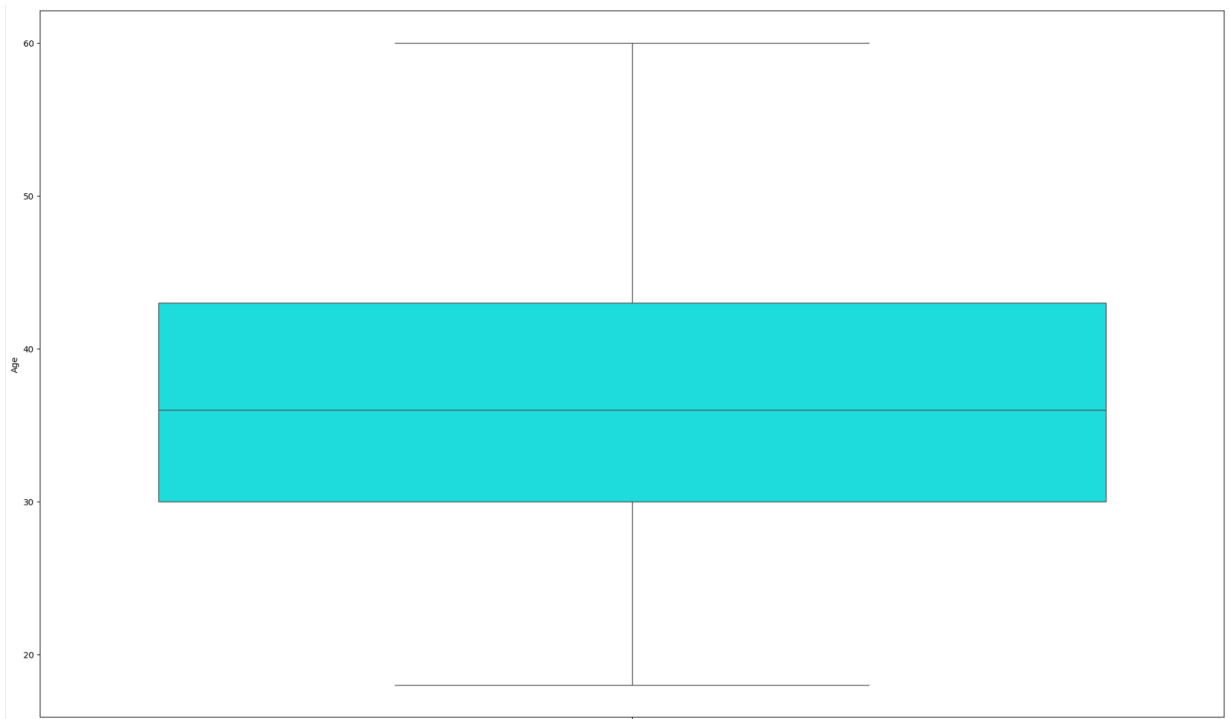
```

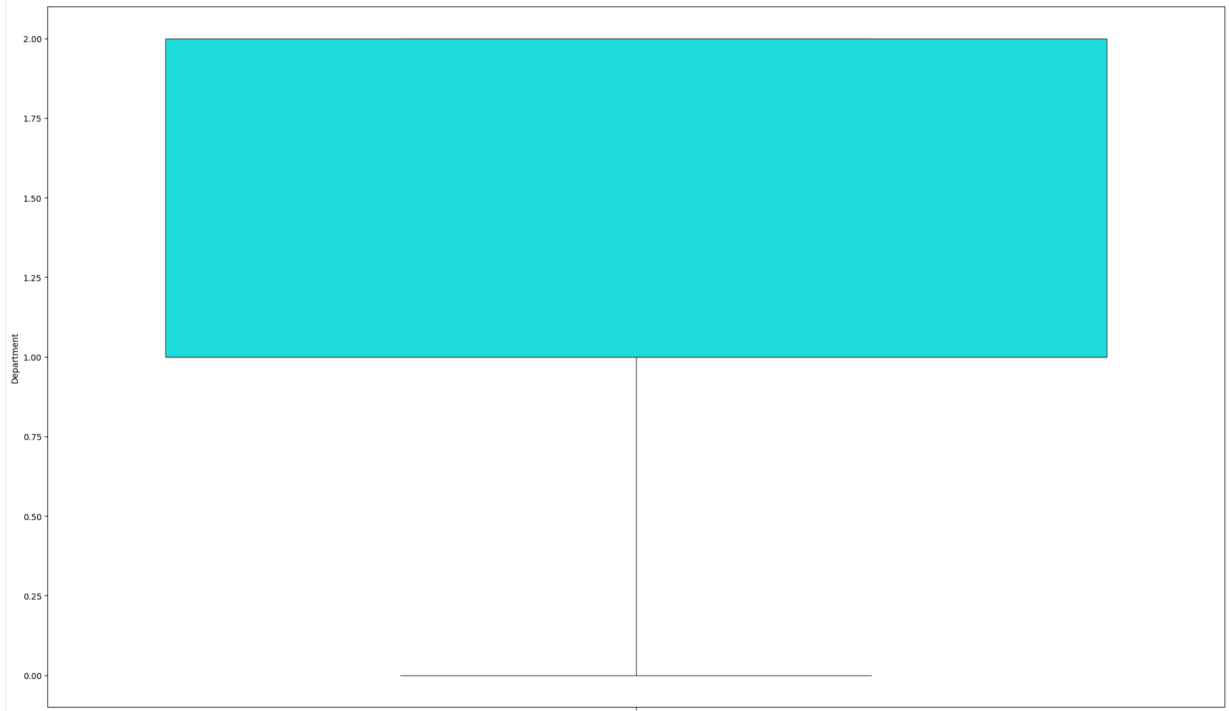
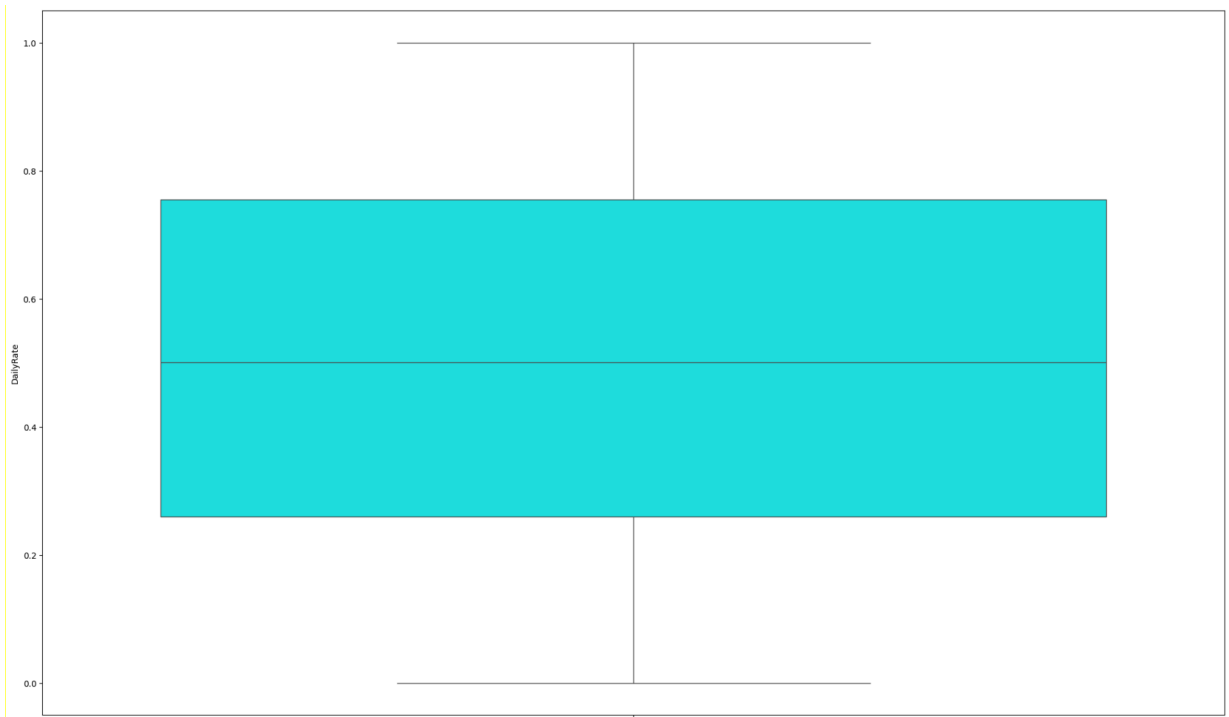
```

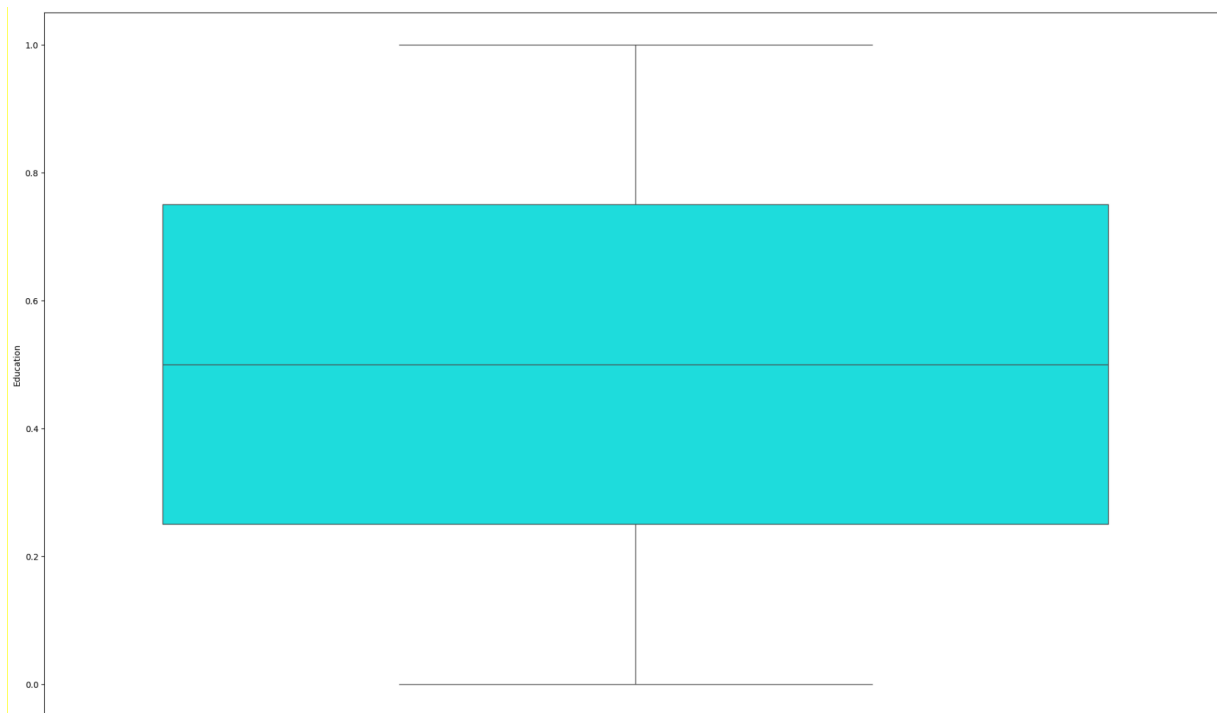
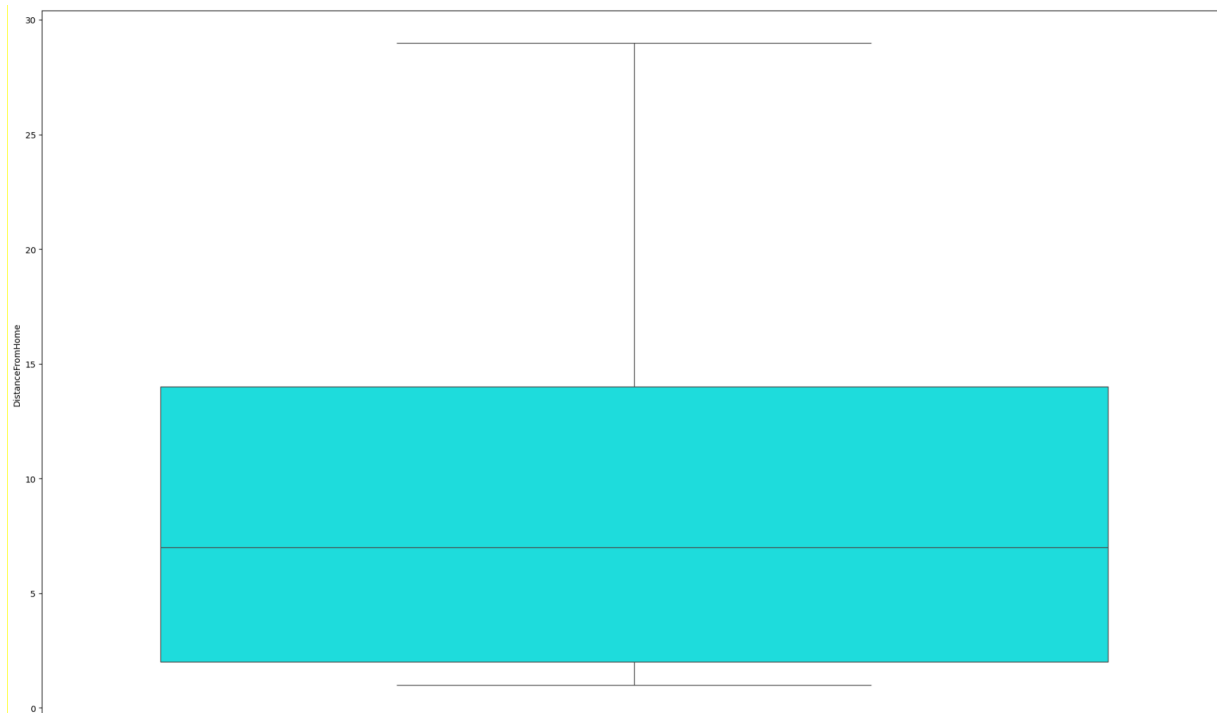
In [ ]: for i in df:
        plt.figure(figsize=(25,15),facecolor='white', edgecolor = 'yellow', linewidth = 0
        sns.boxplot(y=i,data=df, color = 'cyan')
        plt.show()

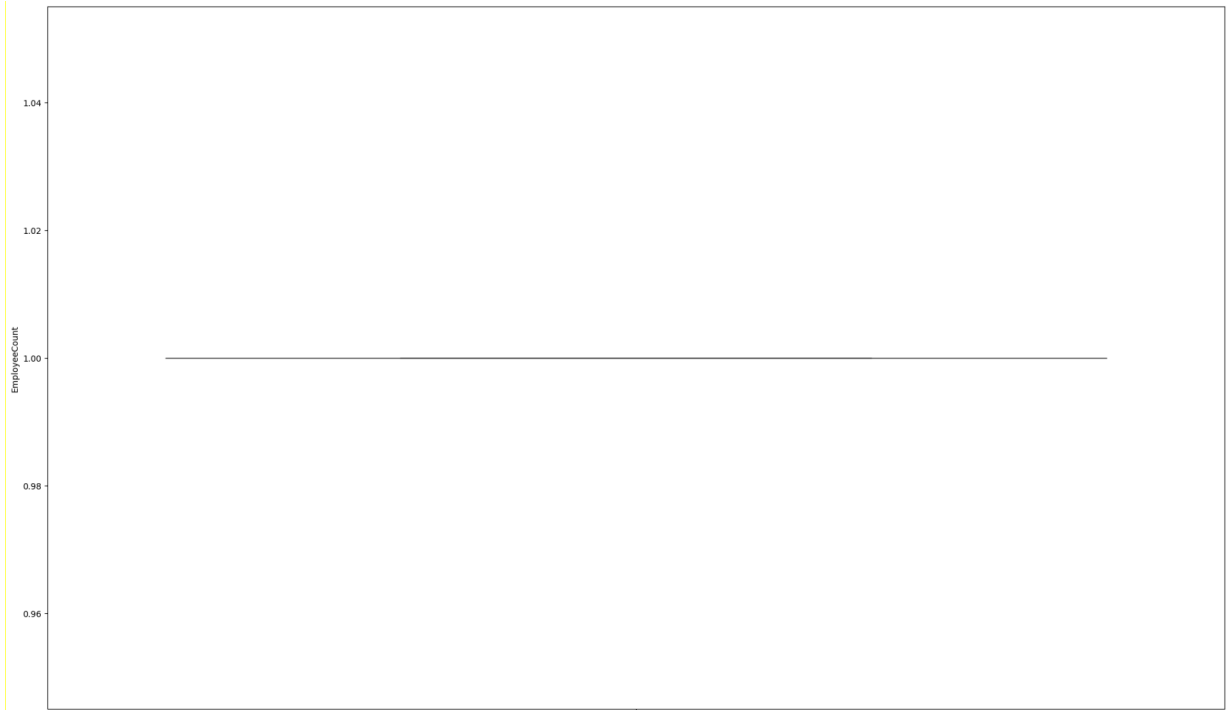
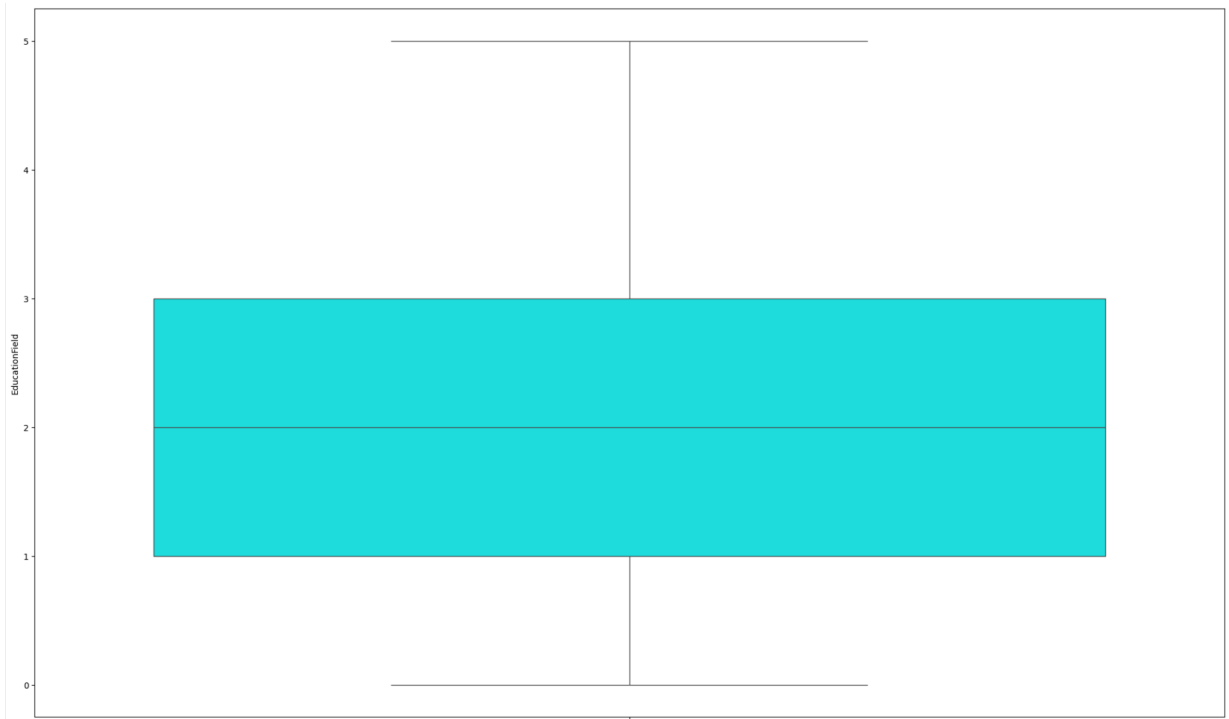
```

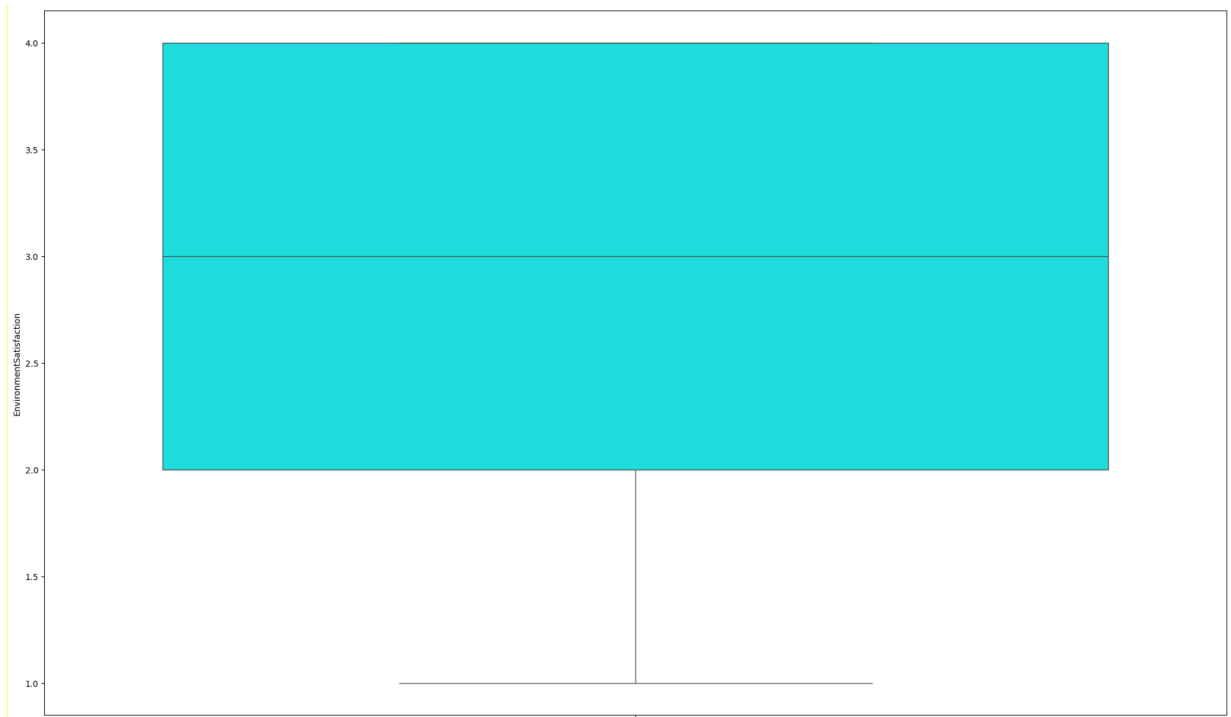


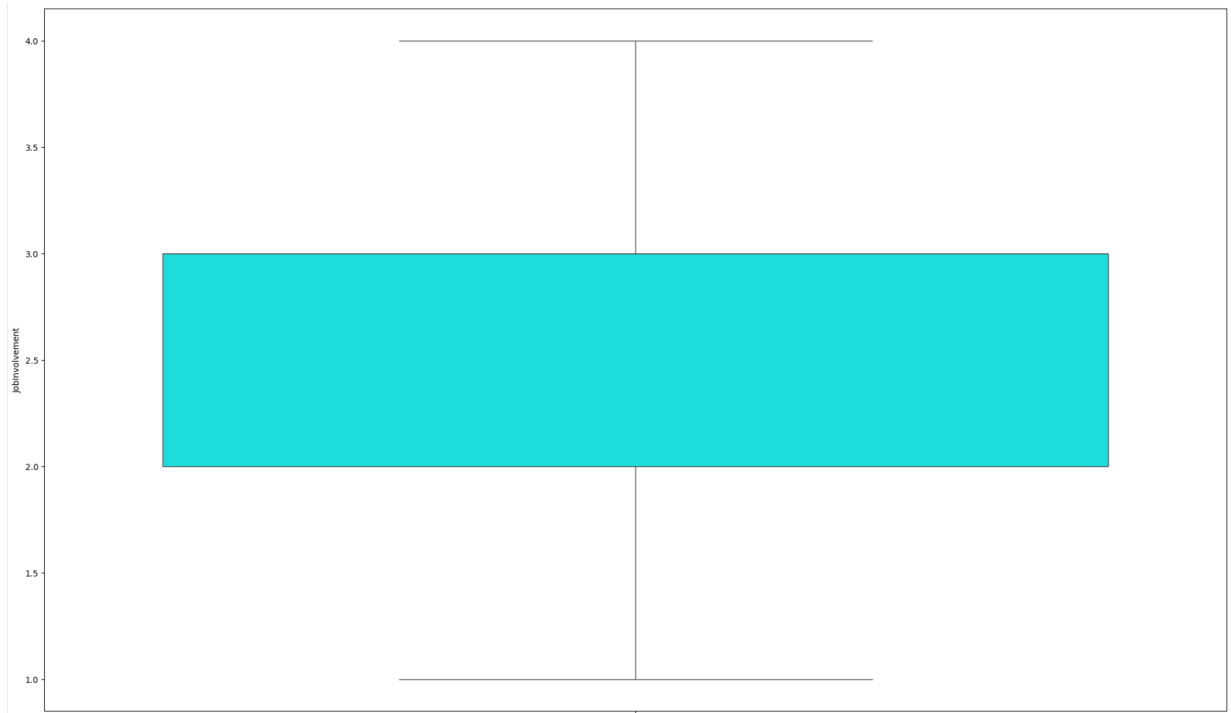
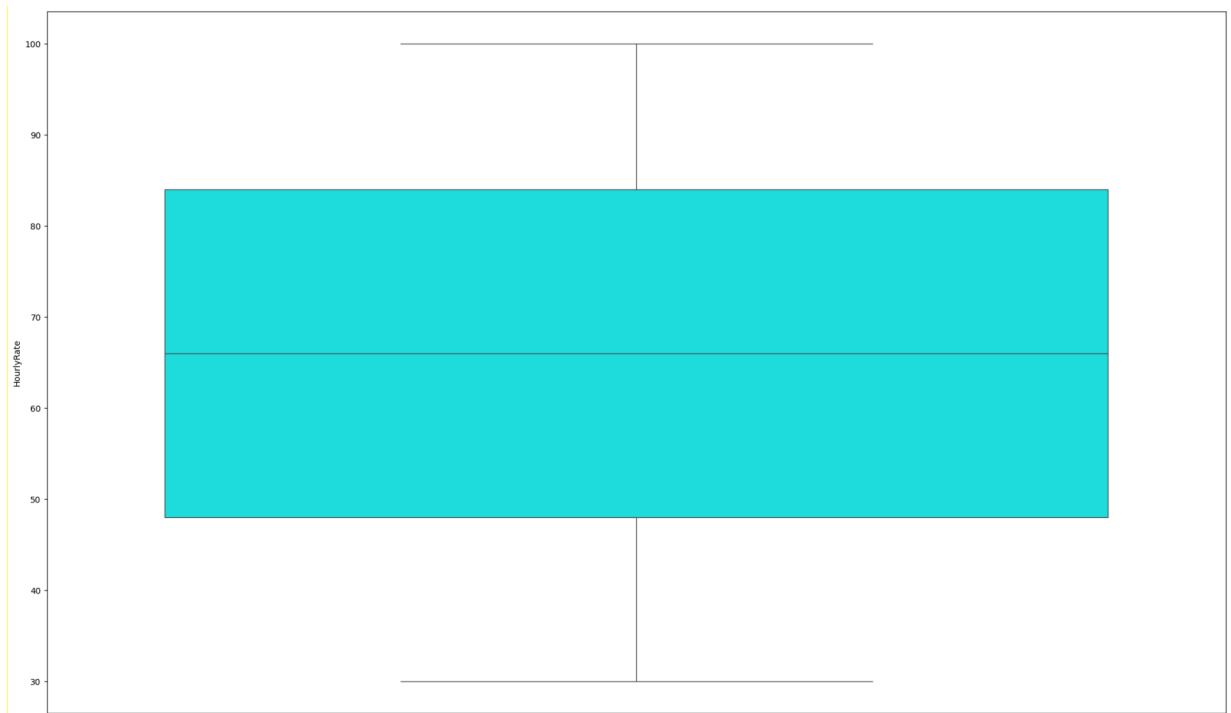


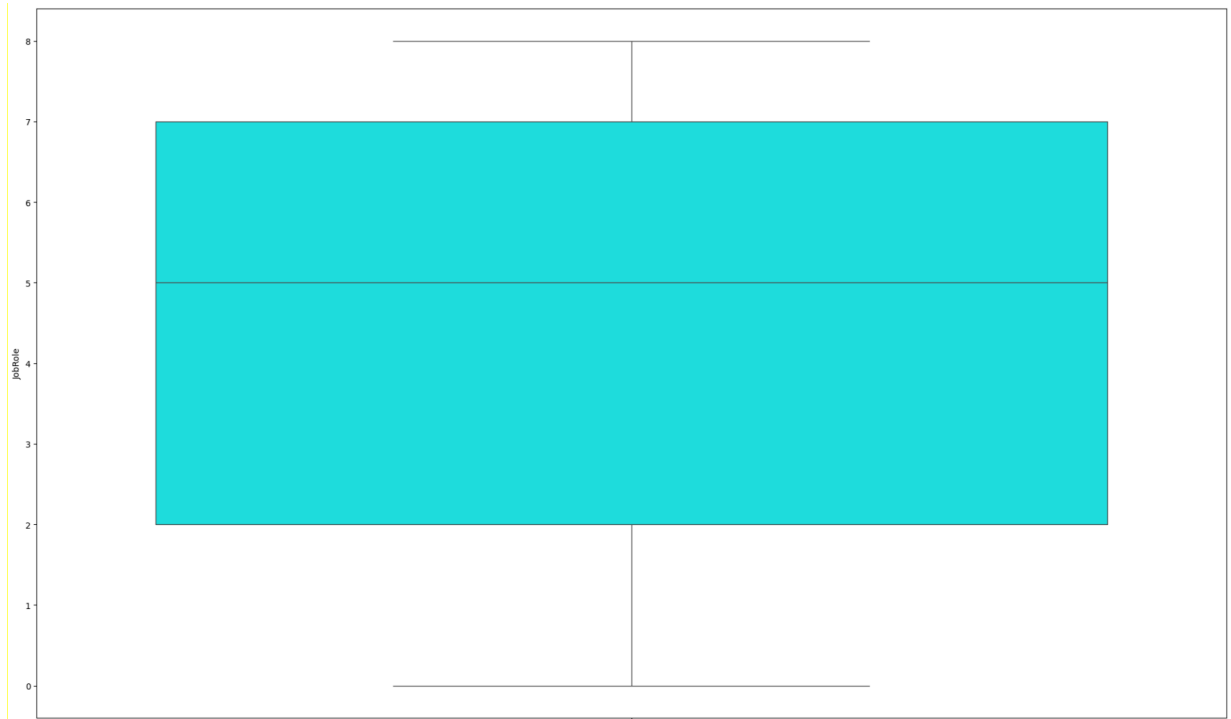
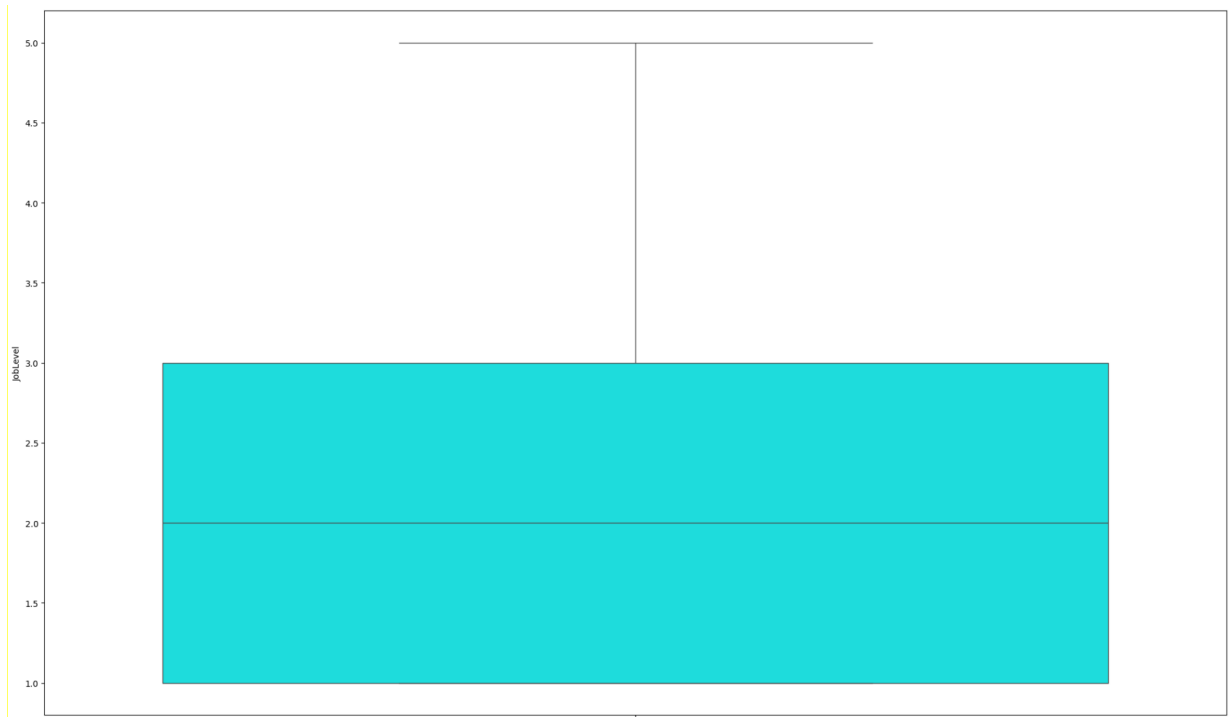


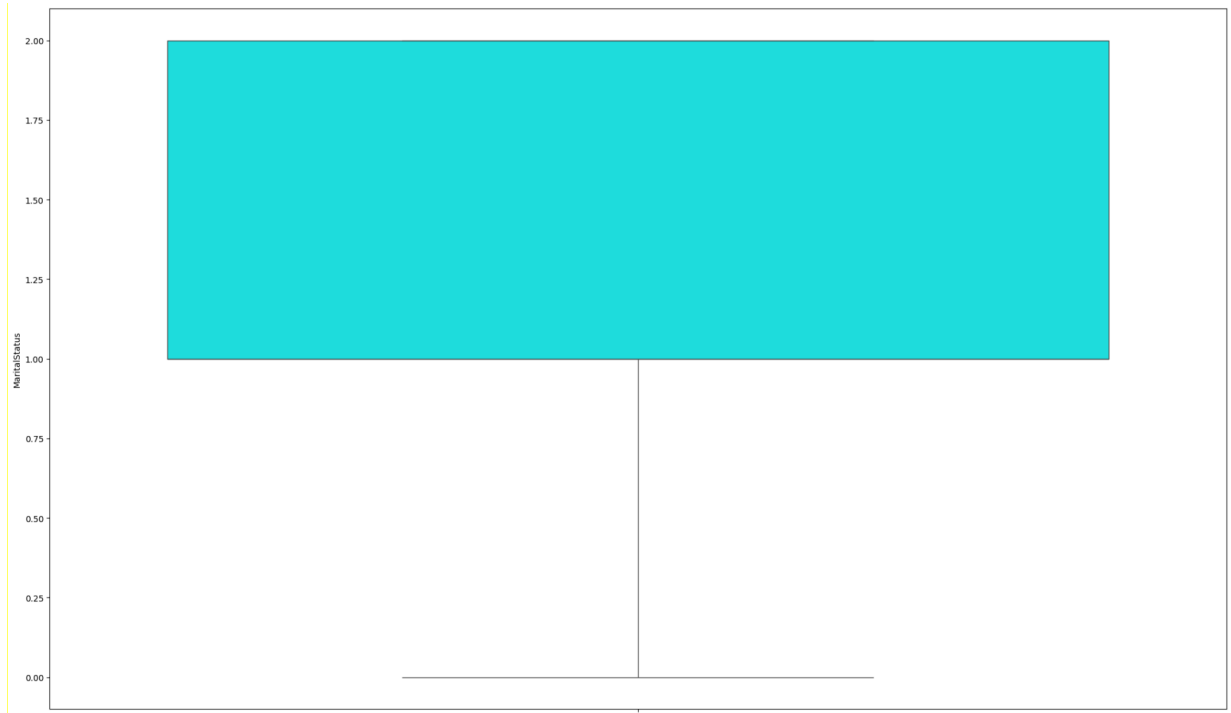
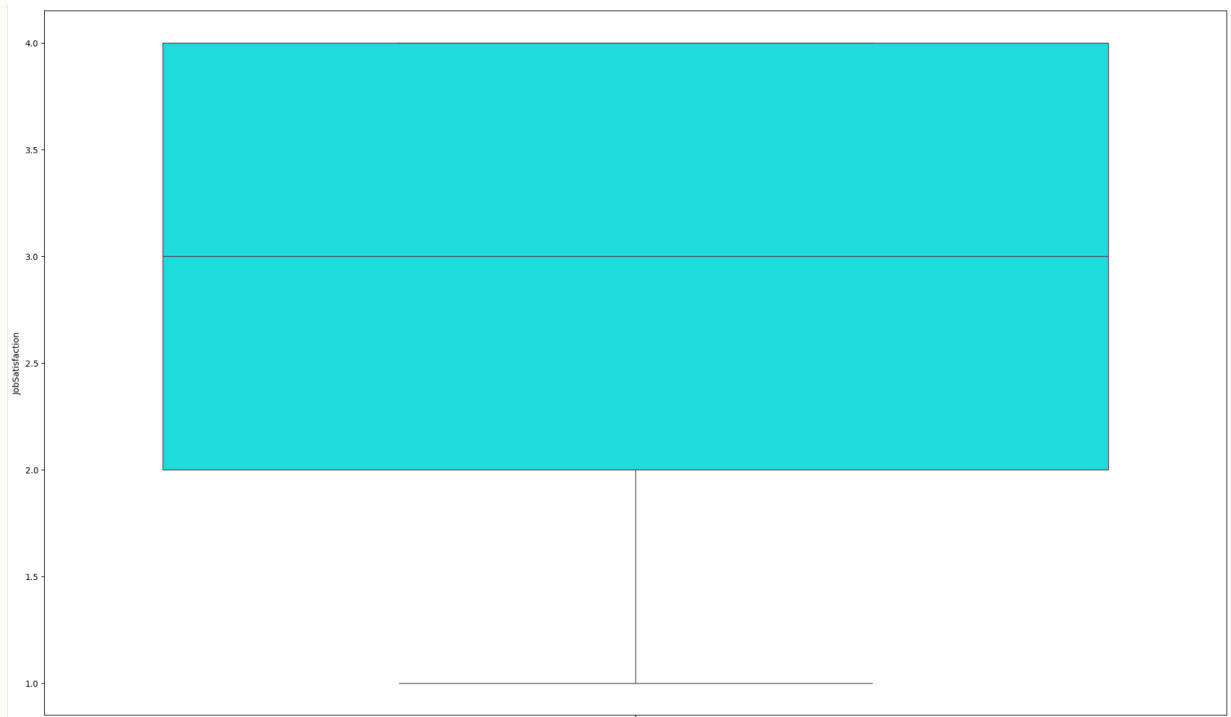


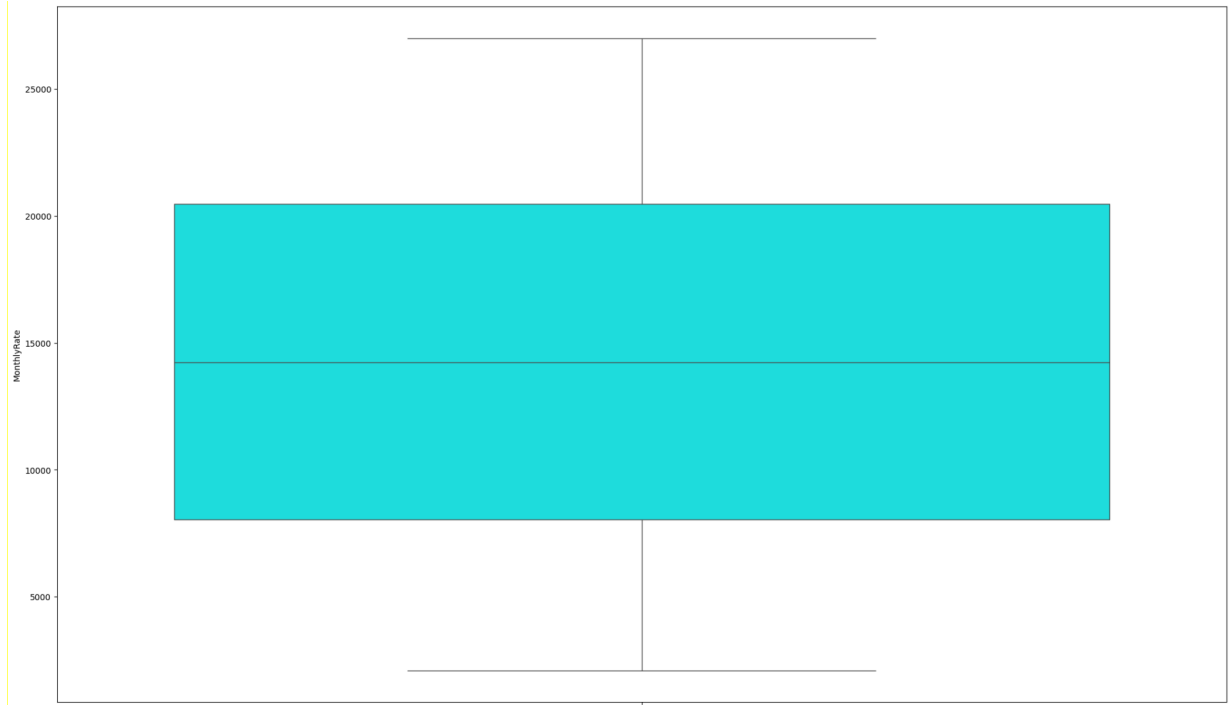
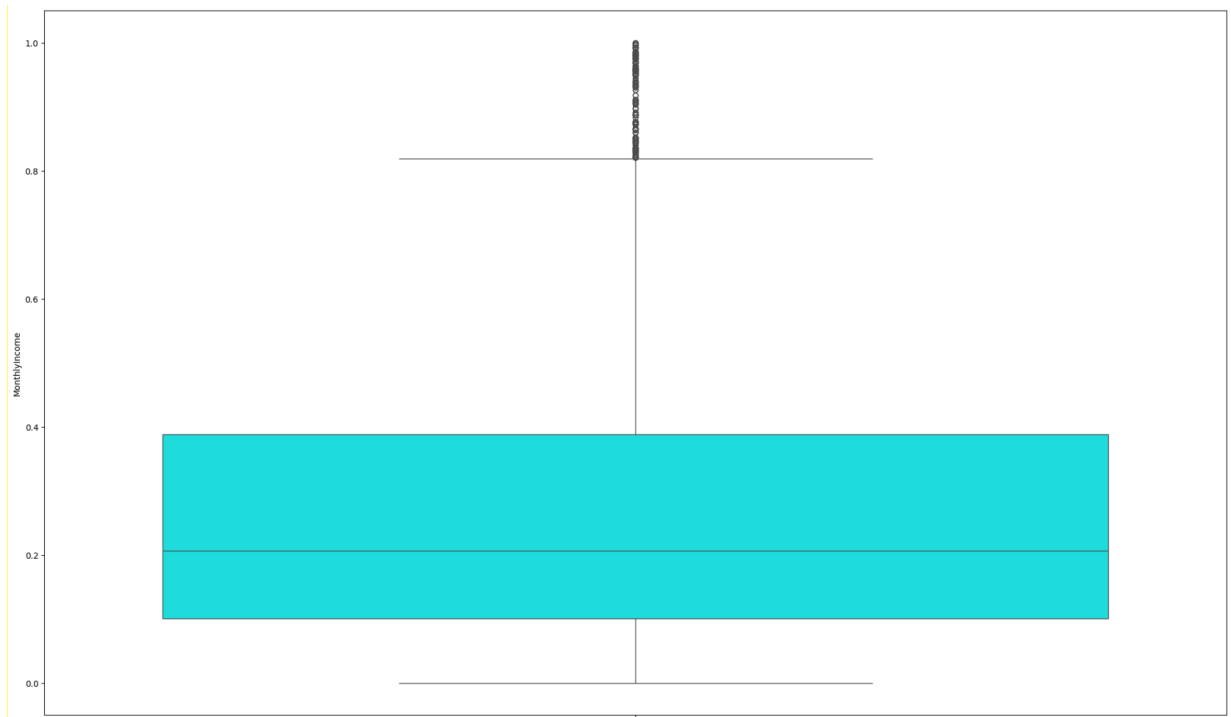


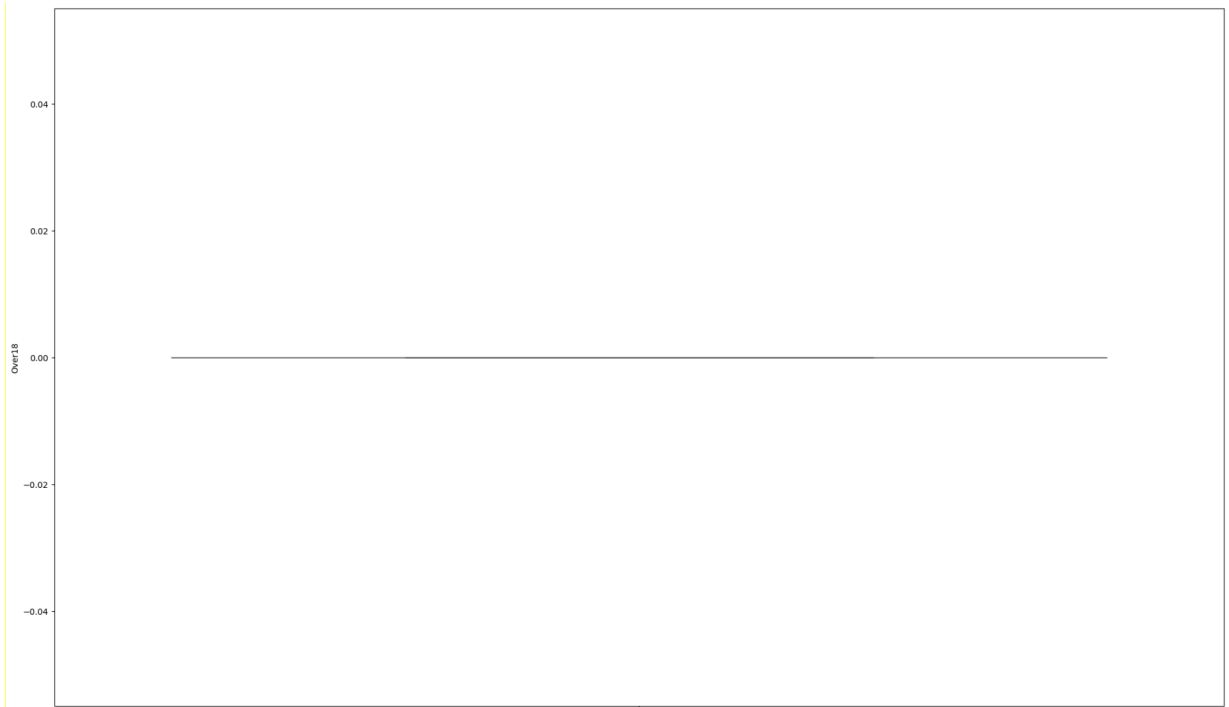
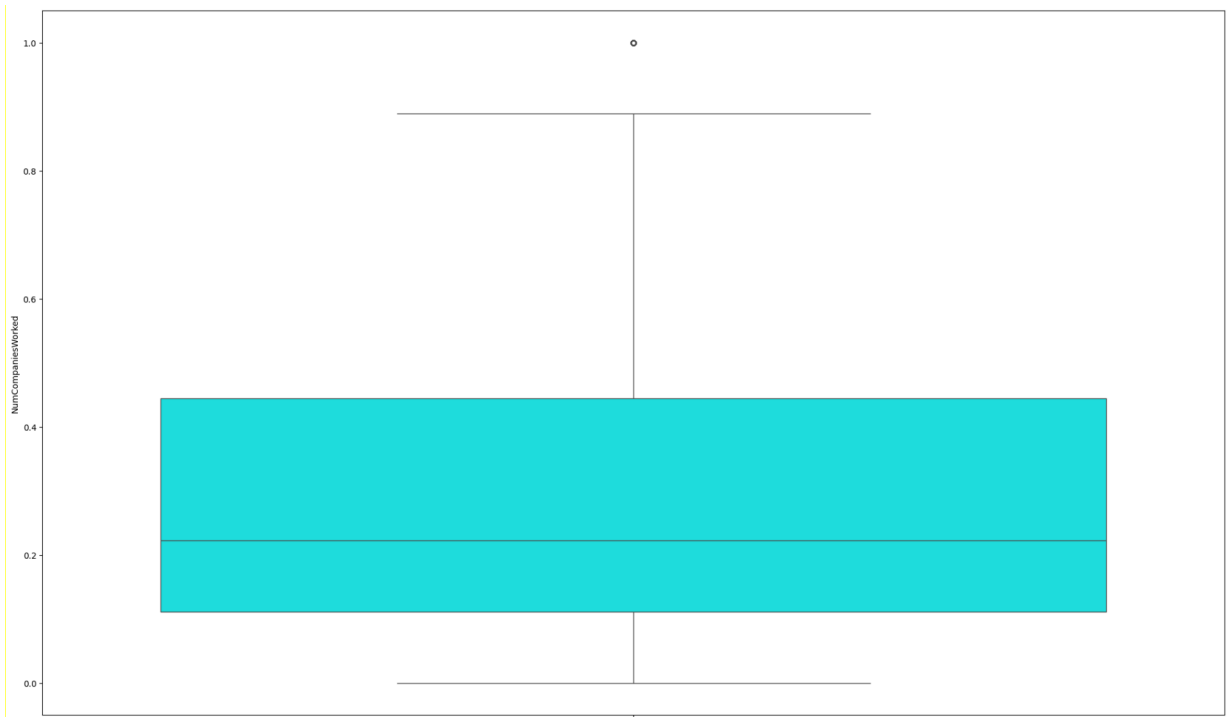


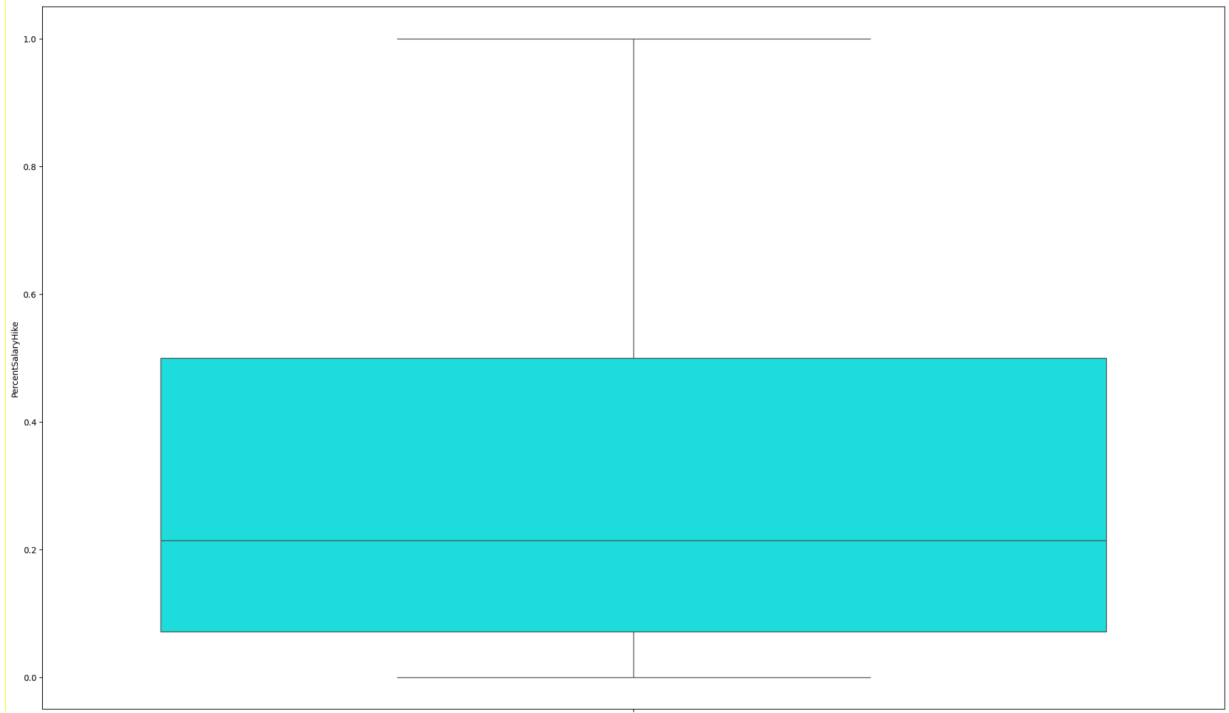


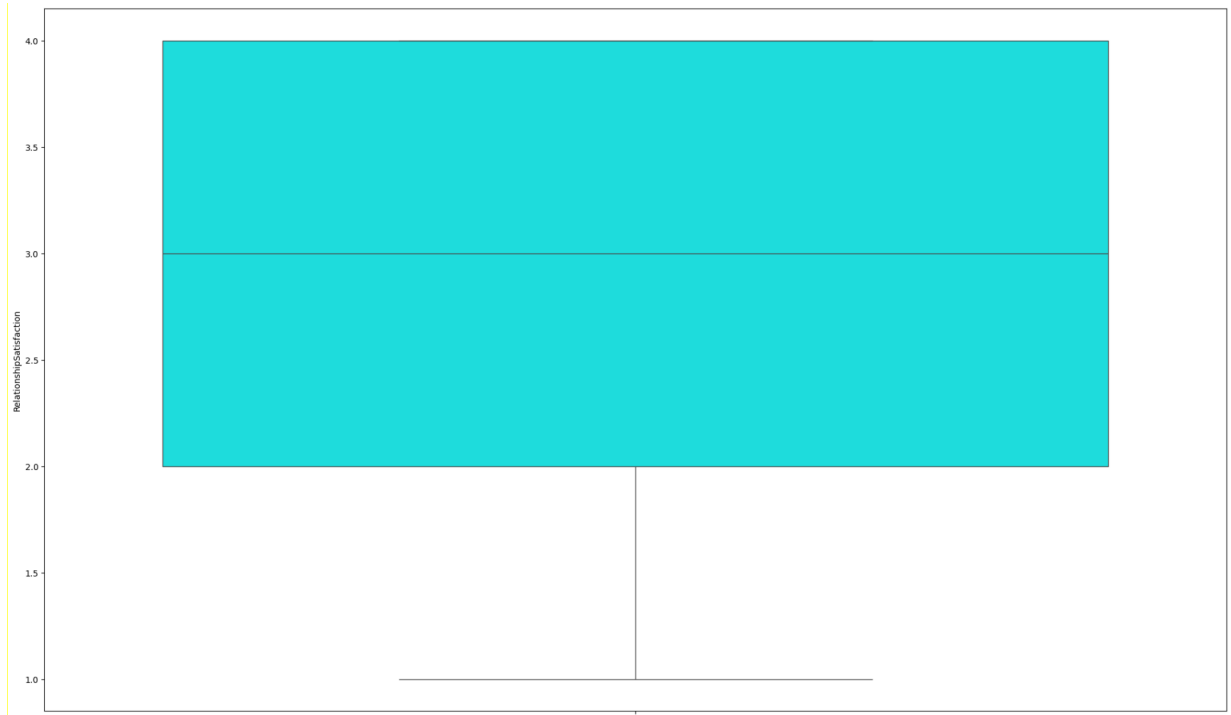
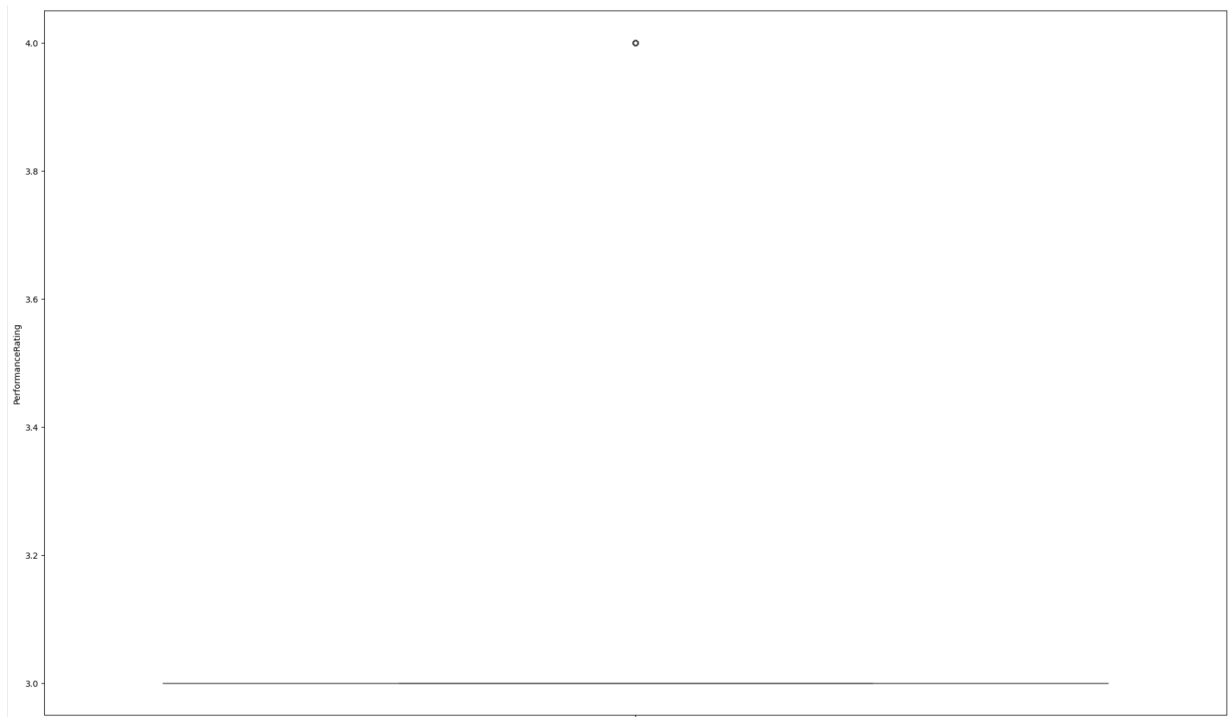


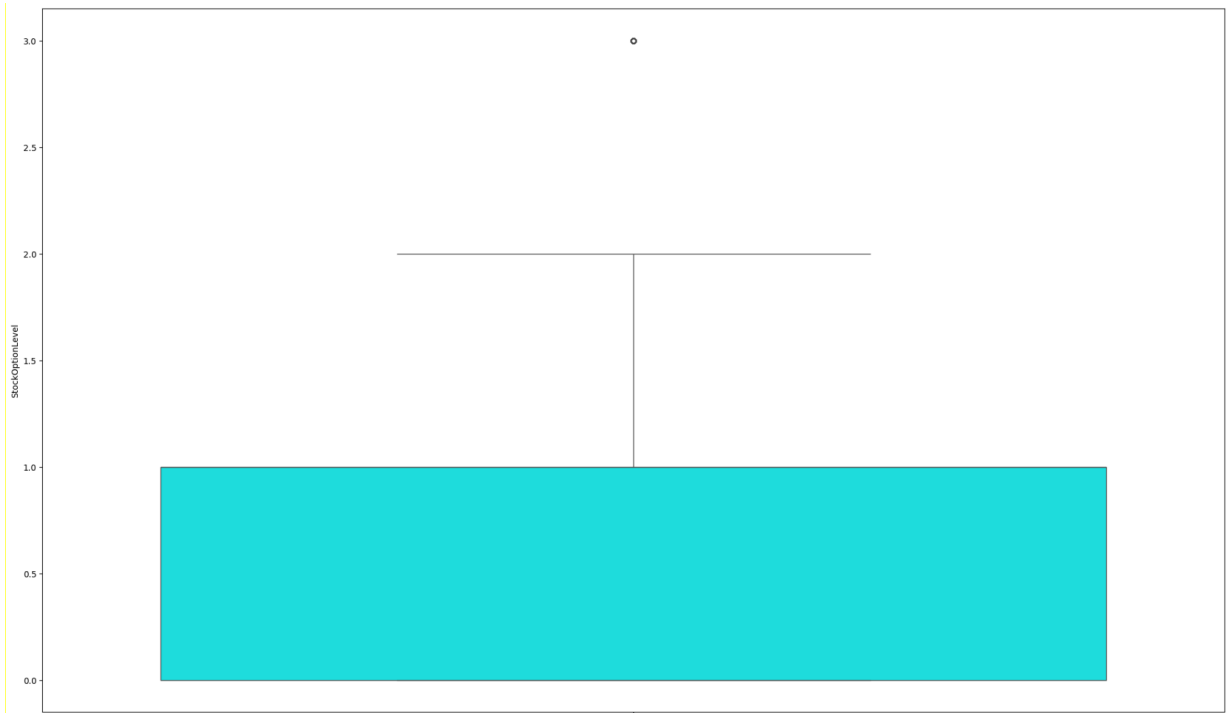
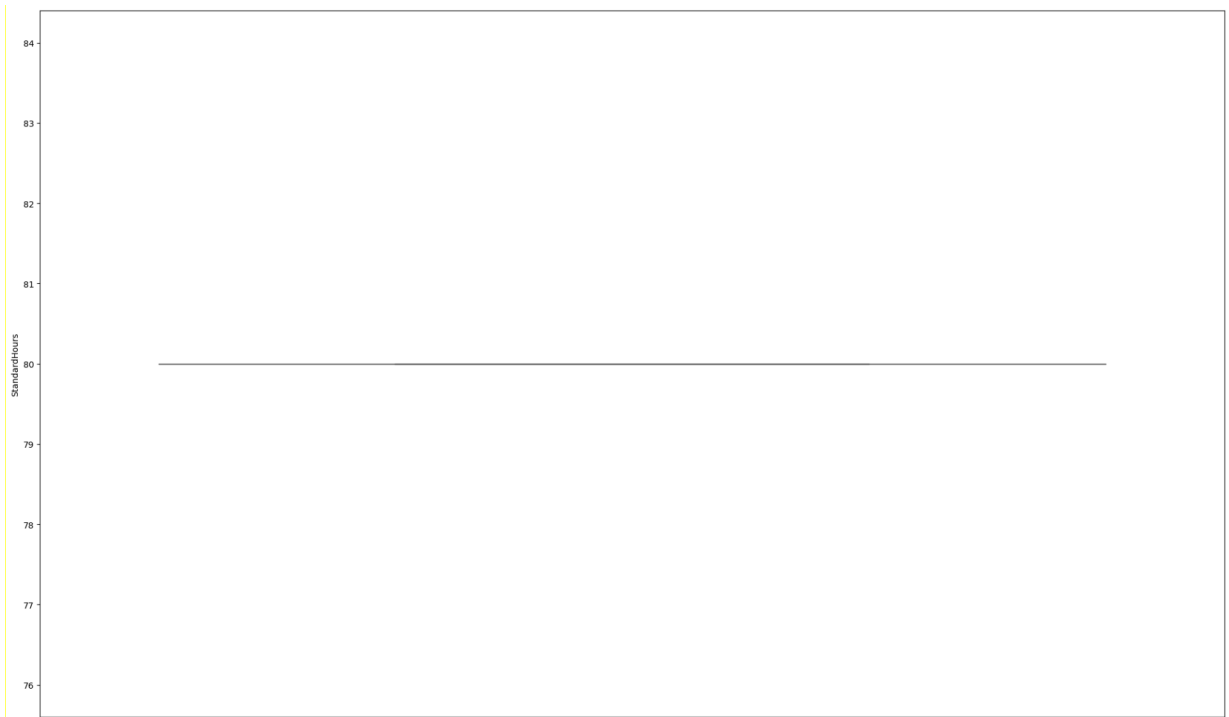


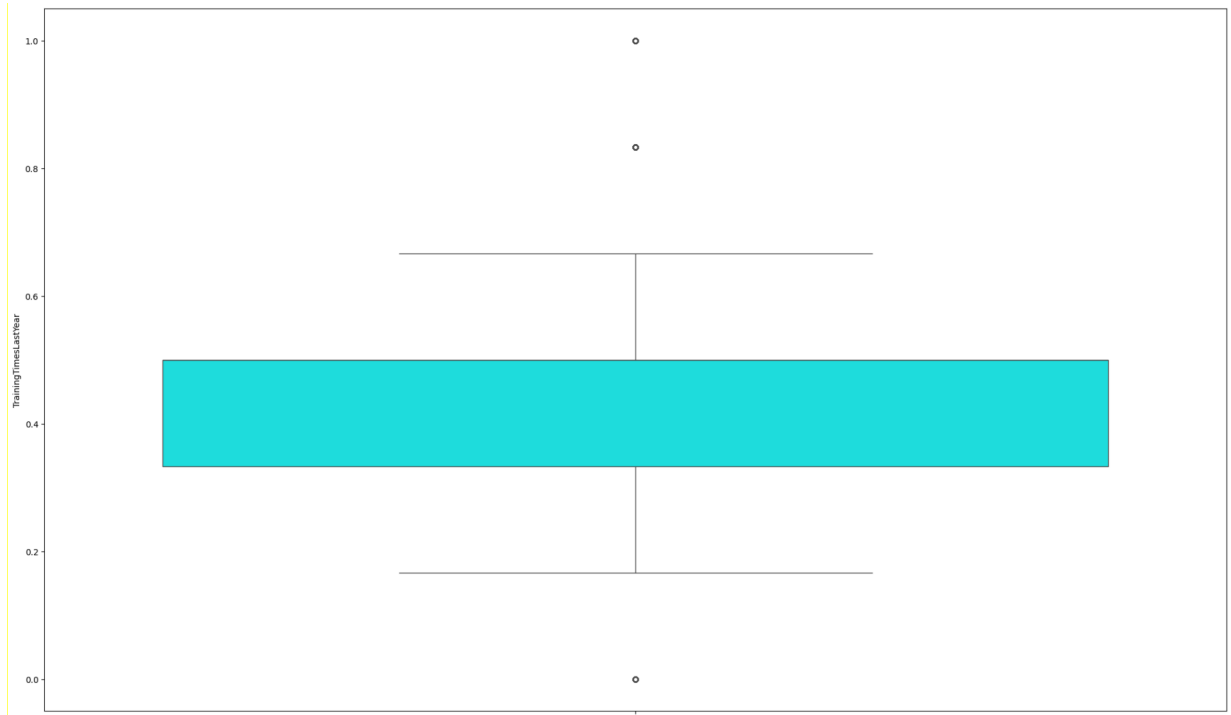
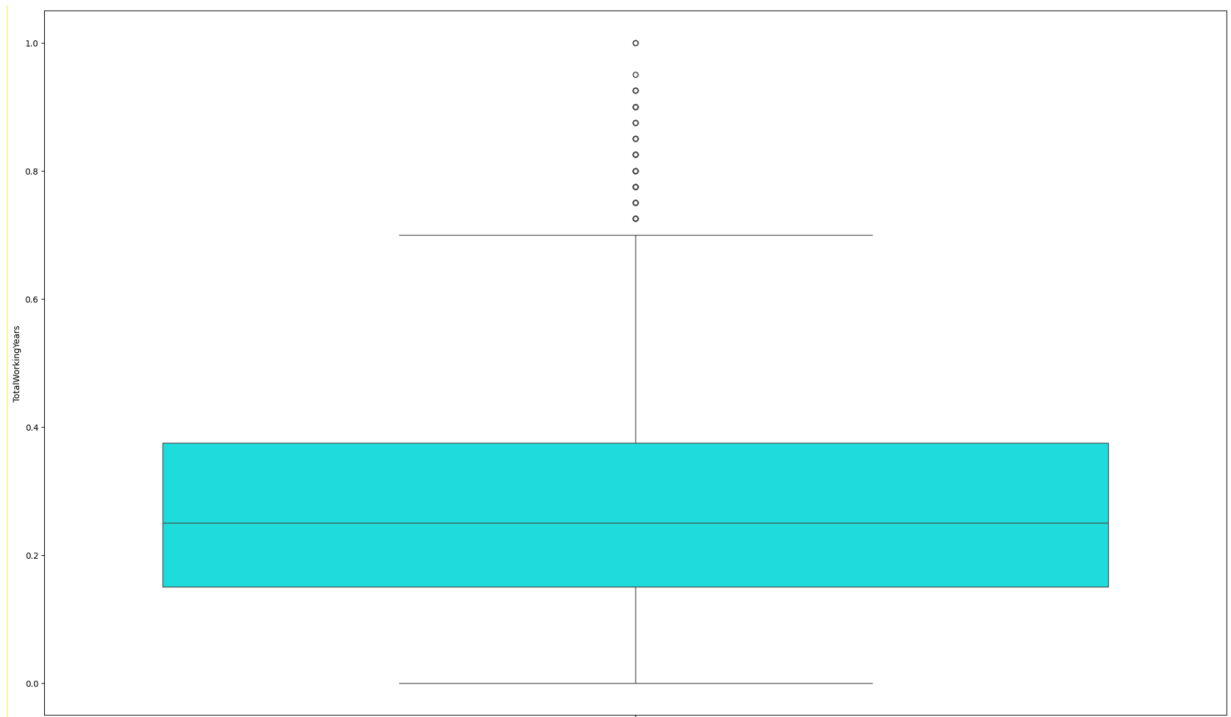


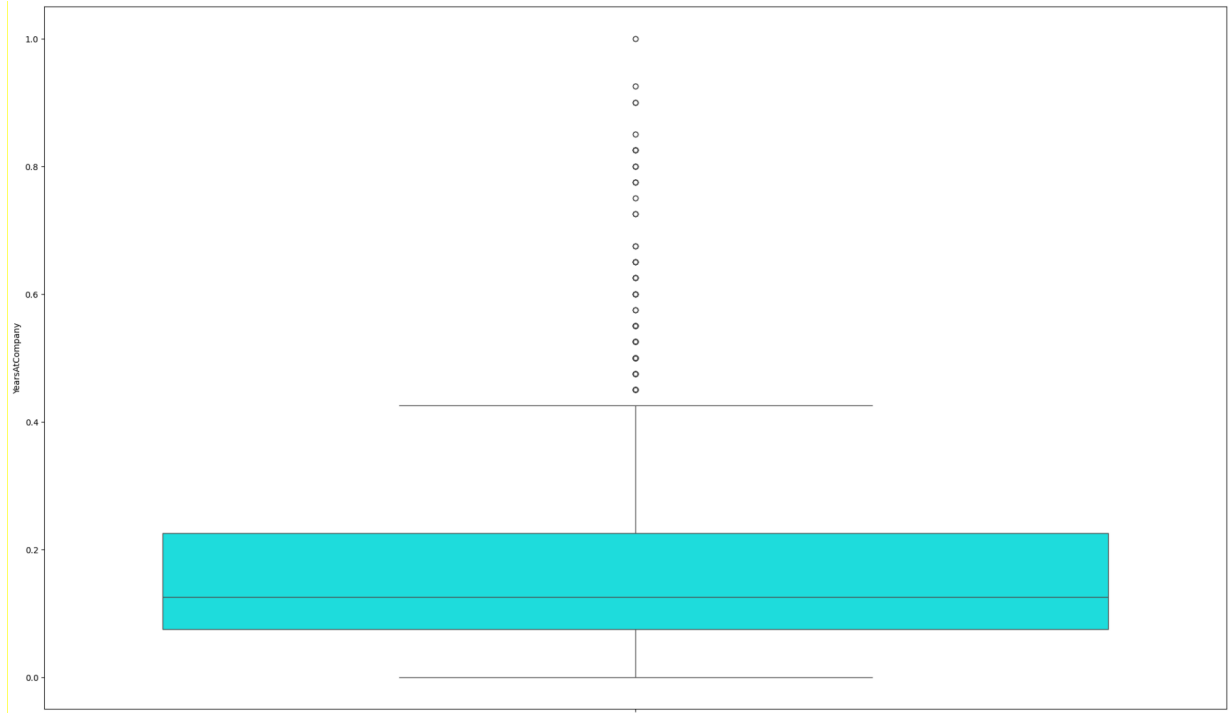
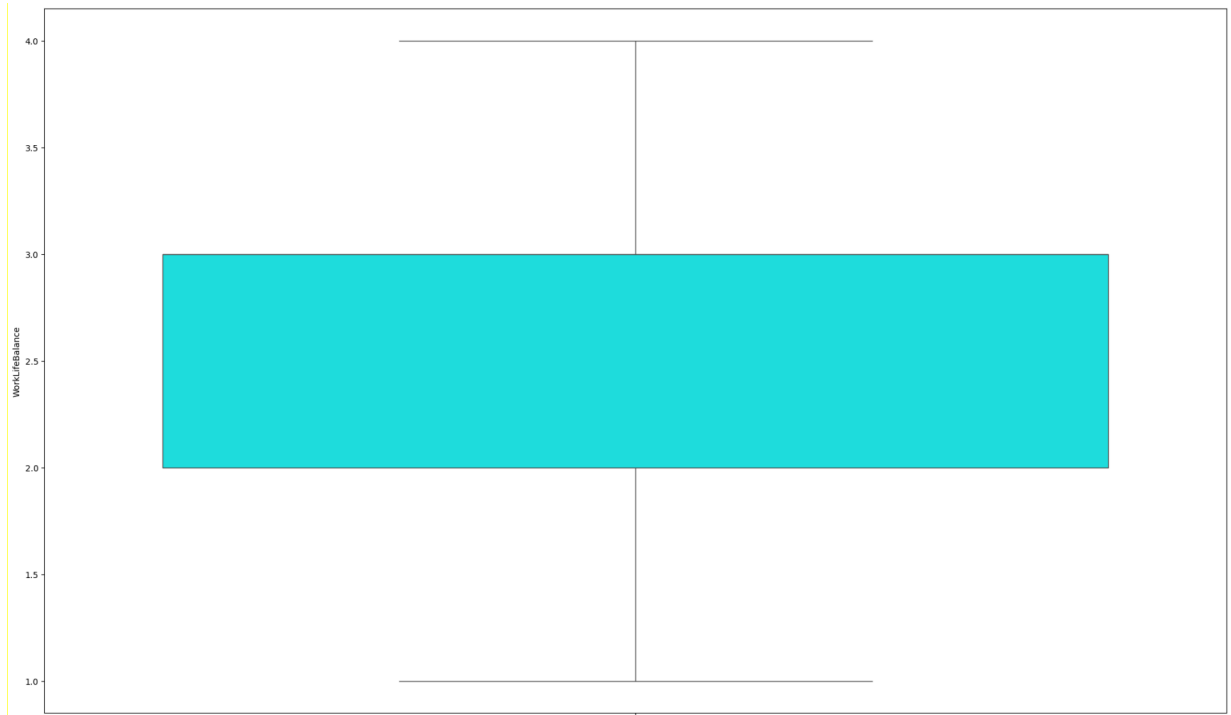


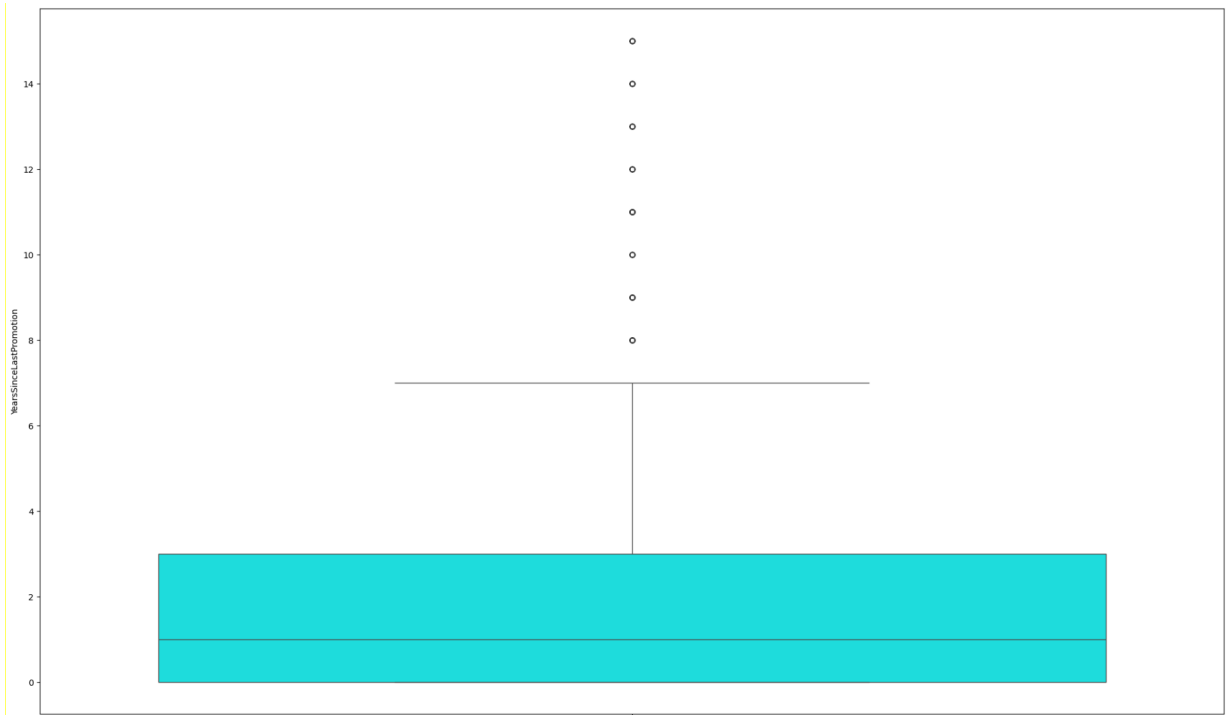
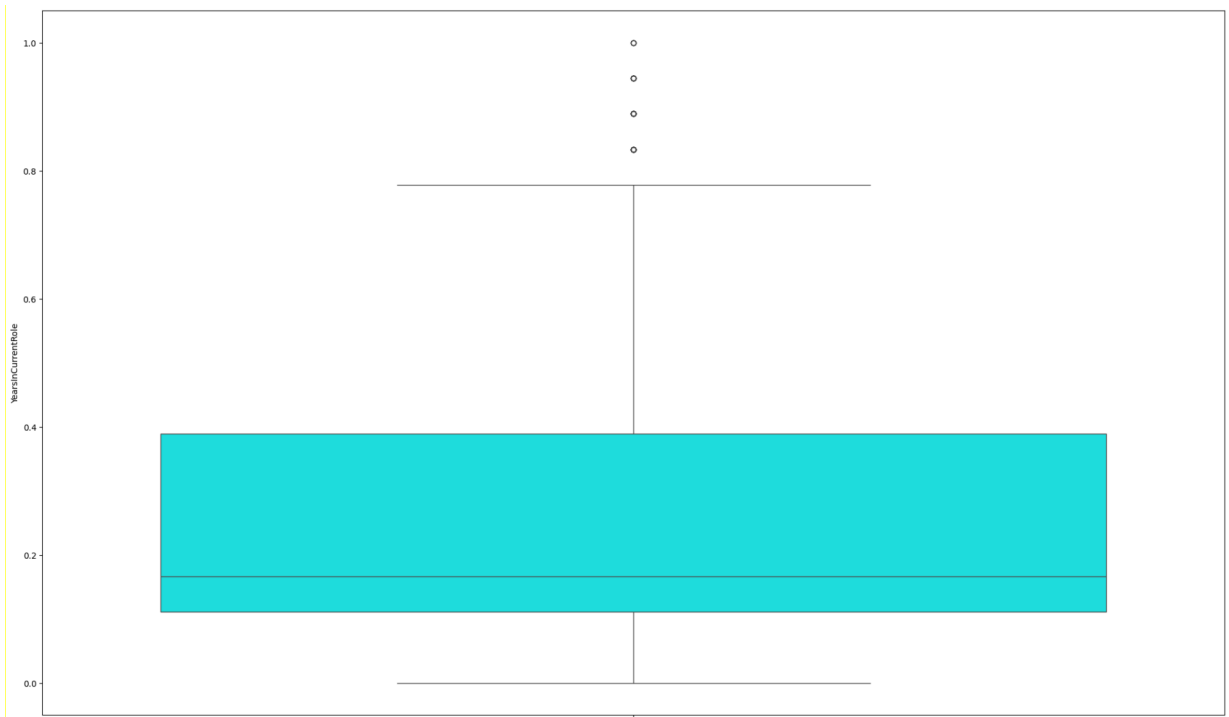


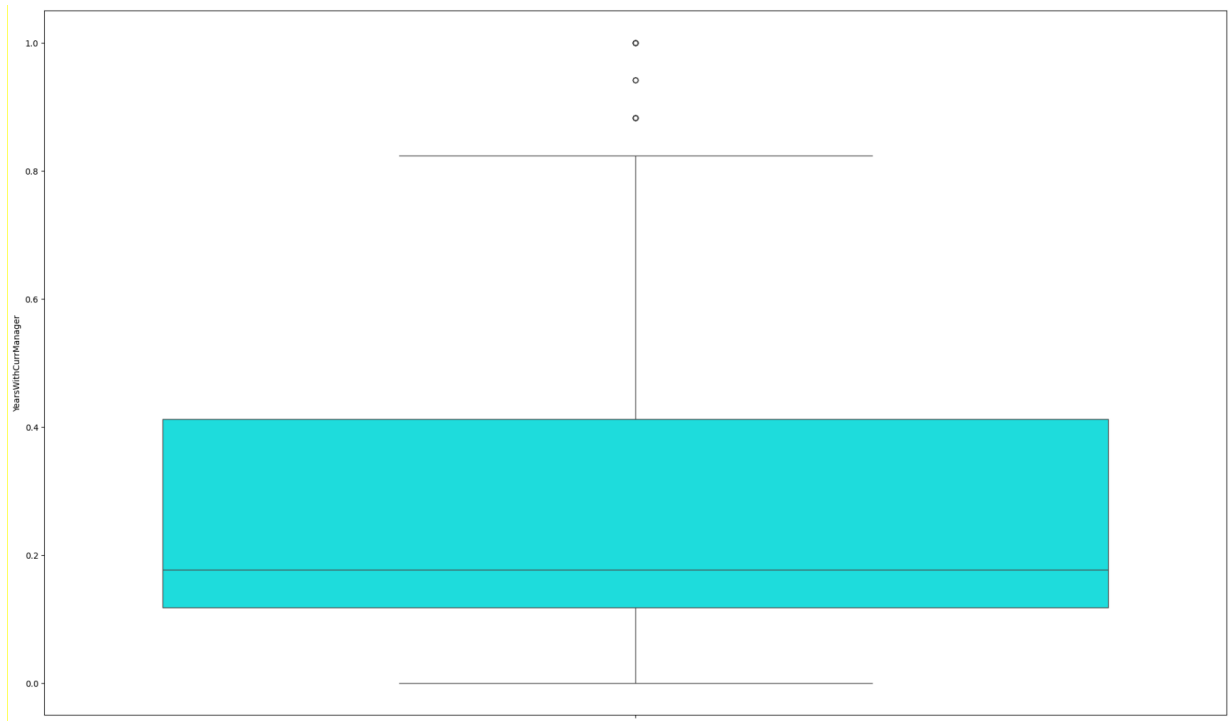












```
In [ ]: #converting categorical datat to numerical data for heat map analysis
```

```
In [ ]: df.Attrition.unique()
```

```
Out[ ]: array(['Yes', 'No'], dtype=object)
```

```
In [ ]: df.Gender.unique()
```

```
Out[ ]: array(['Female', 'Male'], dtype=object)
```

```
In [ ]: from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
```

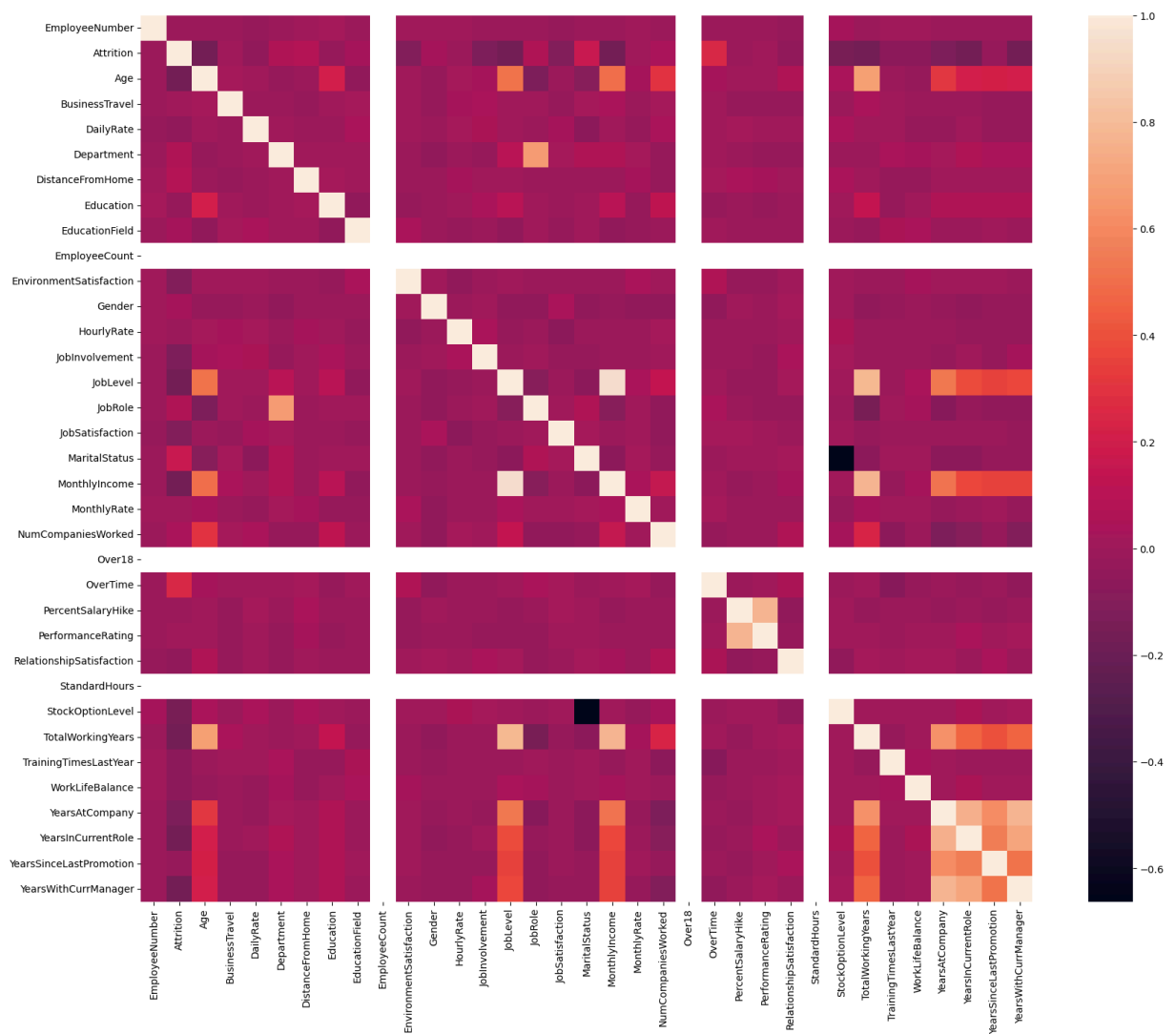
```
In [ ]: df['Attrition']=le.fit_transform(df['Attrition'])
```

```
In [ ]: df['Gender']=le.fit_transform(df['Gender'])
```

Heat map representation

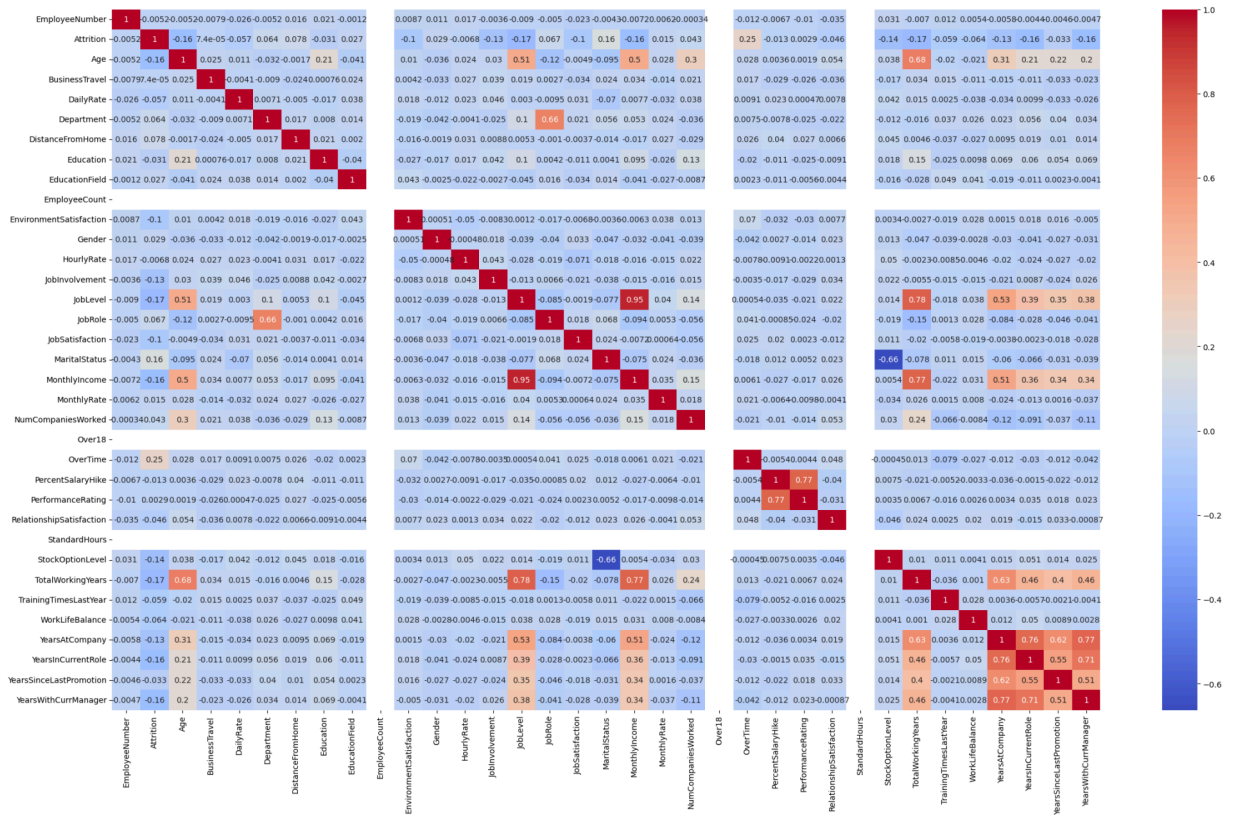
```
In [ ]: plt.figure(figsize=(20,16))
sns.heatmap(df.corr())
```

```
Out[ ]: <Axes: >
```



```
In [ ]: plt.figure(figsize=(28,16))
sns.heatmap(df.corr(),cmap = 'coolwarm',annot = True)
```

```
Out[ ]: <Axes: >
```



- hike in salary seems to favour one's performance rating
- joblevel and total working years directly affects monthly income
- joblevel seems to increase with total working years
- years under the same manager seems to delay once promotion
- different departments have their clutches around coveted jobroles and some departments will lead to a more prominent role than others
- stockoption level decreases with married individuals

Building the Model

```
In [ ]: # preparing model considering all the variables
import statsmodels.formula.api as smf # for regression model

m1 = smf.ols('Attrition ~ Age + BusinessTravel+DailyRate+ Department+
# Summary
m1.summary()
```

Out[]:

OLS Regression Results

Dep. Variable:	Attrition	R-squared:	0.215
Model:	OLS	Adj. R-squared:	0.207
Method:	Least Squares	F-statistic:	26.51
Date:	Thu, 11 Jul 2024	Prob (F-statistic):	2.30e-129
Time:	13:54:14	Log-Likelihood:	-875.27
No. Observations:	2940	AIC:	1813.
Df Residuals:	2909	BIC:	1998.
Df Model:	30		
Covariance Type:	nonrobust		
	coef	std err	t P> t [0.025 0.975]
Intercept	0.0001	1.61e-05	6.426 0.000 7.19e-05 0.000
Age	-0.0039	0.001	-4.120 0.000 -0.006 -0.002
BusinessTravel	-6.62e-05	0.009	-0.007 0.994 -0.018 0.018
DailyRate	-0.0452	0.021	-2.135 0.033 -0.087 -0.004
Department	0.0563	0.016	3.530 0.000 0.025 0.088
DistanceFromHome	0.0035	0.001	4.628 0.000 0.002 0.005
Education	-0.0009	0.024	-0.038 0.970 -0.049 0.047
EducationField	0.0055	0.005	1.201 0.230 -0.003 0.014
EmployeeCount	0.0001	1.61e-05	6.426 0.000 7.19e-05 0.000
EnvironmentSatisfaction	-0.0409	0.006	-7.336 0.000 -0.052 -0.030
Gender	0.0355	0.012	2.848 0.004 0.011 0.060
HourlyRate	-0.0003	0.000	-0.851 0.395 -0.001 0.000
JobInvolvement	-0.0597	0.009	-6.955 0.000 -0.077 -0.043
JobLevel	-0.0335	0.019	-1.786 0.074 -0.070 0.003
JobRole	-0.0029	0.003	-0.870 0.384 -0.010 0.004
JobSatisfaction	-0.0383	0.006	-6.921 0.000 -0.049 -0.027
MaritalStatus	0.0541	0.011	4.820 0.000 0.032 0.076
MonthlyIncome	0.0092	0.081	0.114 0.909 -0.150 0.168
MonthlyRate	4.46e-07	8.56e-07	0.521 0.602 -1.23e-06 2.12e-06
NumCompaniesWorked	0.1474	0.024	6.026 0.000 0.099 0.195

Over18	-3.828e-17	3.29e-17	-1.162	0.245	-1.03e-16	2.63e-17
OverTime	0.2106	0.014	15.482	0.000	0.184	0.237
PercentSalaryHike	-0.0601	0.037	-1.637	0.102	-0.132	0.012
PerformanceRating	0.0301	0.027	1.132	0.258	-0.022	0.082
RelationshipSatisfaction	-0.0216	0.006	-3.822	0.000	-0.033	-0.011
StandardHours	0.0083	0.001	6.426	0.000	0.006	0.011
StockOptionLevel	-0.0247	0.010	-2.577	0.010	-0.043	-0.006
TotalWorkingYears	-0.1186	0.068	-1.738	0.082	-0.252	0.015
TrainingTimesLastYear	-0.0710	0.028	-2.495	0.013	-0.127	-0.015
WorkLifeBalance	-0.0273	0.009	-3.164	0.002	-0.044	-0.010
YearsAtCompany	0.2414	0.085	2.852	0.004	0.075	0.407
YearsInCurrentRole	-0.1900	0.050	-3.831	0.000	-0.287	-0.093
YearsSinceLastPromotion	0.0113	0.002	4.629	0.000	0.006	0.016
YearsWithCurrManager	-0.1804	0.048	-3.755	0.000	-0.275	-0.086
Omnibus:	586.963	Durbin-Watson:	1.917			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	996.046			
Skew:	1.332	Prob(JB):	5.14e-217			
Kurtosis:	4.017	Cond. No.	7.45e+18			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 1.35e-26. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```
In [ ]: #identify and mitigate multicollinearity
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Calculate VIF for each feature
X = df.drop('Attrition', axis=1) # Drop the target variable
vif_data = pd.DataFrame()
vif_data['Feature'] = X.columns
vif_data['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])

print(vif_data)
```

	Feature	VIF
0	EmployeeNumber	1.005566
1	Age	2.054189
2	BusinessTravel	1.016513
3	DailyRate	1.024596
4	Department	1.942153
5	DistanceFromHome	1.017372
6	Education	1.063961
7	EducationField	1.016237
8	EmployeeCount	0.000000
9	EnvironmentSatisfaction	1.017638
10	Gender	1.019538
11	HourlyRate	1.021343
12	JobInvolvement	1.020812
13	JobLevel	11.822281
14	JobRole	1.894286
15	JobSatisfaction	1.021250
16	MaritalStatus	1.841729
17	MonthlyIncome	11.053224
18	MonthlyRate	1.015649
19	NumCompaniesWorked	1.261957
20	Over18	NaN
21	OverTime	1.028655
22	PercentSalaryHike	2.521577
23	PerformanceRating	2.519515
24	RelationshipSatisfaction	1.022005
25	StandardHours	0.000000
26	StockOptionLevel	1.821789
27	TotalWorkingYears	4.824452
28	TrainingTimesLastYear	1.023864
29	WorkLifeBalance	1.018531
30	YearsAtCompany	4.601976
31	YearsInCurrentRole	2.728268
32	YearsSinceLastPromotion	1.678884
33	YearsWithCurrManager	2.782918

```

/usr/local/lib/python3.10/dist-packages/statsmodels/regression/linear_model.py:1783:
RuntimeWarning: divide by zero encountered in scalar divide
    return 1 - self.ssr/self.centered_tss
/usr/local/lib/python3.10/dist-packages/statsmodels/regression/linear_model.py:1783:
RuntimeWarning: invalid value encountered in scalar divide
    return 1 - self.ssr/self.centered_tss

```

```
In [ ]: df.corr()
```

Out[]:

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate
EmployeeNumber	1.000000	-0.005184	-0.005175	-0.007905	-0.025742
Attrition	-0.005184	1.000000	-0.159205	0.000074	-0.056652
Age	-0.005175	-0.159205	1.000000	0.024751	0.010661
BusinessTravel	-0.007905	0.000074	0.024751	1.000000	-0.004086
DailyRate	-0.025742	-0.056652	0.010661	-0.004086	1.000000
Department	-0.005183	0.063991	-0.031882	-0.009044	0.007109
DistanceFromHome	0.016464	0.077924	-0.001686	-0.024469	-0.004985
Education	0.020950	-0.031373	0.208034	0.000757	-0.016806
EducationField	-0.001229	0.026846	-0.040873	0.023724	0.037709
EmployeeCount	NaN	NaN	NaN	NaN	NaN
EnvironmentSatisfaction	0.008712	-0.103369	0.010146	0.004174	0.018355
Gender	0.011037	0.029453	-0.036311	-0.032981	-0.011716
HourlyRate	0.017377	-0.006846	0.024287	0.026528	0.023381
JobInvolvement	-0.003552	-0.130016	0.029820	0.039062	0.046135
JobLevel	-0.009020	-0.169105	0.509604	0.019311	0.002966
JobRole	-0.005007	0.067151	-0.122427	0.002724	-0.009472
JobSatisfaction	-0.022970	-0.103481	-0.004892	-0.033962	0.030571
MaritalStatus	-0.004293	0.162070	-0.095029	0.024001	-0.069586
MonthlyIncome	-0.007188	-0.159840	0.497855	0.034319	0.007707
MonthlyRate	0.006177	0.015170	0.028051	-0.014107	-0.032182
NumCompaniesWorked	-0.000345	0.043494	0.299635	0.020875	0.038153
Over18	NaN	NaN	NaN	NaN	NaN
OverTime	-0.012125	0.246118	0.028062	0.016543	0.009135
PercentSalaryHike	-0.006685	-0.013478	0.003634	-0.029377	0.022704
PerformanceRating	-0.010338	0.002889	0.001904	-0.026341	0.000473
RelationshipSatisfaction	-0.034827	-0.045872	0.053535	-0.035986	0.007846
StandardHours	NaN	NaN	NaN	NaN	NaN
StockOptionLevel	0.031226	-0.137145	0.037510	-0.016727	0.042143
TotalWorkingYears	-0.007047	-0.171063	0.680381	0.034226	0.014515
TrainingTimesLastYear	0.011953	-0.059478	-0.019621	0.015240	0.002453

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate
WorkLifeBalance	0.005370	-0.063939	-0.021490	-0.011256	-0.037848
YearsAtCompany	-0.005779	-0.134392	0.311309	-0.014575	-0.034055
YearsInCurrentRole	-0.004427	-0.160545	0.212901	-0.011497	0.009932
YearsSinceLastPromotion	-0.004575	-0.033019	0.216513	-0.032591	-0.033229
YearsWithCurrManager	-0.004716	-0.156199	0.202089	-0.022636	-0.026363

35 rows × 35 columns

In []: `df.describe()`

Out[]:

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Departme
count	2940.000000	2940.000000	2940.000000	2940.000000	2940.000000	2940.000000
mean	1470.500000	0.161224	36.923810	1.607483	0.501421	1.26054
std	848.849221	0.367800	9.133819	0.665342	0.288791	0.52770
min	1.000000	0.000000	18.000000	0.000000	0.000000	0.00000
25%	735.750000	0.000000	30.000000	1.000000	0.259843	1.00000
50%	1470.500000	0.000000	36.000000	2.000000	0.501074	1.00000
75%	2205.250000	0.000000	43.000000	2.000000	0.755190	2.00000
max	2940.000000	1.000000	60.000000	2.000000	1.000000	2.00000

8 rows × 35 columns



In []: `#resolving scaling issue with normalalization`
`from sklearn.preprocessing import MinMaxScaler`
`norm=MinMaxScaler()`
`df_norm=norm.fit_transform(df)`
`pd.DataFrame(df)`

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFr
0	1	1	41	2	0.715820	2	
1	2	0	49	1	0.126700	1	
2	3	1	37	2	0.909807	1	
3	4	0	33	1	0.923407	1	
4	5	0	27	2	0.350036	1	
...	...	...	...	...	...	...	...
2935	2936	0	36	1	0.559771	1	
2936	2937	0	39	2	0.365784	1	
2937	2938	0	27	2	0.037938	1	
2938	2939	0	49	1	0.659270	2	
2939	2940	0	34	2	0.376521	1	

2940 rows × 35 columns



In []: df_norm

```
Out[ ]: array([[0.00000000e+00, 1.00000000e+00, 5.47619048e-01, ...,
                2.22222222e-01, 0.00000000e+00, 2.94117647e-01],
               [3.40251786e-04, 0.00000000e+00, 7.38095238e-01, ...,
                3.88888889e-01, 6.66666667e-02, 4.11764706e-01],
               [6.80503573e-04, 1.00000000e+00, 4.52380952e-01, ...,
                0.00000000e+00, 0.00000000e+00, 0.00000000e+00],
               ...,
               [9.99319496e-01, 0.00000000e+00, 2.14285714e-01, ...,
                1.11111111e-01, 0.00000000e+00, 1.76470588e-01],
               [9.99659748e-01, 0.00000000e+00, 7.38095238e-01, ...,
                3.33333333e-01, 0.00000000e+00, 4.70588235e-01],
               [1.00000000e+00, 0.00000000e+00, 3.80952381e-01, ...,
                1.66666667e-01, 6.66666667e-02, 1.17647059e-01]])
```

In []: pd.DataFrame(df_norm).describe()

Out[]:

	0	1	2	3	4	5	
count	2940.000000	2940.000000	2940.000000	2940.000000	2940.000000	2940.000000	2940
mean	0.500000	0.161224	0.450567	0.803741	0.501421	0.630272	0
std	0.288822	0.367800	0.217472	0.332671	0.288791	0.263851	0
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0
25%	0.250000	0.000000	0.285714	0.500000	0.259843	0.500000	0
50%	0.500000	0.000000	0.428571	1.000000	0.501074	0.500000	0
75%	0.750000	0.000000	0.595238	1.000000	0.755190	1.000000	0
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1

8 rows × 35 columns



```
In [ ]: from sklearn.svm import SVC
        from sklearn.model_selection import train_test_split
```

```
In [ ]: # splitting data into train and test
        train,test = train_test_split(df, test_size = 0.20)
```

```
In [ ]: print(df.shape)
        print(train.shape)
        print(test.shape)
```

(2940, 35)

(2352, 35)

(588, 35)

```
In [ ]: train = pd.DataFrame(train)
        test = pd.DataFrame(test)
        train.info()
        test.info()
```

<class 'pandas.core.frame.DataFrame'>

Index: 2352 entries, 1791 to 1277

Data columns (total 35 columns):

#	Column	Non-Null Count	Dtype
0	EmployeeNumber	2352 non-null	int64
1	Attrition	2352 non-null	int64
2	Age	2352 non-null	int64
3	BusinessTravel	2352 non-null	int64
4	DailyRate	2352 non-null	float64
5	Department	2352 non-null	int64
6	DistanceFromHome	2352 non-null	int64
7	Education	2352 non-null	float64
8	EducationField	2352 non-null	int64
9	EmployeeCount	2352 non-null	int64
10	EnvironmentSatisfaction	2352 non-null	int64
11	Gender	2352 non-null	int64
12	HourlyRate	2352 non-null	int64
13	JobInvolvement	2352 non-null	int64
14	JobLevel	2352 non-null	int64
15	JobRole	2352 non-null	int64
16	JobSatisfaction	2352 non-null	int64
17	MaritalStatus	2352 non-null	int64
18	MonthlyIncome	2352 non-null	float64
19	MonthlyRate	2352 non-null	int64
20	NumCompaniesWorked	2352 non-null	float64
21	Over18	2352 non-null	int64
22	OverTime	2352 non-null	int64
23	PercentSalaryHike	2352 non-null	float64
24	PerformanceRating	2352 non-null	int64
25	RelationshipSatisfaction	2352 non-null	int64
26	StandardHours	2352 non-null	int64
27	StockOptionLevel	2352 non-null	int64
28	TotalWorkingYears	2352 non-null	float64
29	TrainingTimesLastYear	2352 non-null	float64
30	WorkLifeBalance	2352 non-null	int64
31	YearsAtCompany	2352 non-null	float64
32	YearsInCurrentRole	2352 non-null	float64
33	YearsSinceLastPromotion	2352 non-null	int64
34	YearsWithCurrManager	2352 non-null	float64

dtypes: float64(10), int64(25)

memory usage: 661.5 KB

<class 'pandas.core.frame.DataFrame'>

Index: 588 entries, 2540 to 212

Data columns (total 35 columns):

#	Column	Non-Null Count	Dtype
0	EmployeeNumber	588 non-null	int64
1	Attrition	588 non-null	int64
2	Age	588 non-null	int64
3	BusinessTravel	588 non-null	int64
4	DailyRate	588 non-null	float64
5	Department	588 non-null	int64
6	DistanceFromHome	588 non-null	int64
7	Education	588 non-null	float64
8	EducationField	588 non-null	int64

9	EmployeeCount	588 non-null	int64
10	EnvironmentSatisfaction	588 non-null	int64
11	Gender	588 non-null	int64
12	HourlyRate	588 non-null	int64
13	JobInvolvement	588 non-null	int64
14	JobLevel	588 non-null	int64
15	JobRole	588 non-null	int64
16	JobSatisfaction	588 non-null	int64
17	MaritalStatus	588 non-null	int64
18	MonthlyIncome	588 non-null	float64
19	MonthlyRate	588 non-null	int64
20	NumCompaniesWorked	588 non-null	float64
21	Over18	588 non-null	int64
22	OverTime	588 non-null	int64
23	PercentSalaryHike	588 non-null	float64
24	PerformanceRating	588 non-null	int64
25	RelationshipSatisfaction	588 non-null	int64
26	StandardHours	588 non-null	int64
27	StockOptionLevel	588 non-null	int64
28	TotalWorkingYears	588 non-null	float64
29	TrainingTimesLastYear	588 non-null	float64
30	WorkLifeBalance	588 non-null	int64
31	YearsAtCompany	588 non-null	float64
32	YearsInCurrentRole	588 non-null	float64
33	YearsSinceLastPromotion	588 non-null	int64
34	YearsWithCurrManager	588 non-null	float64

dtypes: float64(10), int64(25)
memory usage: 165.4 KB

```
In [ ]: df.drop(['EmployeeCount', 'Over18', 'StandardHours'], axis=1, inplace=True)
```

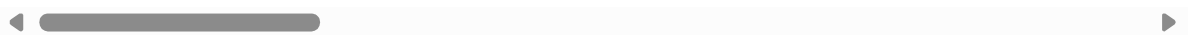
```
In [ ]: # segregating the input and output variable from train and test
train_X = train.iloc[:, 2:]
train_y = train.iloc[:, 1]
test_X = test.iloc[:, 2:]
test_y = test.iloc[:, 1]
```

```
In [ ]: train_X
```

```
Out[ ]:
```

	Age	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	Educati
1791	31	2	0.421618	2	7	0.50	
62	50	2	0.634932	1	7	0.25	
2100	22	2	0.807445	1	1	0.25	
484	31	2	0.302792	2	6	0.75	
2571	32	2	0.516822	1	5	0.25	
...	...	...	...	...	...	...	
1577	25	2	0.098783	2	5	0.50	
2684	44	2	0.586256	1	2	0.50	
153	45	2	0.065855	1	9	0.50	
1189	33	2	0.207588	2	2	0.75	
1277	46	2	0.452398	1	2	0.75	

2352 rows × 33 columns



```
In [ ]: train_y
```

```
Out[ ]: 1791    0
        62     0
        2100   0
        484    0
        2571   0
        ..
        1577    1
        2684    0
        153     0
        1189    0
        1277    0
        Name: Attrition, Length: 2352, dtype: int64
```

```
In [ ]: df.shape
```

```
Out[ ]: (2940, 32)
```

```
In [ ]: # kernel = linear
model_linear = SVC(kernel = "linear")
model_linear.fit(train_X, train_y)
pred_test_linear = model_linear.predict(test_X)

res_linear=np.mean(pred_test_linear == test_y)
```

```
In [ ]: pd.crosstab(pred_test_linear, test_y)
```

Out[]: **Attrition** **0** **1**

row_0			
0	487	79	
1	11	11	

```
In [ ]: # kernel = rbf
model_rbf = SVC(kernel = "rbf")
model_rbf.fit(train_X, train_y)
pred_test_rbf = model_rbf.predict(test_X)
res_rbf=np.mean(pred_test_rbf==test_y)
```

```
In [ ]: # kernel = polynomial
model_poly = SVC(kernel = "poly")
model_poly.fit(train_X, train_y)
pred_test_poly= model_poly.predict(test_X)

res_poly=np.mean(pred_test_poly==test_y)
```

```
In [ ]: # kernel = sigmoid
model_sigmoid = SVC(kernel = "sigmoid")
model_sigmoid.fit(train_X, train_y)
pred_test_sigmoid = model_sigmoid.predict(test_X)

res_sigmoid=np.mean(pred_test_sigmoid==test_y)
```

```
In [ ]: #pd.DataFrame({'Model':['Linear', 'RBF', 'Polynomial', 'Sigmoid'], 'Accuracy':[res_Line
results_df = pd.DataFrame({'Model':['Linear', 'RBF', 'Polynomial', 'Sigmoid'],
                           'Accuracy':[res_linear, res_rbf, res_poly, res_sigmoid]}))
```

```
In [ ]: #print the accuracy results of different kernels
print("Accuracy results for different kernels:")
print(results_df)
```

Accuracy results for different kernels:

	Model	Accuracy
0	Linear	0.846939
1	RBF	0.846939
2	Polynomial	0.846939
3	Sigmoid	0.763605

```
In [ ]: #choose the best model
best_model = results_df.loc[results_df['Accuracy'].idxmax()]
print("\nBest kernel:")
print(best_model)
```

Best kernel:

Model	Linear
Accuracy	0.846939
Name: 0, dtype: object	

***Implementing KNN algorithm ***

```
In [ ]: # converting 1 to Yes and 0 to No
df['Attrition'] = np.where(df['Attrition'] == '1', 'Yes ', df['Attrition'])
df['Attrition'] = np.where(df['Attrition'] == '0', 'No ', df['Attrition'])
```

```
In [ ]: # Normalization function
def norm_func(i):
    x = (i-i.min()) / (i.max()-i.min())
    return (x)
```

```
In [ ]: # Normalized data frame (considering the numerical part of data)
df_n = norm_func(df.iloc[:, 2:])
df_n.describe()
```

```
Out[ ]:
```

	Age	BusinessTravel	DailyRate	Department	DistanceFromHome	Educa'
count	2940.000000	2940.000000	2940.000000	2940.000000	2940.000000	2940.000
mean	0.450567	0.803741	0.501421	0.630272	0.292590	0.478
std	0.217472	0.332671	0.288791	0.263851	0.289482	0.255
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000
25%	0.285714	0.500000	0.259843	0.500000	0.035714	0.250
50%	0.428571	1.000000	0.501074	0.500000	0.214286	0.500
75%	0.595238	1.000000	0.755190	1.000000	0.464286	0.750
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000

8 rows × 30 columns



```
In [ ]: X = np.array(df_n.iloc[:,2:]) # Predictors
Y = np.array(df['Attrition']) # Target
```

```
In [ ]: X.shape
```

```
Out[ ]: (2940, 28)
```

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2)
```

```
In [ ]: from sklearn.neighbors import KNeighborsClassifier
```

```
In [ ]: from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors = 29)
knn.fit(X_train, Y_train)
```

```
pred = knn.predict(X_test)
pred
```

[illegible]

```
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ',
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ',
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ',
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ',
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ',
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ',
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'Yes ',
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ',
'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ', 'No ',
'No ', 'No ', 'No '], dtype=object)
```

```
In [ ]: # Evaluate the model
from sklearn.metrics import accuracy_score
print(accuracy_score(Y_test, pred))
pd.crosstab(Y_test, pred, rownames = ['Actual'], colnames= ['Predictions'])
```

0.8469387755102041

Out[]: **Predictions** **No** **Yes**

Actual		
No	488	0
Yes	90	10

```
In [ ]: # error on train data
pred_train = knn.predict(X_train)
print(accuracy_score(Y_train, pred_train))
pd.crosstab(Y_train, pred_train, rownames=['Actual'], colnames = ['Predictions'])
```

0.8537414965986394

Out[]: **Predictions** **No** **Yes**

Actual		
No	1972	6
Yes	338	36

```
In [ ]: # creating empty list variable
acc = []

# running KNN algorithm for 3 to 50 nearest neighbours(odd numbers) and
# storing the accuracy values

for i in range(3,50,2):
    neigh = KNeighborsClassifier(n_neighbors=i)
    neigh.fit(X_train, Y_train)
    train_acc = np.mean(neigh.predict(X_train) == Y_train)
    test_acc = np.mean(neigh.predict(X_test) == Y_test)
    Difference=train_acc-test_acc
    acc.append([i,train_acc, test_acc,Difference])
```

```
In [ ]: df_1=pd.DataFrame(acc)
```

```
In [ ]: df_1.columns='K','train_acc','test_acc','difference'
```

```
In [ ]: df_1
```

```
Out[ ]:
```

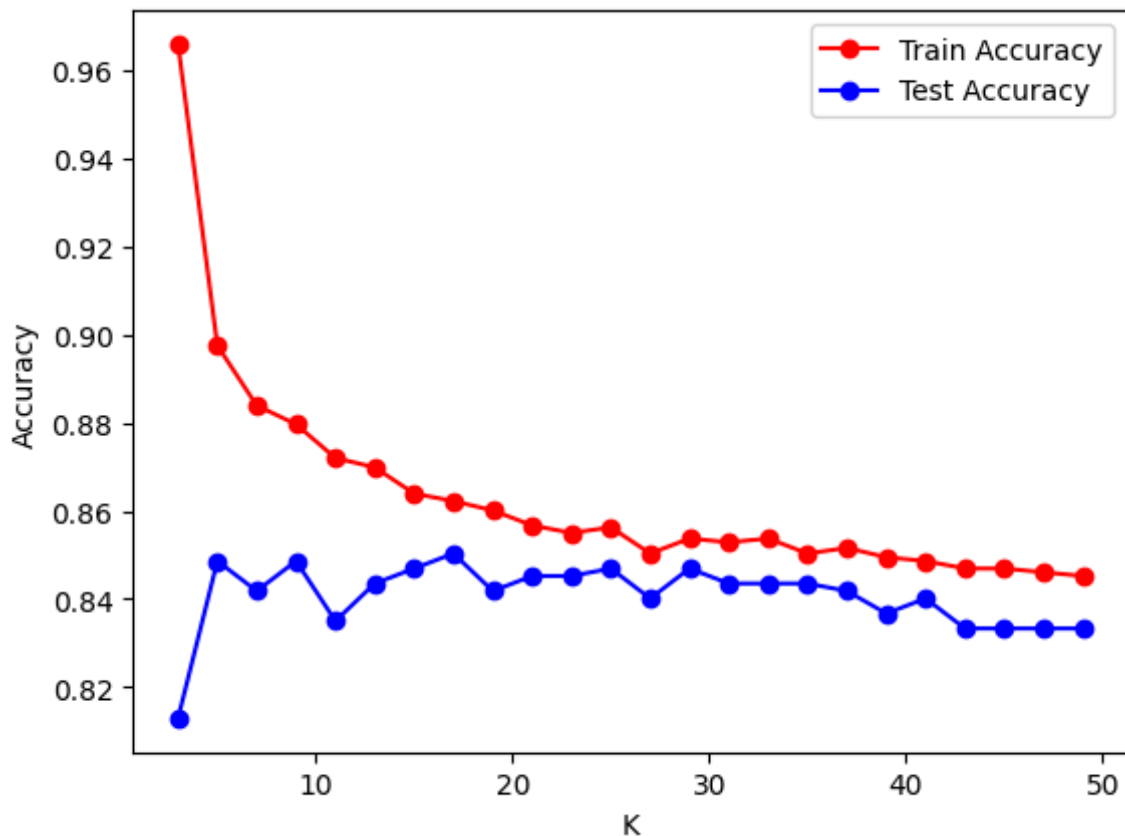
	K	train_acc	test_acc	difference
0	3	0.965986	0.812925	0.153061
1	5	0.897534	0.848639	0.048895
2	7	0.883929	0.841837	0.042092
3	9	0.879677	0.848639	0.031037
4	11	0.872024	0.835034	0.036990
5	13	0.869898	0.843537	0.026361
6	15	0.863946	0.846939	0.017007
7	17	0.862245	0.850340	0.011905
8	19	0.860119	0.841837	0.018282
9	21	0.856718	0.845238	0.011480
10	23	0.855017	0.845238	0.009779
11	25	0.856293	0.846939	0.009354
12	27	0.850340	0.840136	0.010204
13	29	0.853741	0.846939	0.006803
14	31	0.852891	0.843537	0.009354
15	33	0.853741	0.843537	0.010204
16	35	0.850340	0.843537	0.006803
17	37	0.851616	0.841837	0.009779
18	39	0.849490	0.836735	0.012755
19	41	0.848639	0.840136	0.008503
20	43	0.846939	0.833333	0.013605
21	45	0.846939	0.833333	0.013605
22	47	0.846088	0.833333	0.012755
23	49	0.845238	0.833333	0.011905

29NN and 37NN have least difference but with respect to higher test accuracy 29NN is the best K for the given dataset

```
In [ ]: # train accuracy plot
plt.plot(np.arange(3,50,2),[i[1] for i in acc],"ro-")

# test accuracy plot
plt.plot(np.arange(3,50,2),[i[2] for i in acc],"bo-")
plt.xlabel('K')
plt.ylabel('Accuracy')
plt.legend(['Train Accuracy','Test Accuracy'])
```

Out[]: <matplotlib.legend.Legend at 0x7955f985f190>



DECISION TREE

```
In [ ]: # Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
X_train, Y_train, X_test, Y_test = train_test_split(X, Y, test_size=0.3, random_state=0)
print(X.shape, Y.shape, X_train.shape, Y_train.shape, X_test.shape, Y_test.shape)

#handling imbalance in matrix columns and rows with SMOTE
smote = SMOTE(random_state=0)
X_train_smote, Y_train_smote = smote.fit_resample(X_train, Y_train)
```

(2940, 28) (2940,) (2058, 28) (2058,) (882, 28) (882,)

```
In [ ]: #feature Scaling
from sklearn.preprocessing import StandardScaler
st_X = StandardScaler()
```

```
X_train_smote= st_X.fit_transform(X_train_smote)
X_test=st_X.transform(X_test)
```

```
In [ ]: #Fitting Decision Tree classifier to the training set
        from sklearn.tree import DecisionTreeClassifier
        classifier= DecisionTreeClassifier(criterion='entropy', random_state=0,max_depth=5,
        print(classifier.fit(X_train_smote, Y_train_smote))
```

```
DecisionTreeClassifier(criterion='entropy', max_depth=5, random_state=0)
```

```
In [ ]: #Predicting the test set result
        Y_pred= classifier.predict(X_test)
        print(Y_test,Y_pred)
```

[illegible]

[illegible]

```
'Yes ' 'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'No ' 'Yes ' 'Yes ' 'Yes '
'No ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'No ' 'No ' 'No ' 'Yes '
'No ' 'No ' 'Yes ' 'No ' 'No ' 'Yes ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes '
'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes '
'Yes ' 'No ' 'Yes ' 'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes '
'Yes ' 'Yes ' 'Yes ' 'Yes ' 'No ' 'No ' 'Yes ' 'No ' 'Yes ' 'Yes ' 'No '
'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'No '
'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'No ' 'Yes ' 'Yes ' 'Yes '
'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'No ' 'Yes ' 'No ' 'Yes ' 'Yes ' 'No '
'Yes ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'No ' 'No ' 'Yes ' 'Yes '
'Yes ' 'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'No ' 'Yes ' 'Yes '
'No ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'No ' 'Yes ' 'Yes ' 'No ' 'Yes '
'Yes ' 'No ' 'No ' 'Yes ' 'No ' 'No ' 'Yes ' 'No ' 'No ' 'Yes ' 'No '
'Yes ' 'Yes ' 'No ' 'No ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'No '
'No ' 'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'No ' 'No ' 'Yes ' 'No ' 'Yes '
'Yes ' 'Yes ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes '
'No ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes ' 'Yes '
'Yes ' 'Yes ' 'No ' 'Yes ' 'Yes ' 'Yes ' 'No ' 'No ' 'Yes ' 'Yes ' 'Yes '
'No ' 'No ' 'Yes ' 'Yes ' 'Yes ']
```

```
In [ ]: Y1_pred= classifier.predict(X_train_smote)
print(Y_train,Y1_pred)
```

```
['No ' 'Yes ' 'No ' ... 'No ' 'No ' 'Yes ' ] ['No ' 'No ' 'No ' ... 'Yes ' 'Yes ' 'Ye
s ']
```

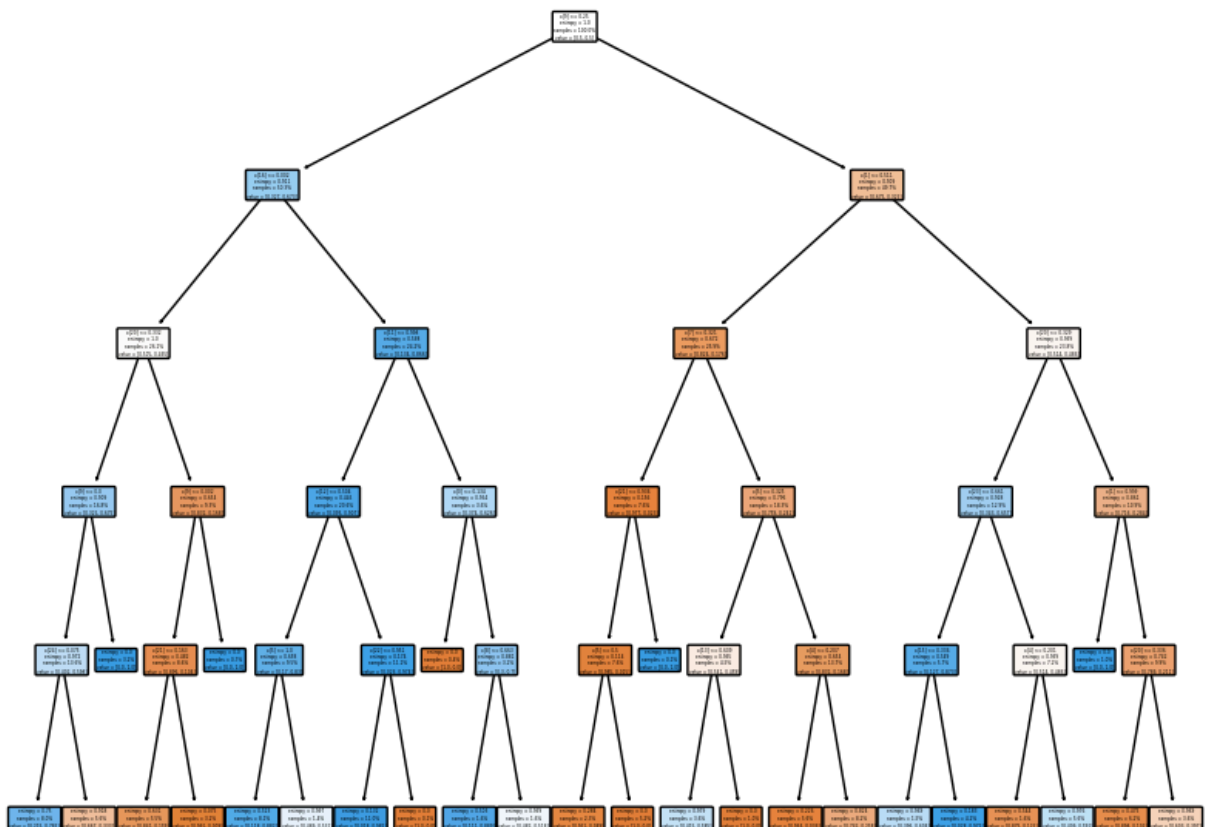
```
In [ ]: Y_test.shape
```

```
Out[ ]: (882,)
```

```
In [ ]: Y_pred.shape
```

```
Out[ ]: (588,)
```

```
In [ ]: from sklearn import tree
import matplotlib.pyplot as plt
plt.figure(figsize=(10,8))
tree.plot_tree(classifier,rounded=True,filled=True,proportion=True)
plt.show()
```



KMEANS CLUSTERING

```
In [ ]: from sklearn.cluster import KMeans
```

```
In [ ]: ##### scree plot or elbow curve #####
TWSS = []
k = list(range(2, 20))

for i in k:
    kmeans = KMeans(n_clusters = i)
    kmeans.fit(df_norm)
    TWSS.append(kmeans.inertia_)
```

[illegible]

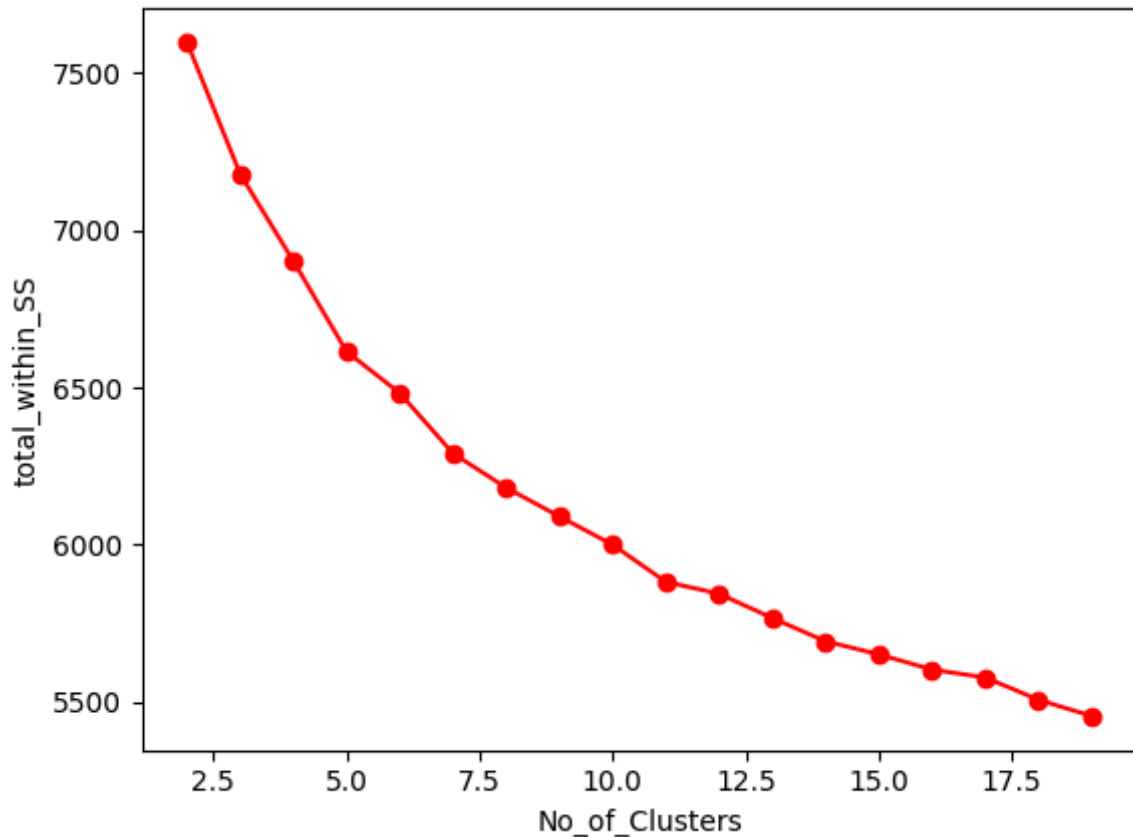
```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarnin
g: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the value
of `n_init` explicitly to suppress the warning
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarnin
g: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the value
of `n_init` explicitly to suppress the warning
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarnin
g: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the value
of `n_init` explicitly to suppress the warning
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarnin
g: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the value
of `n_init` explicitly to suppress the warning
  warnings.warn(
```

```
In [ ]: TWSS
```

```
Out[ ]: [7600.219499957431,
        7179.834468653447,
        6902.313530425628,
        6614.245650786479,
        6481.713978822318,
        6289.961132802255,
        6180.638822805658,
        6089.095993116785,
        5999.062396301318,
        5880.83314927286,
        5843.04524651471,
        5765.62230153967,
        5691.733584421892,
        5650.072221290366,
        5601.756890128,
        5576.433019200591,
        5505.41202531304,
        5453.768532114849]
```

```
In [ ]: import matplotlib.pyplot as plt
        # Scree plot
        plt.plot(k, TWSS, 'ro-');
        plt.xlabel("No_of_Clusters");
        plt.ylabel("total_within_SS")
```

```
Out[ ]: Text(0, 0.5, 'total_within_SS')
```



```
In [ ]: # Selecting 5 clusters from the above scree plot which is the optimum number of clu
model = KMeans(n_clusters = 5)
model.fit(df_n)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarnin
g: The default value of `n_init` will change from 10 to 'auto' in 1.4. Set the value
of `n_init` explicitly to suppress the warning
warnings.warn(
```

```
Out[ ]: ▼      KMeans
KMeans(n_clusters=5)
```

```
In [ ]: model.labels_
```

```
Out[ ]: array([1, 0, 1, ..., 0, 3, 3], dtype=int32)
```

```
In [ ]: mb = pd.Series(model.labels_) # converting numpy array into pandas series object
df['Clust'] = mb
```

```
In [ ]: df.head()
```

```
Out[ ]:
```

	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFromH
0	1	Yes	41	2	0.715820	2	
1	2	No	49	1	0.126700	1	
2	3	Yes	37	2	0.909807	1	
3	4	No	33	1	0.923407	1	
4	5	No	27	2	0.350036	1	

5 rows × 33 columns



```
In [ ]: df.shape
```

```
Out[ ]: (2940, 33)
```

```
In [ ]: last_column = df.columns[-1]
reordered_columns = [last_column] + list(df.columns[:-1])
df = df[reordered_columns]
df.head()
```

```
Out[ ]:
```

	Clust	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	DistanceFromH
0	1	1	Yes	41	2	0.715820	2	
1	0	2	No	49	1	0.126700	1	
2	1	3	Yes	37	2	0.909807	1	
3	1	4	No	33	1	0.923407	1	
4	3	5	No	27	2	0.350036	1	

5 rows × 33 columns



```
In [ ]: df.iloc[:, 3:].groupby(df.Clust).mean()
```

```
Out[ ]:      Age  BusinessTravel  DailyRate  Department  DistanceFromHome  Education  E
```

	Age	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	E
Clust							
0	36.380734	1.582569	0.501448	1.220183	9.756881	0.458716	
1	35.585526	1.625000	0.511430	1.273026	9.674342	0.461349	
2	48.187135	1.719298	0.499994	1.216374	8.508772	0.519006	
3	34.670860	1.566038	0.485060	1.278826	9.163522	0.468553	
4	35.836667	1.610000	0.518089	1.273333	8.730000	0.501667	

5 rows × 30 columns

```
In [ ]: df.to_excel("Kmeans_HR_Attrition.xlsx")
```

```
In [ ]: #By implementing various models i have come to the conclusion that SVM for Linear a
```

IMPLEMENTING ARTIFICIAL NEURAL NETWORK

```
In [ ]: pip install keras.utils
```

```
Collecting keras.utils
  Using cached keras_utils-1.0.13-py3-none-any.whl
Requirement already satisfied: Keras>=2.1.5 in /usr/local/lib/python3.10/dist-packages (from keras.utils) (2.15.0)
Installing collected packages: keras.utils
Successfully installed keras.utils-1.0.13
```

```
In [ ]: from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.neural_network import MLPClassifier
from keras.utils import np_utils
from sklearn.model_selection import train_test_split
import tensorflow as tf
```

```
In [ ]: df.head()
```

```
Out[ ]:
```

	Clust	EmployeeNumber	Attrition	Age	BusinessTravel	DailyRate	Department	Distance
0	1	1	Yes	41	2	0.715820	2	
1	0	2	No	49	1	0.126700	1	
2	1	3	Yes	37	2	0.909807	1	
3	1	4	No	33	1	0.923407	1	
4	3	5	No	27	2	0.350036	1	

5 rows × 33 columns

```
In [ ]: # splitting the dataset
X=df_norm[:,2:]
Y=df_norm[:, 1]
print(X.shape,Y.shape)
```

(2940, 33) (2940,)

```
In [ ]: # splitting the data into train and test( 80:20)
X_train, X_test, Y_train, Y_test = train_test_split(X,Y,test_size=0.2, random_state
X_train
```

```
Out[ ]: array([[0.30952381, 1.          , 0.41947029, ..., 0.22222222, 0.          ,
0.05882353],
[0.38095238, 1.          , 0.11166786, ..., 0.38888889, 0.33333333,
0.41176471],
[0.4047619 , 1.          , 0.5583393 , ..., 0.5          , 0.          ,
0.47058824],
...,
[0.4047619 , 1.          , 0.46385111, ..., 0.5          , 0.4          ,
0.47058824],
[0.54761905, 1.          , 0.24695777, ..., 0.11111111, 0.06666667,
0.11764706],
[0.0952381 , 0.5        , 0.82605583, ..., 0.          , 0.          ,
0.          ]])
```

```
In [ ]: Y_train
```

```
Out[ ]: array([0., 0., 0., ..., 0., 0., 1.])
```

```
In [ ]: Y_train.shape
X_train.shape
```

```
Out[ ]: (2352, 33)
```

```
In [ ]: X_train
```

```
Out[ ]: array([[0.30952381, 1.          , 0.41947029, ..., 0.22222222, 0.          ,
                0.05882353],
               [0.38095238, 1.          , 0.11166786, ..., 0.38888889, 0.33333333,
                0.41176471],
               [0.4047619 , 1.          , 0.5583393 , ..., 0.5          , 0.          ,
                0.47058824],
               ...,
               [0.4047619 , 1.          , 0.46385111, ..., 0.5          , 0.4          ,
                0.47058824],
               [0.54761905, 1.          , 0.24695777, ..., 0.11111111, 0.06666667,
                0.11764706],
               [0.0952381 , 0.5        , 0.82605583, ..., 0.          , 0.          ,
                0.          ]])
```

```
In [ ]: # specifying and compiling the model
def design_mlp():
    model = Sequential()
    model.add(Dense(150,input_shape =(None,2352, 33),activation="relu"))
    model.add(Dense(200,activation="tanh"))
    model.add(Dense(100,activation="tanh"))
    model.add(Dense(1000,activation="tanh"))
    model.add(Dense(1,activation="relu"))
    model.compile(loss="mean_squared_error",optimizer="adam",metrics=['mean_squared
    return model
```

```
In [ ]: model = design_mlp()
```

```
In [ ]: # fitting the model
model.fit(X_train,Y_train,epochs=100, batch_size=8)
```

Epoch 1/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 2/100
294/294 [=====] - 1s 5ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 3/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 4/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 5/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 6/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 7/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 8/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 9/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 10/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 11/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 12/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 13/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 14/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 15/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 16/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 17/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 18/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 19/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594

_error: 0.1594
Epoch 20/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 21/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 22/100
294/294 [=====] - 1s 5ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 23/100
294/294 [=====] - 2s 8ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 24/100
294/294 [=====] - 2s 6ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 25/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 26/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 27/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 28/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 29/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 30/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 31/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 32/100
294/294 [=====] - 2s 6ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 33/100
294/294 [=====] - 2s 8ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 34/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 35/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 36/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 37/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 38/100

294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 39/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 40/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 41/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 42/100
294/294 [=====] - 2s 6ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 43/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 44/100
294/294 [=====] - 2s 6ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 45/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 46/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 47/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 48/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 49/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 50/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 51/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 52/100
294/294 [=====] - 2s 5ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 53/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 54/100
294/294 [=====] - 2s 8ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 55/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 56/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594

Epoch 57/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 58/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 59/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 60/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 61/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 62/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 63/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 64/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 65/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 66/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 67/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 68/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 69/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 70/100
294/294 [=====] - 1s 5ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 71/100
294/294 [=====] - 2s 8ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 72/100
294/294 [=====] - 1s 5ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 73/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 74/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594
Epoch 75/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared_error: 0.1594

_error: 0.1594
Epoch 76/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 77/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 78/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 79/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 80/100
294/294 [=====] - 2s 6ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 81/100
294/294 [=====] - 2s 8ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 82/100
294/294 [=====] - 1s 5ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 83/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 84/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 85/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 86/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 87/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 88/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 89/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 90/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 91/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 92/100
294/294 [=====] - 1s 5ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 93/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 94/100

```

294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 95/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 96/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 97/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 98/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 99/100
294/294 [=====] - 1s 4ms/step - loss: 0.1594 - mean_squared
_error: 0.1594
Epoch 100/100
294/294 [=====] - 2s 7ms/step - loss: 0.1594 - mean_squared
_error: 0.1594

```

Out[]: <keras.src.callbacks.History at 0x79559dab57e0>

```

In [ ]: # Evaluating the model on test data
eval_score_test = model.evaluate(X_test,Y_test,verbose=1)
accuracy=(1-eval_score_test[1])*100
accuracy

```

```

19/19 [=====] - 0s 6ms/step - loss: 0.1684 - mean_squared_e
rror: 0.1684

```

Out[]: 83.16326588392258

```

In [ ]: # Evaluating the model on test data
eval_score_train = model.evaluate(X_train,Y_train,verbose=1)
accuracy=(1-eval_score_train[1])*100
accuracy

```

```

74/74 [=====] - 0s 5ms/step - loss: 0.1594 - mean_squared_e
rror: 0.1594

```

Out[]: 84.05612260103226

10th July Completed ANNMLP on the HR_Attrition dataset. test data accuracy is lower than training data accuracy so the model is not overfitted and neither underfitted

Documentation section

Dataset preprocessing:

The given dataset data set Data set - HR Employee Attrition.csv has been thoroughly scrutinized for missing values and the data set has been cleaned by techniques to combat issues such as

- 1.Dealing with Outliers
- 2.Standardization and Normalization
- 3.Handling Duplicate Data
- 4.Addressing Inconsistent Data
- 5.Data Transformation
- 6.Handling Categorical Data

Data transformations

The categorical data in the data set have been converted to numerical data by one-hot encoding technique or label encoding for effective EDA(Exploratory data analysis)

The data was normalized to fix the scaling issue for further processing using normalization function for specific EDA analysis and minmaxscaler for other purposes as well

Exploratory data analysis:

EDA has been performed across various plots and graphs signifying the importance of valuable features and how they affect the target decision. Comments regarding each graphical analysis is highlighted in the comments under each cell.

Machine learning models

various machine learning algorithms as SVM,KNN along with a few classifying and clustering techniques such as decision tree and Kmeans have been utilized to make the best use of the data provided to us to design an optimal model

Accuracy results:

I. ML models:

Best accuracy obtained from the chosen models:

Best kernel: SVM

Model-Linear

Accuracy-0.846939

KNN K=29

Accuracy - 0.846938

II.ANNMLP:

Artificial neural network has been constructed with the help of libraries such as tensorflow using keras

Accuracy results after running 100 epochs with batch size of 8:

test data accuracy - 83.16

training data accuracy - 84.05

